

Verified Calculus

From the Natural Numbers to Symbolic Solvers in Lean 4

A zero-to-hero journey

*through real analysis
with the number systems built from scratch in Lean 4*

Core Lean first, Mathlib after: build it once by hand, then wield the industrial version.

Lean 4 · Real Analysis · Symbolic Computation

Preface

This book rebuilds calculus from a single inductive type in Lean 4 to a working command-line symbolic solver. It proceeds in two deliberate phases.

Phase one (Part I): core Lean, no libraries. No external mathematics libraries are imported. Every definition—addition on the natural numbers, the integers and rationals as quotients, the real numbers as a Cauchy completion—is constructed explicitly and its properties proved by hand. The point of this phase is that you should see, once in your life, that there is no magic underneath: the real numbers can be built, keystroke by keystroke, from `zero` and `succ`.

Phase two (Parts II–VII): Mathlib. From Chapter 6 onward the book switches—openly, and by design—to **Mathlib**, the Lean community’s mathematical library. Having built $\mathbb{R}$ once yourself, you will use the industrial-strength version of it (Mathlib’s `Real`) and Mathlib’s proof tactics (`linarith`, `norm_num`, `positivity`, ...) for the analysis that follows. This mirrors real practice: nobody re-proves commutativity of addition before doing analysis, but everybody is better at analysis for having done it once. Chapter 6 marks the switch explicitly and shows how to set up a Lean project that depends on Mathlib. The capstone solver of Part VII is again plain core Lean (it computes over `Float`), except for its correctness theorems in Chapter 36, which use Mathlib’s analysis library.

Every Lean listing in this book is machine-checked: the Part I and capstone listings against a bare Lean toolchain (`CalculusVerification.lean`), the Mathlib-based listings against Mathlib (`CalculusMathlibVerification.lean`). Where a listing deliberately leaves a proof as `sorry`, the surrounding text says so and marks it as an exercise.

Prerequisites. Familiarity with proof-based mathematics (induction, contradiction, epsilon-delta arguments seen before). No prior Lean 4 experience required: Appendix A is a primer on every Lean construct and tactic used in this book, including what several of them desugar to under the hood.

Structure.

- **Part I** constructs $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, and $\mathbb{R}$ via Cauchy completion (core Lean, no libraries).
- **Part II** develops the topology of $\mathbb{R}$: sequences, limits, continuity (from here on, over Mathlib’s `Real`).
- **Part III** covers differential calculus through Taylor’s theorem.
- **Part IV** covers integral calculus and the Fundamental Theorem.
- **Part V** treats infinite series and power series.
- **Part VI** extends to $\mathbb{R}^n$: multivariable calculus and vector analysis.

- **Part VII** is the capstone: *Symboliculus*, a step-by-step symbolic calculus solver as a runnable Lean 4 CLI.

Contents

Preface	i
I Building the Real Numbers from Scratch	1
1 The Natural Numbers	2
1.1 The Peano Axioms	3
1.2 Arithmetic by Recursion	5
1.3 The Order on $\mathbb{N}$	7
2 The Integers	13
2.1 Constructing $\mathbb{Z}$	13
2.2 Ring Axioms for $\mathbb{Z}$	15
3 The Rationals	17
3.1 Constructing $\mathbb{Q}$	17
4 Why $\mathbb{Q}$ Is Not Enough	20
4.1 The Irrationality of $\sqrt{2}$	20
4.2 Cauchy Sequences Without Rational Limits	21
5 The Real Numbers	24
5.1 Cauchy Completion of $\mathbb{Q}$	25
5.2 Completeness and Key Properties of $\mathbb{R}$	28
II Topology of $\mathbb{R}$ and Limits	31
6 Sequences and Convergence	32
6.1 From <code>MyReal</code> to Mathlib	33
6.2 Convergence of Sequences	34
6.3 Algebra of Limits	35
6.4 Important Limit Theorems	36
7 Limits of Functions	38
7.1 The ε - δ Definition	38
7.2 One-sided Limits and Limits at Infinity	39
7.3 Algebra of Functional Limits	40

8	Continuity	41
8.1	Continuous Functions	42
8.2	Properties of Continuous Functions	43
8.3	The Intermediate Value Theorem	43
8.4	Uniform Continuity	44
8.5	Pathological Examples	44
9	Topology of $\mathbb{R}$	46
9.1	Open and Closed Sets	46
9.2	Compactness	47
9.3	Connectedness	47
III	Differential Calculus	49
10	The Derivative	50
10.1	The Difference Quotient and the Derivative	51
10.2	Derivatives of Elementary Functions	52
11	Differentiation Rules	55
11.1	The Four Fundamental Rules	55
11.2	Implicit and Logarithmic Differentiation	56
12	Mean Value Theorems	58
12.1	Rolle's Theorem and the MVT	58
12.2	L'Hôpital's Rule	59
13	Higher Derivatives and Taylor's Theorem	60
13.1	Higher-Order Derivatives	61
13.2	Taylor's Theorem	61
14	Applications of Differentiation	63
14.1	Optimization	63
14.2	Newton's Method	64
IV	Integral Calculus	66
15	The Riemann Integral	67
15.1	Partitions and Riemann Sums	68
15.2	Properties of the Integral	70
16	The Fundamental Theorem of Calculus	72
16.1	FTC Part 1: The Integral as an Antiderivative	73
16.2	FTC Part 2: The Evaluation Theorem	73

17 Techniques of Integration	75
17.1 Substitution (Change of Variables)	75
17.2 Integration by Parts	75
17.3 Partial Fraction Decomposition	76
17.4 Trigonometric Substitutions	76
17.5 Trigonometric Integrals	76
18 Improper Integrals	78
V Sequences and Series	80
19 Infinite Series	81
19.1 Convergence Tests	82
20 Power Series	84
21 Taylor and Maclaurin Series	86
VI Multivariable Calculus	88
22 $\mathbb{R}^n$ and Its Topology	89
22.1 Vectors in $\mathbb{R}^n$	89
22.2 Open Sets, Convergence, and Compactness in $\mathbb{R}^n$	90
23 Partial Derivatives and Differentiability	92
23.1 Partial Derivatives	92
23.2 Total Differentiability	92
24 The Gradient and Directional Derivatives	94
25 Higher-Order Partial Derivatives	96
26 Multivariable Optimization	98
26.1 Critical Points and the Second Derivative Test	98
26.2 Lagrange Multipliers	98
27 Multiple Integrals	100
27.1 Double Integrals and Iterated Integration	100
27.2 Change of Variables in $\mathbb{R}^n$	101
28 Vector Fields and Line Integrals	102
29 Surface Integrals	104
30 The Three Great Theorems	105
30.1 Green's Theorem	106
30.2 Stokes' Theorem	106

30.3	The Divergence Theorem	107
30.4	Unification: Generalised Stokes' Theorem	107
VII	The Capstone: <i>Symboliculus</i>	109
31	Expression Trees	111
31.1	The <code>Expr</code> Inductive Type	111
31.2	Evaluation	111
31.3	Pretty-Printing	112
31.4	Parsing	113
31.5	Algebraic Simplification	114
32	The Differentiation Engine	116
32.1	The <code>DiffStep</code> Trace Type	116
32.2	The <code>diff</code> Function	116
32.3	Rendering the Trace	119
33	The Integration Engine	120
33.1	Strategies and the <code>IntStep</code> Trace	120
33.2	Direct Lookup Table	121
33.3	U-Substitution Detector	122
33.4	Integration by Parts Heuristic	123
33.5	Top-Level <code>integrate</code> Dispatcher	125
34	Step-by-Step Trace Rendering	127
34.1	The <code>SolveTrace</code> Rose Tree	127
34.2	Rendering to Text	128
35	The CLI	130
35.1	Lake Project Structure	130
35.2	<code>Main.lean</code> — Argument Parsing and Entry Point	131
35.3	Building and Running	132
36	Correctness, Incompleteness, and What Comes Next	134
36.1	What We Can Prove: Correctness of Differentiation	134
36.2	What We Cannot Decide: The Risch Algorithm	137
36.3	Connections to Neighbouring Fields	138
36.4	Open Problems and Extensions	139
A	A Lean 4 Primer	140
A.1	Declarations	140
A.2	Core Tactics	142
A.3	Quotients: <code>Quot</code> , <code>Quot.mk</code> , <code>Quot.lift</code> , <code>Quot.sound</code>	146
A.4	Mathlib Tactics (Parts II–VII)	147

Part I

Building the Real Numbers from Scratch

Chapter 1

The Natural Numbers

Every edifice needs a foundation. For mathematics, that foundation is the natural numbers. Rather than treating them as self-evident, we ask what they *are*, and answer with five axioms due to Giuseppe Peano.

Remark

Historical Note: What Are the Natural Numbers?

For most of human history, natural numbers were treated as primitives — self-evident objects that needed no definition. The ancient Greeks distinguished between *arithmetic* (the study of number) and *logistic* (calculation), but never asked what a number fundamentally *is*. This question only became pressing in the nineteenth century, when mathematicians began demanding rigorous foundations for analysis.

The crisis was triggered by analysis itself. Cauchy, Bolzano, and Weierstrass had built calculus on a foundation of limits and convergence, but their definitions ultimately rested on an intuitive notion of “real number” that nobody had made precise. To define $\mathbb{R}$ rigorously, you first need $\mathbb{Q}$; to define $\mathbb{Q}$ you need $\mathbb{Z}$; to define $\mathbb{Z}$ you need $\mathbb{N}$. And $\mathbb{N}$, it turned out, was the hardest of all. Several mathematicians tackled the problem simultaneously in the 1870s–1880s. **Richard Dedekind** (1872) defined the integers and rationals via equivalence classes — the same construction we use in Chapters 2 and 3 — and proposed defining $\mathbb{N}$ itself via what he called “simply infinite systems,” an abstract structure satisfying conditions equivalent to Peano’s axioms. **Gottlob Frege** (1884) attempted to define natural numbers as *logical objects*: the number 2 is the class of all two-element sets. Frege’s program was far more ambitious — he wanted to show arithmetic was pure logic, with no mathematical primitives at all (a position called **logicism**). **Giuseppe Peano** (1889) took the more pragmatic approach we follow here: simply axiomatise $\mathbb{N}$ directly, accepting 0 and S as primitives, and derive everything else.

The logicist program collapsed when Bertrand Russell sent Frege a letter in 1902 pointing out that the unrestricted comprehension principle underlying Frege’s set theory was contradictory — this is **Russell’s paradox**: the set of all sets that do not contain themselves can neither contain itself nor not contain itself. Frege’s life’s work was destroyed. Russell and Whitehead subsequently

rebuilt logicism on a more careful type-theoretic foundation in *Principia Mathematica* (1910–1913), requiring hundreds of pages to prove that $1 + 1 = 2$. Peano’s axiomatic approach, by contrast, proved robust. Combined with Zermelo–Fraenkel set theory (1908–1922), it became the standard foundation for mathematics. The five axioms we state below are the distillation of this century of foundational work.

1.1 The Peano Axioms

Definition: The Peano Axioms

The **natural numbers** are a set $\mathbb{N}$ with a distinguished element $0 \in \mathbb{N}$ and a function $S : \mathbb{N} \rightarrow \mathbb{N}$ (**successor**) satisfying:

- P1.** $0 \in \mathbb{N}$.
- P2.** If $n \in \mathbb{N}$ then $S(n) \in \mathbb{N}$.
- P3.** S is injective: $S(m) = S(n) \Rightarrow m = n$.
- P4.** 0 is not a successor: $S(n) \neq 0$ for all n .
- P5. Induction:** If $P(0)$ holds and $P(n) \Rightarrow P(S(n))$, then $P(n)$ holds for all n .

We write $1 := S(0)$, $2 := S(1)$, $3 := S(2)$, etc. Axiom P5 ensures no “extra” elements exist beyond what is reachable from 0 by applying S .



no predecessor (P4)

injective (P3)

It is worth pausing on what makes these five axioms so powerful. P1 and P2 say the natural numbers are closed under S . P3 says S never collapses two different numbers into one — every natural number has a unique predecessor. P4 says 0 is genuinely the starting point, not in the middle of some cycle. Without P3 and P4 together, we could have a “number system” like $\{0, 1, 2\}$ where $S(2) = 0$, which satisfies P1, P2, and P5 but is not the natural numbers. P5 — the induction axiom — is the one that rules out such finite imposters by insisting every element is reachable from 0 .

In Lean 4, an inductive type with two constructors captures exactly these axioms: `zero` corresponds to 0 , and `succ` corresponds to S . Crucially, the Lean kernel enforces P3 and P4 automatically: distinct constructors are always disjoint (`zero ≠ succ n`), and constructors are always injective (`succ m = succ n ⇒ m = n`). P5 is the

elimination principle generated by the `inductive` declaration — pattern matching on a `MyNat` is literally induction.

If the listing below is your first encounter with Lean 4, do not worry about the syntax: Appendix A (*A Lean 4 Primer*) explains every keyword and tactic this book uses — `inductive`, `def`, `theorem`, `induction with`, `simp`, `exact`, and the rest — with small worked examples and, where illuminating, the lower-level term each tactic expands to. Read it alongside this chapter, or dip into it whenever a new construct appears.

Lean 4

```
-- Our own natural numbers, no imports needed
inductive MyNat : Type where
  | zero : MyNat
  | succ : MyNat → MyNat
  deriving Repr

-- Naming the first few
def MyNat.one   : MyNat := MyNat.succ MyNat.zero
def MyNat.two   : MyNat := MyNat.succ MyNat.one
def MyNat.three : MyNat := MyNat.succ MyNat.two

-- The induction principle is generated automatically.
-- Pattern matching IS structural induction in Lean.
theorem mynat_ind (P : MyNat → Prop)
  (h0 : P MyNat.zero)
  (hs : ∀ n, P n → P (MyNat.succ n)) :
  ∀ n, P n := fun n => by
    induction n with
    | zero      => exact h0
    | succ n ih => exact hs n ih
```

Remark

Kronecker, Brouwer, and the Constructivist Challenge

Leopold Kronecker famously declared in 1886: “God made the integers; all else is the work of man” (*Die ganzen Zahlen hat der liebe Gott gemacht, alles andere ist Menschenwerk*). He meant this as a compliment to $\mathbb{Z}$ but a criticism of the irrational numbers, infinite sets, and non-constructive proofs that were proliferating in analysis. For Kronecker, a mathematical object had to be *explicitly constructible* from the integers in finitely many steps; anything else was metaphysics.

This position was radicalised by **L.E.J. Brouwer** (1907–) into **intuitionism**: mathematics is a mental activity, and a mathematical object exists only when it can be *mentally constructed*. On this view, the law of excluded middle ($P \vee \neg P$) is not automatically valid, because there may be propositions P for which we can neither construct a proof of P nor a proof of $\neg P$. A proof of $\neg(\neg P)$ (not-not- P) is *not* the same as a proof of P , because it merely rules

out a disproof without providing a construction.

Lean 4 is, at its core, a **dependent type theory** based on the Calculus of Inductive Constructions, which is broadly constructive: proofs in Lean are *programs*, and every proof of an existential $\exists x, P(x)$ must exhibit a specific witness. However, Lean also provides `Classical.choice`, which non-constructively selects a witness from any nonempty type, enabling the full law of excluded middle. The theorems in this book use classical logic freely — we prove existence by contradiction where convenient, we use the least upper bound property which requires choice, and so on.

A fully constructive treatment of real analysis — one that Brouwer would endorse — is substantially harder. Bishop’s *Constructive Analysis* (1967) showed it can be done, but the proofs are longer and some classical results must be weakened. The Lean/Mathlib library contains both classical and constructive portions, and flags which results require `Classical` axioms.

The key point for us: when we write **inductive MyNat**, we are defining $\mathbb{N}$ *constructively* — each natural number is literally a finite sequence of `succ` constructors. But when we later invoke the least upper bound property of $\mathbb{R}$ to prove the Intermediate Value Theorem, we are using classical logic. Both are legitimate mathematics; they just live in different epistemic universes.

1.2 Arithmetic by Recursion

The Peano axioms give us $\mathbb{N}$ as a bare set with a successor function. We have not yet said what $+$ or $\times$ mean — those must be *defined*, not assumed. The tool for doing so is the recursion principle: to define a function $f : \mathbb{N} \rightarrow X$, it suffices to specify $f(0)$ and a rule computing $f(S(n))$ from $f(n)$. This is just the constructive reading of induction: define the function by building it bottom-up from the base case.

The definitions below may look strange at first — why define $n + S(m)$ rather than $S(n) + m$? Recursing on the *second* argument is a convention (it matches Peano’s original definition and the one Lean’s own `Nat` uses); recursing on the first would work just as well. What matters for termination is only that each recursive call peels one `succ` constructor off a designated argument, so the recursion is structurally decreasing. The commutativity $m + n = n + m$ is then a non-trivial *theorem* that requires induction to prove, not something built into the definition.

Definition: Addition and Multiplication on $\mathbb{N}$

Addition is defined by recursion on the second argument:

$$n + 0 := n, \quad n + S(m) := S(n + m).$$

Multiplication:

$$n \cdot 0 := 0, \quad n \cdot S(m) := (n \cdot m) + n.$$

Lean 4

```

@[simp] def MyNat.add : MyNat → MyNat → MyNat
| n, .zero    ⇒ n
| n, .succ m  ⇒ .succ (n.add m)

-- Two cases only. Overlapping patterns (e.g. a zero+n shortcut) break
-- simp's equation lemmas, so we keep the definition non-overlapping
-- and prove zero_add as a theorem instead.

@[simp] def MyNat.mul : MyNat → MyNat → MyNat
| _, .zero    ⇒ .zero
| n, .succ m  ⇒ (n.mul m).add n

instance : Add MyNat := ⟨MyNat.add⟩
instance : Mul MyNat := ⟨MyNat.mul⟩

-- Bridge lemmas: connect + and * notation through the typeclass
-- instance to the raw definitions so simp can unfold them.
-- Without these, simp [MyNat.add] sees + but can't unfold it.
@[simp] theorem MyNat.add_def (m n : MyNat) : m + n = m.add n := rfl
@[simp] theorem MyNat.mul_def (m n : MyNat) : m * n = m.mul n := rfl

-- zero + n = n. Not immediate from the definition since add
-- recurses on the SECOND argument; needs induction on n.
theorem MyNat.zero_add (n : MyNat) : MyNat.zero.add n = n := by
  induction n with
  | zero      ⇒ rfl
  | succ n ih ⇒ simp [ih]

-- n.succ.add m = (n.add m).succ. Succ is on the LEFT argument
-- which the definition doesn't recurse on; needs induction on m.
theorem MyNat.succ_add (n m : MyNat) : n.succ.add m = (n.add m).succ :=
  by
    induction m with
    | zero      ⇒ rfl
    | succ m ih ⇒ simp [ih]

-- Commutativity: uses both zero_add and succ_add as helpers.
theorem MyNat.add_comm (m n : MyNat) : m.add n = n.add m := by
  induction n with
  | zero      ⇒ simp [MyNat.zero_add]
  | succ n ih ⇒ simp [MyNat.succ_add, ih]

-- Associativity: follows directly from the definition by induction on c
--
theorem MyNat.add_assoc (a b c : MyNat) : (a.add b).add c = a.add (b.add
  c) := by
  induction c with
  | zero      ⇒ rfl

```

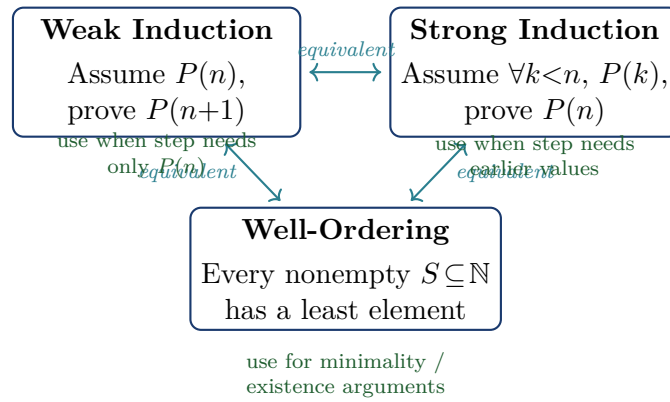
| succ c ih $\Rightarrow$ **simp** [ih]

1.3 The Order on $\mathbb{N}$

Arithmetic alone does not give us a way to compare natural numbers — we need an ordering. The right definition turns out to be purely additive: $m \leq n$ means there is some k that witnesses the gap $n - m$ (without subtraction!). This is the only way to define order from addition without assuming subtraction exists.

The *well-ordering* property is what distinguishes $\mathbb{N}$ from other ordered sets like $\mathbb{Z}$ or $\mathbb{Q}$: every nonempty subset has a smallest element. This is the principle underlying nearly every induction argument we will make in the rest of the book — if something fails, it fails at some *smallest* n , and analysing that n gives the contradiction. Strong induction and well-ordering are two faces of the same coin, as the proof below shows.

The three forms of induction are logically equivalent; choose whichever gives the most natural proof:



Definition: Natural Order

$m \leq n$ if there exists $k \in \mathbb{N}$ with $m + k = n$. $m < n$ means $m \leq n$ and $m \neq n$.

Theorem: $(\mathbb{N}, \leq)$ is a Well-Ordered Total Order

$\leq$ is reflexive, antisymmetric, transitive, and total. Moreover, every nonempty subset of $\mathbb{N}$ has a least element (**well-ordering**).

Proof. Reflexivity: $m + 0 = m$. Antisymmetry: if $m + k = n$ and $n + j = m$ then $m + k + j = m$ so $k = j = 0$, giving $m = n$. Transitivity: $m + k = n$ and $n + j = p$ gives $m + (k + j) = p$. Totality: by induction on n . Well-ordering follows from the equivalence theorem below. $\square$

Theorem: The Three Forms of Induction Are Equivalent

The following three principles are logically equivalent — each can be derived from the others using only the definitions:

1. **Weak induction (P5):** If $P(0)$ and $\forall n, P(n) \Rightarrow P(n+1)$, then $\forall n, P(n)$.
2. **Strong induction:** If $\forall n, (\forall k < n, P(k)) \Rightarrow P(n)$, then $\forall n, P(n)$.
3. **Well-ordering:** Every nonempty $S \subseteq \mathbb{N}$ has a least element.

Proof: (1) $\Rightarrow$ (2). Suppose strong induction's hypothesis holds: $(\forall k < n, P(k)) \Rightarrow P(n)$ for all n . Define $Q(n) := \forall k \leq n, P(k)$. We prove $\forall n, Q(n)$ by *weak* induction.

Base case $Q(0)$: We need $P(0)$. The strong hypothesis at $n = 0$ reads $(\forall k < 0, P(k)) \Rightarrow P(0)$. The premise is vacuously true (no $k < 0$ exists), so $P(0)$ holds.

Step: Assume $Q(n)$, i.e., $\forall k \leq n, P(k)$. For $Q(n+1)$ we need $P(n+1)$ additionally. The strong hypothesis at $n+1$ reads $(\forall k \leq n, P(k)) \Rightarrow P(n+1)$, and $Q(n)$ is exactly the premise. So $P(n+1)$ holds and $Q(n+1)$ follows.

By weak induction $\forall n, Q(n)$, so $\forall n, P(n)$. $\square$

Proof: (2) $\Rightarrow$ (3). Let $S \subseteq \mathbb{N}$ be nonempty; assume for contradiction it has no least element. Set $P(n) := n \notin S$ and apply strong induction.

Step: Suppose $\forall k < n, k \notin S$. If $n \in S$, then n would be the least element of S (all smaller elements are excluded). But S has no least element — contradiction. So $n \notin S$, i.e., $P(n)$.

By strong induction $\forall n, n \notin S$, so $S = \emptyset$ — contradicting S nonempty. $\square$

Proof: (3) $\Rightarrow$ (1). Assume well-ordering. Suppose $P(0)$ holds and $\forall n, P(n) \Rightarrow P(n+1)$. Let $S = \{n \in \mathbb{N} : \neg P(n)\}$ be the set of *counterexamples*.

Suppose $S \neq \emptyset$. By well-ordering, S has a least element m . Since $P(0)$ holds, $0 \notin S$, so $m \neq 0$, giving $m = k+1$ for some k . Since $k < m$ and m is the *least* element of S , we have $k \notin S$, so $P(k)$ holds. By the step hypothesis $P(k) \Rightarrow P(k+1) = P(m)$, so $m \notin S$ — contradiction.

Therefore $S = \emptyset$ and $\forall n, P(n)$. $\square$

Formalising this is our first taste of a recurring reality: before Lean will even let us *state* $k < n$ for our own `MyNat`, we must define the order and register it with the `LE/LT` type classes, and before the equivalence proofs go through we need a small toolbox of order lemmas ($k \leq 0 \Rightarrow k = 0$, nothing is below zero, $k < n+1 \Rightarrow k \leq n$, ...). None of this exists until we build it — `MyNat` is ours alone, so Lean's lemmas about its built-in `Nat` do not apply.

Lean 4

```
-- The order itself: m ≤ n iff some k witnesses the gap.
def MyNat.le (m n : MyNat) : Prop := ∃ k : MyNat, m.add k = n
def MyNat.lt (m n : MyNat) : Prop := m.le n ∧ m ≠ n

-- Register with the LE/LT type classes so ≤ and < notation work.
instance : LE MyNat := ⟨MyNat.le⟩
```



```

instance : LT MyNat := (MyNat.)lt

theorem MyNat.le_refl (n : MyNat) : n ≤ n := (.zero, )rfl

-- Key lemma: n + k.succ ≠ n (can't add a positive and return to
-- start). The succ case needs succ_add in the simp set to line the
-- goal up with the induction hypothesis.
theorem MyNat.no_overflow (n m : MyNat) : n.add m.succ ≠ n := by
  induction n with
  | zero      ⇒ simp [MyNat.add]
  | succ n ih ⇒ simp [MyNat.add, MyNat.succ_add] at *; exact ih

-- Corollary: if n + k = n then k = 0
theorem MyNat.add_eq_self (n k : MyNat) : n.add k = n → k = .zero := by
  cases k with
  | zero      ⇒ intros; rfl
  | succ k    ⇒ intro h; exact absurd h (MyNat.no_overflow n k)

-- If a + b = 0 then a = 0 (and by symmetry b = 0)
theorem MyNat.add_eq_zero_left {a b : MyNat}
  (h : a.add b = .zero) : a = .zero := by
  cases a with
  | zero      ⇒ rfl
  | succ a    ⇒ cases b <|> simp [MyNat.add] at h

-- Antisymmetry: uses add_eq_self to avoid case-splitting on witnesses
theorem MyNat.le_antisymm {n m : MyNat}
  (h1 : n ≤ m) (h2 : m ≤ n) : n = m := by
  obtain ⟨k, ⟩hk := h1
  obtain ⟨j, ⟩hj := h2
  -- n + k = m and m + j = n, so n + (k + j) = n
  have hkj : n.add (k.add j) = n := by
    calc n.add (k.add j)
      = (n.add k).add j := (MyNat.add_assoc n k j).symm
      _ = m.add j       := by rw [show n.add k = m from hk]
      _ = n             := hj
  -- Therefore k + j = 0, so k = 0, so n = m
  have h0 : k.add j = .zero := MyNat.add_eq_self n _ hkj
  have hk0 : k = .zero      := MyNat.add_eq_zero_left h0
  simp [hk0, MyNat.add] at hk
  exact hk

```

With the order in place we can build the toolbox and derive strong induction from weak induction by the $Q(n)$ trick from the proof above.

Lean 4

```

-- The order toolbox: four small lemmas the big proofs lean on.
theorem MyNat.le_zero_mp {n : MyNat} (h : n ≤ .zero) : n = .zero := by

```

```

obtain ⟨k, ⟩hk := h
exact MyNat.add_eq_zero_left hk

theorem MyNat.not_lt_zero (n : MyNat) : ¬ n < .zero := by
  intro ⟨hle, ⟩hne
  exact hne (MyNat.le_zero_mp hle)

theorem MyNat.lt_or_eq_of_le {n m : MyNat} (h : n ≤ m) : n < m ∨ n = m
  := by
  obtain ⟨k, ⟩hk := h
  cases k with
  | zero ⇒ exact Or.inr (by simp [MyNat.add] using hk)
  | succ k ⇒
    refine Or.inl ⟨⟨k.succ, ⟩hk, ?⟩_
    intro heq
    subst heq
    exact MyNat.no_overflow n k hk

theorem MyNat.lt_succ_mp {n m : MyNat} (h : n < m.succ) : n ≤ m := by
  obtain ⟨⟨k, ⟩hk, ⟩hne := h
  cases k with
  | zero ⇒ exact absurd (by simp [MyNat.add] using hk) hne
  | succ k ⇒
    exact ⟨k, by simp [MyNat.add] using ⟩hk

-- (2) Strong induction -- derived from weak by the Q(n) trick.
-- Define Q n := ∀ k ≤ n, P k, prove Q by weak induction,
-- then P follows since P(n) is implied by Q(n).
theorem MyNat.strong_ind (P : MyNat → Prop)
  (step : ∀ n, (∀ k, k < n → P k) → P n) :
  ∀ n, P n := by
  suffices h : ∀ n, ∀ k, k ≤ n → P k from
    fun n ⇒ h n n (MyNat.le_refl n)
  intro n
  induction n with
  | zero ⇒
    intro k hk
    have hkz : k = MyNat.zero := MyNat.le_zero_mp hk
    subst hkz
    exact step .zero (fun k hlt ⇒ absurd hlt (MyNat.not_lt_zero k))
  | succ n ih ⇒
    intro k hk
    rcases MyNat.lt_or_eq_of_le hk with hlt | rfl
    · exact ih k (MyNat.lt_succ_mp hlt)
    · exact step (.succ n) (fun j hj ⇒ ih j (MyNat.lt_succ_mp hj))

```

Finally, well-ordering. The classical proof — “pick any witness; either it is minimal or something smaller is in S , and descend” — becomes a strong induction on the witness. Totality of the order (`lt_or_ge`) supplies the “something smaller”

step, and `Classical.em` performs the case split on whether the current witness is already minimal.

Lean 4

```

theorem MyNat.succ_ne_self (n : MyNat) : n.succ ≠ n := by
  induction n with
  | zero      ⇒ simp
  | succ n ih ⇒ intro h; exact ih (MyNat.succ.inj h)

theorem MyNat.le_succ_self (n : MyNat) : n ≤ n.succ := { .one, } rfl

theorem MyNat.lt_succ_self (n : MyNat) : n < n.succ :=
  (MyNat.le_succ_self n, fun h ⇒ MyNat.succ_ne_self n h.) symm

theorem MyNat.succ_le_of_lt {n m : MyNat} (h : n < m) : n.succ ≤ m := by
  obtain ⟨k, hk, hne := h
  cases k with
  | zero      ⇒ exact absurd (by simp [MyNat.add] using hk) hne
  | succ k    ⇒ exact ⟨k, by subst hk; simp [MyNat.add, MyNat.succ_add]⟩

theorem MyNat.lt_succ_of_lt {n m : MyNat} (h : n < m) : n < m.succ := by
  obtain ⟨k, hk, _ := h
  constructor
  · exact ⟨k.succ, by simp [MyNat.add, hk]⟩
  · intro heq
    subst heq
    exact MyNat.no_overflow m k
    (by simp [MyNat.add, MyNat.succ_add] using hk)

-- Totality: any two naturals are comparable.
theorem MyNat.lt_or_ge (n m : MyNat) : n < m ∨ m ≤ n := by
  induction m with
  | zero      ⇒ exact Or.inr ⟨n, MyNat.zero_add n⟩
  | succ m ih ⇒
    rcases ih with hlt | hge
    · exact Or.inl (MyNat.lt_succ_of_lt hlt)
    · rcases MyNat.lt_or_eq_of_le hge with hlt | rfl
      · exact Or.inr (MyNat.succ_le_of_lt hlt)
      · exact Or.inl (MyNat.lt_succ_self _)

theorem MyNat.lt_of_not_le {a b : MyNat} (hn : ¬ a ≤ b) : b < a :=
  (MyNat.lt_or_ge b a).resolve_right hn

-- (3) Well-ordering -- every nonempty S ⊆ MyNat has a least element.
-- Strong induction on the witness: either n is already minimal, or
-- S contains some k < n and we recurse on k.
theorem MyNat.well_ordering (S : MyNat → Prop) (hne : ∃ n, S n) :
  ∃ m, S m ∧ ∀ k, S k → m ≤ k := by
  obtain ⟨n, hn := hne

```

```

refine MyNat.strong_ind
  (fun n => S n -> ∃ m, S m ∧ ∀ k, S k -> m ≤ k) ?_ n hn
intro n ih hSn
rcases Classical.em ∀( k, S k -> n ≤ k) with hmin | hnotmin
· exact ⟨n, hSn, ⟩hmin
· rw [Classical.not_∀] at hnotmin
  obtain ⟨k, ⟩hk := hnotmin
  rw [Classical.not_imp] at hk
  obtain ⟨hSk, ⟩hkn := hk
  exact ih k (MyNat.lt_of_not_le hkn) hSk

```

Exercises

1. Prove $a \cdot (b + c) = a \cdot b + a \cdot c$ for $\mathbb{N}$ by induction on c .
2. Prove the cancellation law: $a + c = b + c \Rightarrow a = b$.
3. Define exponentiation recursively and prove $n^{j+k} = n^j \cdot n^k$.
4. Implement truncated subtraction $m \dot{-} n$ (returns 0 when $n > m$) and prove $m \dot{-} m = 0$.
5. Prove the division algorithm: for $b > 0$, there exist unique q, r with $r < b$ and $a = bq + r$.
6. Prove that $\mathbb{N}$ satisfies P3 (injectivity of succ) and P4 (0 is not a successor) from the inductive type definition.

Chapter 2

The Integers

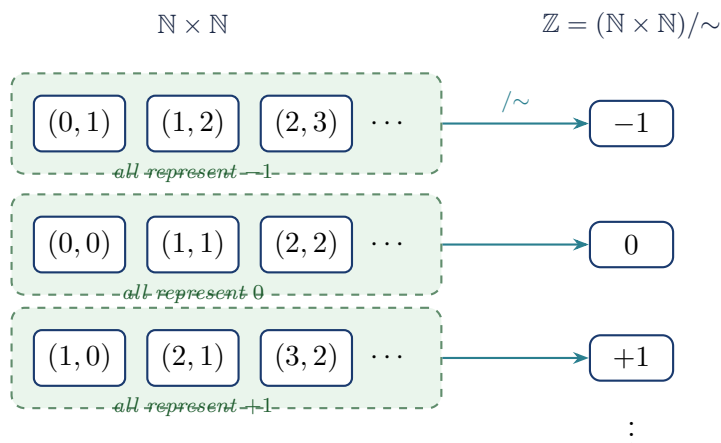
The natural numbers have no subtraction: $3 + x = 2$ has no solution in $\mathbb{N}$. We introduce negative numbers via a quotient construction—an integer is a *difference* of two naturals.

2.1 Constructing $\mathbb{Z}$

The key idea is that a negative number is not a new primitive thing but rather a *relationship between two natural numbers*: the integer -3 is just the claim that something is 3 less than something else. More precisely, the pair $(2, 5)$ represents $2 - 5 = -3$, but so does $(0, 3)$, $(7, 10)$, and $(100, 103)$. All of these pairs should be identified as the same integer.

This is our first encounter with the **quotient construction**: we have a “too big” set ($\mathbb{N} \times \mathbb{N}$, with many redundant representatives for each integer) and an equivalence relation that collapses the redundancy. The integers are what remains after making all equivalent pairs identical. The same idea appeared implicitly in everyday arithmetic — when you learned that $\frac{2}{4} = \frac{1}{2}$, you were working with a quotient — and we will use it once more in Chapter 3 to build $\mathbb{Q}$.

Notice the elegance of the equivalence condition $(a, b) \sim (c, d) \iff a + d = b + c$: it avoids subtraction entirely, expressing $a - b = c - d$ using only addition, which we already have on $\mathbb{N}$.



Definition: The Integer Equivalence Relation

Define $\sim$ on $\mathbb{N} \times \mathbb{N}$ by:

$$(a, b) \sim (c, d) \iff a + d = b + c.$$

Intuitively, (a, b) represents $a - b$, and two pairs represent the same integer iff their differences agree. The integers are $\mathbb{Z} := (\mathbb{N} \times \mathbb{N})/\sim$.

Theorem: $\sim$ is an Equivalence Relation

Proof. Refl: $(a, b) \sim (a, b)$ since $a + b = b + a$ by commutativity in $\mathbb{N}$. **Symm:** $a + d = b + c \Rightarrow c + b = d + a$, so $(c, d) \sim (a, b)$. **Trans:** From $a + d = b + c$ and $c + f = d + e$, add both equations to get $a + d + c + f = b + c + d + e$; cancel $c + d$ (cancellation law for $\mathbb{N}$) to obtain $a + f = b + e$, so $(a, b) \sim (e, f)$. $\square$

Lean 4

```
-- We build Z on top of the MyNat from Chapter 1.
-- (a, b) represents the integer a - b.
-- Note: IntRel uses Nat (built-in) so omega and simp work cleanly.
def IntRel : (Nat × Nat) → (Nat × Nat) → Prop
| (a, b), (c, d) => a + d = b + c

theorem IntRel.refl : ∀ p : Nat × Nat, IntRel p p
| (a, b) => by simp [IntRel, Nat.add_comm]

theorem IntRel.symm : ∀ p q, IntRel p q → IntRel q p := by
  intro (a, b) (c, d) h; simp [IntRel] at *; omega

-- Transitivity: uses the cancellation law for MyNat
theorem IntRel.trans : ∀ p q r,
  IntRel p q → IntRel q r → IntRel p r := by
  intro (a, b) (c, d) (e, f) h1 h2
  simp [IntRel] at *
  omega

-- MyInt as the quotient
def MyInt : Type := Quot IntRel

def mkInt (a b : Nat) : MyInt := Quot.mk IntRel (a, b)

-- The canonical embedding Nat → MyInt
def intOfNat (n : Nat) : MyInt := mkInt n 0

-- Familiar constants
def intZero : MyInt := mkInt 0 0
def intOne : MyInt := mkInt 1 0
```

```

-- Negation: -(a - b) = b - a
def intNeg (x : MyInt) : MyInt :=
  Quot.lift (fun p => mkInt p.2 p.1)
    (by intro (a, b) (c, d) h
      apply Quot.sound
      simp [IntRel] at *; omega) x

```

2.2 Ring Axioms for $\mathbb{Z}$

Now that we have $\mathbb{Z}$ as a set, we must equip it with arithmetic. The definitions look natural once you remember that (a, b) means $a - b$: adding two differences adds numerators and denominators componentwise, and multiplying uses the FOIL identity $(a - b)(c - d) = (ac + bd) - (ad + bc)$.

The subtle point is **well-definedness**. When we write $[(a, b)] + [(c, d)] := [(a + c, b + d)]$, we are defining addition on *equivalence classes* by choosing *representatives*. This is only legitimate if swapping in a different representative for $[(a, b)]$ or $[(c, d)]$ gives the same equivalence class on the right-hand side. Checking well-definedness is not optional: it is the mathematical price of the quotient construction, and it must be paid for every operation we define. In Lean 4 this is enforced by the type system via `Quot.lift`, which demands a proof of well-definedness before it will build the lifted function. For a *binary* operation there is no ready-made `Quot.lift2` in core Lean: we lift one argument at a time with two nested `Quot.lifts`, and we owe Lean *two* well-definedness proofs — one for each argument. Chapter 5 carries this out in full for addition on the reals.

A *commutative* ring with $1 \neq 0$ satisfying $ab = 0 \Rightarrow a = 0$ or $b = 0$ is called an **integral domain**. The last property says there are no “zero divisors” — no two nonzero elements whose product is zero. $\mathbb{Z}$ is an integral domain; this fact will be used in the transitivity proof for $\approx$ in Chapter 3, and it is what allows us to cancel a common nonzero factor.

Definition: Addition and Multiplication on $\mathbb{Z}$

$[(a, b)] + [(c, d)] := [(a + c, b + d)]$
 $[(a, b)] \cdot [(c, d)] := [(ac + bd, ad + bc)]$
 $-[(a, b)] := [(b, a)]$
 (The multiplication formula follows from $(a - b)(c - d) = ac + bd - (ad + bc)$.)

Theorem: $\mathbb{Z}$ is a Commutative Ring

$\mathbb{Z}$ satisfies all ring axioms; in particular it is an integral domain: $ab = 0 \Rightarrow a = 0$ or $b = 0$.

The proof of each axiom proceeds by lifting from $\mathbb{N} \times \mathbb{N}$ representatives (where $\mathbb{N}$ here means our `MyNat` from Chapter 1), reducing each axiom to the arithmetic facts

proved there. Well-definedness requires checking that the result is independent of the chosen representative.

Exercises

1. Implement `addInt` and `mulInt` in Lean 4 with well-definedness proofs.
2. Prove $a \cdot 0 = 0$ in $\mathbb{Z}$ directly from the definition.
3. Prove $(-1) \cdot a = -a$ in $\mathbb{Z}$.
4. Define the order on $\mathbb{Z}$ and prove it is a total order compatible with addition and multiplication by positives.
5. Prove $\mathbb{Z}$ has no zero divisors by reducing to the analogous fact for $\mathbb{N}$.

Chapter 3

The Rationals

The integers lack division: $2x = 3$ has no solution in $\mathbb{Z}$. We apply the same quotient trick a second time, this time forming fractions.

3.1 Constructing $\mathbb{Q}$

We are repeating the same trick as Chapter 2, but one level up. There we turned $\mathbb{N}$ into $\mathbb{Z}$ by formally introducing differences $a - b$. Now we turn $\mathbb{Z}$ into $\mathbb{Q}$ by formally introducing quotients a/b . The equivalence relation $\frac{a}{b} = \frac{c}{d} \iff ad = bc$ is the cross-multiplication criterion you learned in school, now elevated to a definition.

The requirement that the denominator be nonzero ($b \in \mathbb{Z}^*$) is essential: $\frac{a}{0}$ is not a number. In Lean 4 we enforce this by storing a proof `den_pos : 0 < den` inside the `MyRat` structure — the type system makes it impossible to construct a rational number with denominator zero. This is an example of **dependent types** at work: the type of a value depends on a proof.

One subtlety: we defined `MyRat` as a `structure` (a concrete record) rather than a quotient type. To use it correctly we must always work with the equivalence relation `ratEquiv` explicitly when stating theorems, rather than appealing to definitional equality. An alternative design using `Quot` would match our $\mathbb{Z}$ construction more closely, and is left as an exercise.

Definition: The Rational Equivalence Relation

Let $\mathbb{Z}^* := \mathbb{Z} \setminus \{0\}$. Define $\approx$ on $\mathbb{Z} \times \mathbb{Z}^*$ by:

$$(a, b) \approx (c, d) \iff a \cdot d = b \cdot c.$$

The rationals are $\mathbb{Q} := (\mathbb{Z} \times \mathbb{Z}^*)/\approx$. The class $[(a, b)]$ represents a/b .

Theorem: $\approx$ is an Equivalence Relation

Proof. Reflexivity ($ad = da$) and symmetry are immediate. **Trans:** From $ad = bc$ and $cf = de$, multiply to get $adf = bcf = bde$; cancel $d \neq 0$ (using that $\mathbb{Z}$ is an integral domain) to obtain $af = be$. $\square$

Definition: Field Operations on $\mathbb{Q}$

$$\frac{a}{b} + \frac{c}{d} := \frac{ad + bc}{bd}, \quad \frac{a}{b} \cdot \frac{c}{d} := \frac{ac}{bd}, \quad \left(\frac{a}{b}\right)^{-1} := \frac{b}{a} \ (a \neq 0).$$

Theorem: $\mathbb{Q}$ is an Ordered Field

$\mathbb{Q}$ is a field under these operations. It admits a total order compatible with addition and multiplication: $a/b < c/d$ (for $b, d > 0$) iff $ad < bc$ in $\mathbb{Z}$. This order makes $\mathbb{Q}$ an ordered field.

Theorem: Density of $\mathbb{Q}$

Between any two distinct rationals there is another: for $p < q$ in $\mathbb{Q}$, the midpoint $(p + q)/2$ satisfies $p < (p + q)/2 < q$.

Theorem: Archimedean Property of $\mathbb{Q}$

For any $q > 0$ in $\mathbb{Q}$, there exists $n \in \mathbb{N}$ with $1/n < q$.

Despite density and the Archimedean property, $\mathbb{Q}$ has gaps—the subject of Chapter 4.

Lean 4

```
-- Rational numbers using integers
structure MyRat where
  num : Int
  den : Int
  den_pos : 0 < den

def ratEquiv (p q : MyRat) : Prop :=
  p.num * q.den = q.num * p.den

def addRat (p q : MyRat) : MyRat :=
  { num      := p.num * q.den + q.num * p.den,
    den      := p.den * q.den,
    den_pos  := Int.mul_pos p.den_pos q.den_pos }

def mulRat (p q : MyRat) : MyRat :=
  { num      := p.num * q.num,
    den      := p.den * q.den,
    den_pos  := Int.mul_pos p.den_pos q.den_pos }

def invRat (p : MyRat) (h : p.num ≠ 0) : MyRat :=
  if hp : 0 < p.num then
    { num := p.den, den := p.num, den_pos := hp }
  else
```

```

{ num := -p.den, den := -p.num,
  den_pos := by omega }

-- The midpoint of two rationals. The positivity proof is assembled
-- by hand from Int.mul_pos (in Part II, Mathlib's `positivity`
-- tactic will discharge goals like this in one line).
def midpoint (p q : MyRat) : MyRat :=
{ num      := p.num * q.den + q.num * p.den,
  den      := 2 * p.den * q.den,
  den_pos  := Int.mul_pos (Int.mul_pos (by omega) p.den_pos) q.den_pos
}

```

Exercises

1. Prove that $\mathbb{Q}$ has characteristic 0: $n \cdot 1_{\mathbb{Q}} \neq 0$ for all $n > 0$.
2. Prove the field axioms (additive inverse, multiplicative inverse for nonzero) for $\mathbb{Q}$.
3. Prove well-definedness of `addRat` with respect to `ratEquiv`.
4. Show $\mathbb{Q}$ is countable: exhibit an injection $\mathbb{Q} \hookrightarrow \mathbb{N} \times \mathbb{N}$.

Chapter 4

Why $\mathbb{Q}$ Is Not Enough

Remark

Historical Note: The Crisis of the Irrational

The discovery that $\sqrt{2}$ is irrational reportedly shook the Pythagorean brotherhood to its foundations. The Pythagoreans believed that all of reality was expressible in ratios of whole numbers — “all is number,” in their slogan. The proof that the diagonal of a unit square cannot be so expressed was not merely an abstract theorem; it was an existential threat to their worldview.

The legend that Hippasus of Metapontum was drowned for revealing this result to outsiders is probably apocryphal, but it captures the genuine philosophical shock. The Greeks called such lengths *alogos* (without ratio) or *arrhetos* (inexpressible) — words that eventually became our “irrational.” Euclid includes the proof in Book X of the *Elements* (c. 300 BCE), buried deep enough that most readers never reached it.

The proof we give below is essentially Euclid’s, though he phrased it geometrically rather than algebraically. The key ingredient is what we would call the **fundamental theorem of arithmetic** — the unique factorisation of integers into primes — which Euclid also proved in Book IX. The modern algebraic form (if p is prime and $p \mid n^2$ then $p \mid n$) was made explicit by Gauss in the *Disquisitiones Arithmeticae* (1801).

For two thousand years after the Greeks, mathematicians worked with irrational numbers as geometric magnitudes (lengths, areas, volumes) rather than as numbers in their own right. It was only the arithmetisation of analysis in the nineteenth century that forced mathematicians to ask: what *is* $\sqrt{2}$, precisely? The answer — Dedekind cuts, or Cauchy sequences, or any other rigorous construction of $\mathbb{R}$ — did not arrive until 1872.

4.1 The Irrationality of $\sqrt{2}$

The simplest demonstration that $\mathbb{Q}$ is incomplete is the equation $x^2 = 2$. This has an obvious geometric solution — the diagonal of a unit square has length exactly $\sqrt{2}$ — yet no fraction p/q satisfies it. The ancient Greeks found this result shocking: the Pythagoreans reportedly tried to keep it secret, and legend has it that Hippasus

of Metapontum was drowned for revealing it.

The proof is a classic argument by contradiction using the unique factorisation of integers. It generalises immediately: the same argument shows $\sqrt{n} \notin \mathbb{Q}$ whenever n is not a perfect square. In Lean 4 we can give the full formal proof using only the divisibility machinery of `Nat`, which Lean's standard library provides — we do not need our full `MyInt` or `MyRat` constructions for this particular theorem.

Theorem: $\sqrt{2} \notin \mathbb{Q}$

No rational number squares to 2.

Proof. Suppose $p/q = \sqrt{2}$ with $\gcd(p, q) = 1$. Then $p^2 = 2q^2$, so p is even; write $p = 2k$. Then $4k^2 = 2q^2$, so $q^2 = 2k^2$, making q even. But then $\gcd(p, q) \geq 2$ — contradiction. $\square$

Lean 4

```
-- If n*n is even, n is even: split on n mod 2. If n were odd,
-- n*n mod 2 = 1, contradicting 2 | n*n. All arithmetic by omega.
theorem even_of_sq_even (n : Nat) (h : 2 | n * n) : 2 | n := by
  rcases Nat.mod_two_eq_zero_or_one n with h0 | h1
  · exact Nat.dvd_of_mod_eq_zero h0
  · exfalse
    have h2 : (n * n) % 2 = 1 := by rw [Nat.mul_mod, h1]
    have h0 : (n * n) % 2 = 0 := Nat.mod_eq_zero_of_dvd h
    omega

theorem sqrt2_irrational :
  ∀ p q : Nat, q ≠ 0 → Nat.Coprime p q → p * p ≠ 2 * (q * q) := by
  intro p q hq hcop heq
  -- p^2 = 2q^2, so p is even; write p = 2k
  have h2p : 2 | p := even_of_sq_even p ⟨q * q, ⟩heq
  obtain ⟨k, ⟩rfl := h2p
  -- Then 4k^2 = 2q^2, so q^2 = 2k^2, making q even too
  have hexp : 2 * k * (2 * k) = 2 * (2 * (k * k)) := by ac_rfl
  have heq' : 2 * (2 * (k * k)) = 2 * (q * q) := by rw ←[ hexp]; exact
    heq
  have hq2 : q * q = 2 * (k * k) := by omega
  have h2q : 2 | q := even_of_sq_even q ⟨k * k, ⟩hq2
  -- But then 2 divides gcd(p, q) = 1 -- contradiction
  have hgcd : 2 | Nat.gcd (2 * k) q := Nat.dvd_gcd ⟨k, ⟩rfl h2q
  rw [hcop] at hgcd
  omega
```

4.2 Cauchy Sequences Without Rational Limits

Irrationality of $\sqrt{2}$ shows $\mathbb{Q}$ is missing individual points. But the deeper failure of $\mathbb{Q}$ is *sequential*: you can be moving steadily toward a destination, getting closer and

closer to it with every step, and yet the destination simply does not exist inside $\mathbb{Q}$. This is the incompleteness that Cauchy sequences expose.

The Cauchy condition $|q_m - q_n| < \varepsilon$ for all large m, n captures the intuition of “the terms are bunching together”. It is a condition purely on the sequence itself — it does not mention any limit. A sequence can be Cauchy without us being able to name what it converges to. In a *complete* space, being Cauchy is enough to guarantee convergence; in $\mathbb{Q}$, it is not.

Newton’s method for $\sqrt{2}$ is the archetypal example. Starting from $q_1 = 1$, each step $q_{n+1} = (q_n + 2/q_n)/2$ is a rational operation on a rational number, so every term is rational. The terms get exponentially close to each other (the error squares at each step — this is quadratic convergence). Yet the sequence has nowhere to go inside $\mathbb{Q}$, because its limit $\sqrt{2}$ does not live there.

Definition: Cauchy Sequence in $\mathbb{Q}$

(q_n) is **Cauchy** if $\forall \varepsilon > 0 \in \mathbb{Q}, \exists N, \forall m, n > N, |q_m - q_n| < \varepsilon$.

Remark

Historical Note: Bolzano Before Cauchy

The Cauchy sequence is named after Augustin-Louis Cauchy, who used the convergence criterion in his landmark textbook *Cours d’analyse* (1821). But the concept was understood, at least implicitly, earlier. **Bernard Bolzano** — a Bohemian priest and mathematician working in near-total obscurity in Prague — stated and proved essentially the same criterion in 1817, four years before Cauchy’s textbook. Bolzano’s work went unread for decades; he is now recognised as one of the founders of rigorous analysis, but only posthumously. Cauchy himself was not entirely rigorous by modern standards. His 1821 proof that the limit of a convergent series of continuous functions is continuous contains a gap (he implicitly assumed *uniform* convergence, a concept that would only be articulated by Weierstrass decades later). Abel found a counterexample in 1826. This illustrates the iterative nature of mathematical rigour: each generation tightens the screws left loose by the previous one.

The name “Cauchy sequence” also obscures an irony: Cauchy believed the real numbers were already well-understood, and never felt the need to *construct* $\mathbb{R}$ from $\mathbb{Q}$. He simply assumed the reals existed and used convergence as a criterion for a sequence to have a limit *in* $\mathbb{R}$. The explicit construction of $\mathbb{R}$ using Cauchy sequences was carried out by **Georg Cantor** (1872), independently of Dedekind. Cantor was motivated by his work on trigonometric series, where he needed to understand exactly which sets of points could be the “exceptional sets” of a Fourier series. This led him directly to the question of what a real number fundamentally is — and from there to his revolutionary set theory, transfinite cardinals, and one of the most productive and turbulent careers in the history of mathematics.

Example 4.1. Newton’s method for $\sqrt{2}$: $q_1 = 1, q_{n+1} = (q_n + 2/q_n)/2$. The sequence $1, 3/2, 17/12, 577/408, \dots$ is Cauchy in $\mathbb{Q}$ but converges to $\sqrt{2} \notin \mathbb{Q}$.

Theorem: $\mathbb{Q}$ is Not Complete

There exists a Cauchy sequence in $\mathbb{Q}$ with no limit in $\mathbb{Q}$.

Proof. The Newton sequence above is Cauchy (the error is roughly *squared* at each step — quadratic convergence) and its limit satisfies $L^2 = 2$, which has no rational solution. $\square$

Remark

Cauchy completion generalizes: given any *metric space* — a set equipped with a distance function $d(x, y)$ that is symmetric, vanishes exactly when $x = y$, and satisfies the triangle inequality ($\mathbb{R}$ with $d(x, y) = |x - y|$ is the prototype) — we can form its completion by the same quotient of Cauchy sequences. This construction is used to define p -adic numbers, L^2 spaces, and completions of normed rings.

Exercises

1. Prove $\sqrt{3} \notin \mathbb{Q}$ and generalize: $\sqrt{p} \notin \mathbb{Q}$ for any prime p .
2. Show the sequence $q_n = \sum_{k=0}^n 1/k!$ is Cauchy in $\mathbb{Q}$.
3. Prove: any convergent sequence in $\mathbb{Q}$ is Cauchy.
4. Prove: any Cauchy sequence in $\mathbb{Q}$ is bounded.
5. Define a Dedekind cut and verify that $L = \{q \in \mathbb{Q} : q < 0 \vee q^2 < 2\}$ is a valid cut representing $\sqrt{2}$.

Chapter 5

The Real Numbers

Remark

Historical Note: The Year 1872 and the Construction of the Reals

The year 1872 saw a remarkable simultaneous publication of rigorous constructions of the real numbers. **Richard Dedekind**'s *Stetigkeit und irrationale Zahlen* (Continuity and Irrational Numbers) introduced Dedekind cuts — the approach of defining a real number as a partition of $\mathbb{Q}$ into a lower set and an upper set. **Georg Cantor**'s paper, published the same year, defined real numbers as equivalence classes of Cauchy sequences of rationals — the approach we follow. **Karl Weierstrass** had been teaching a similar construction in his Berlin lectures since the 1860s, though he did not publish it.

Dedekind was motivated by a moment of dissatisfaction in the classroom. While teaching calculus in Göttingen in 1858, he found himself unable to give a rigorous proof that a monotone bounded sequence converges — the very theorem we prove in Part II. He realised the proof ultimately rested on geometric intuition about the “continuity of the line,” and set out to make that continuity precise without appealing to geometry. His cuts were the result, written down immediately in 1858 but not published for fourteen years.

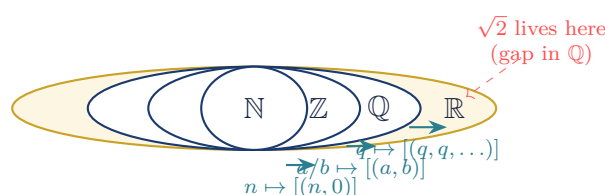
The two approaches — cuts and Cauchy sequences — produce isomorphic structures, as expected from the uniqueness theorem. Mathematicians have personal preferences: Dedekind cuts are more immediately geometric and make the order structure transparent; Cauchy sequences are more algebraic and generalise directly to any metric space (not just $\mathbb{Q}$). Dedekind's approach is used in most undergraduate analysis textbooks (Hardy, Rudin's *Principles*); Cantor's approach appears in more algebraic treatments and is the natural one for p -adic numbers.

A third construction, due to **Weierstrass**, builds $\mathbb{R}$ as infinite decimal expansions with a careful equivalence relation ($0.999\dots = 1.000\dots$). This is the most computationally concrete but the most painful to make rigorous. A fourth approach, due to **John Conway** (1976), constructs the “surreal numbers” — a vast ordered field containing $\mathbb{R}$, the ordinals, and infinitesimal quantities — via an elegant game-theoretic definition. Conway's construction is arguably the most beautiful but the least practical for analysis.

A note on non-standard analysis. In 1960, Abraham Robinson showed

that it is consistent (relative to ZFC) to have a complete ordered field containing genuine infinitesimals — numbers ε with $0 < \varepsilon < 1/n$ for all $n \in \mathbb{N}$. This is called **non-standard analysis** (or **hyperreal analysis**), and it vindicates the intuitions of Leibniz, who always thought of dx as a literal infinitesimal rather than a limit. Non-standard analysis gives rigorous proofs of many calculus theorems that are shorter and more intuitive than their standard counterparts, though the logical overhead of setting up the framework is substantial.

We now close the gaps in $\mathbb{Q}$ by formally adjoining the limits of all Cauchy sequences.



5.1 Cauchy Completion of $\mathbb{Q}$

The Cauchy completion is a bold move: instead of hunting for the missing points and trying to insert them one by one, we *declare* that every Cauchy sequence *is* a real number. If the sequence $(1, 1.4, 1.41, 1.414, \dots)$ wants to converge somewhere but $\mathbb{Q}$ has no such place, we simply *create* a new object and call it $\sqrt{2}$. Every Cauchy sequence that “should” converge to the same value defines the same real number.

This is the third and final quotient construction in Part I. The pattern is always the same:

1. Start with a large set of “candidates” (pairs of naturals; pairs of integers; Cauchy sequences).
2. Declare an equivalence relation that collapses redundant candidates (same difference; same ratio; same limit).
3. Define the new number system as the quotient.
4. Lift arithmetic to equivalence classes and check well-definedness.

The one genuinely new ingredient here, compared to Chapters 2 and 3, is that the candidate set $\mathcal{C}$ is *infinite-dimensional*: its elements are sequences, not pairs. The well-definedness checks therefore involve limit arguments (show that sums and products of Cauchy sequences are Cauchy), not just algebraic identities.

Definition: Equivalence of Cauchy Sequences

Two Cauchy sequences (a_n) and (b_n) in $\mathbb{Q}$ are equivalent—written $(a_n) \sim (b_n)$ —if:

$$\forall \varepsilon > 0 \in \mathbb{Q}, \exists N, \forall n > N, |a_n - b_n| < \varepsilon.$$

The real numbers are $\mathbb{R} := \mathcal{C}/\sim$, where $\mathcal{C}$ is the set of all Cauchy sequences in $\mathbb{Q}$.

Theorem: $\sim$ is an Equivalence Relation on $\mathcal{C}$ **Proof. Refl:** $|a_n - a_n| = 0 < \varepsilon$ for all n . **Symm:** $|b_n - a_n| = |a_n - b_n| < \varepsilon$.**Trans:** $|a_n - c_n| \leq |a_n - b_n| + |b_n - c_n|$; apply each hypothesis with $\varepsilon/2$. $\square$ **Definition: Arithmetic on $\mathbb{R}$** $[(a_n)] + [(b_n)] := [(a_n + b_n)]$ and $[(a_n)] \cdot [(b_n)] := [(a_n b_n)]$.The embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$ sends q to the constant sequence $[(q, q, q, \dots)]$.**Theorem: Products and Sums of Cauchy Sequences Are Cauchy**If (a_n) and (b_n) are Cauchy in $\mathbb{Q}$, so are $(a_n + b_n)$ and $(a_n b_n)$.*Proof. Sum:* Triangle inequality with $\varepsilon/2$. *Product:* Use boundedness (Cauchy $\Rightarrow$ bounded, shown separately). With $|a_n| \leq M_a$ and $|b_n| \leq M_b$:

$$|a_m b_m - a_n b_n| \leq M_a |b_m - b_n| + M_b |a_m - a_n|.$$

Choose N so both terms are $< \varepsilon/2$. $\square$

One housekeeping matter before the code: we are still in phase one — core Lean, no libraries — and core Lean provides the type `Rat` but *no* absolute value on it, and no $|\cdot|$ notation. True to the from-scratch spirit, we roll our own, `rabs`, together with the two facts the construction below actually uses (the remaining standard properties — $|ab| = |a||b|$, $|a - b| = |b - a|$, and so on — make good exercises). The proofs lean on `grind`, core Lean's general-purpose arithmetic closer.

Lean 4

```
-- Absolute value on Rat, from scratch (core Lean has none).
def rabs (q : Rat) : Rat := if q < 0 then -q else q

theorem rabs_zero : rabs 0 = 0 := by simp [rabs]

-- The triangle inequality -- the workhorse of all of analysis.
theorem rabs_add (a b : Rat) : rabs (a + b) ≤ rabs a + rabs b := by
  simp only [rabs]; grind
```

Lean 4

```
-- Our real numbers as a quotient of Cauchy sequences over Rat
-- (Using Lean's built-in Rat for the field of fractions)
def IsCauchy (a : Nat → Rat) : Prop :=
  ∀ eps : Rat, eps > 0 →
    ∃ N : Nat, ∀ m n : Nat, m > N → n > N →
      rabs (a m - a n) < eps

structure CauchySeq where
```

```

seq      : Nat → Rat
cauchy   : IsCauchy seq

def CauchyEquiv (a b : CauchySeq) : Prop :=
  ∀ eps : Rat, eps > 0 →
    ∃ N : Nat, ∀ n : Nat, n > N →
      rabs (a.seq n - b.seq n) < eps

-- MyReal is the quotient
def MyReal : Type := Quot CauchyEquiv

-- Constant sequence embeds Q into R
def ofRat (q : Rat) : MyReal :=
  Quot.mk CauchyEquiv
  { seq      := fun _ => q,
    cauchy := fun eps heps => ⟨0, fun _ _ _ => by
      have e : q - q = (0:Rat) := by grind
      rw [e, rabs_zero]; exact ⟩heps }

-- Addition of Cauchy sequences: the eps/2 argument from the
-- theorem above, verbatim.
def addCauchy (a b : CauchySeq) : CauchySeq :=
  { seq      := fun n => a.seq n + b.seq n,
    cauchy := fun eps heps => by
      obtain ⟨Na, ⟩ha := a.cauchy (eps/2) (by grind)
      obtain ⟨Nb, ⟩hb := b.cauchy (eps/2) (by grind)
      refine ⟨max Na Nb, fun m n hm hn => ?⟩_
      have h1 := ha m n (by omega) (by omega)
      have h2 := hb m n (by omega) (by omega)
      have key : (a.seq m + b.seq m) - (a.seq n + b.seq n)
        = (a.seq m - a.seq n) + (b.seq m - b.seq n) := by grind
      have tri := rabs_add (a.seq m - a.seq n) (b.seq m - b.seq n)
      rw [key]; grind }

```

Now the payoff of the well-definedness discussion from Chapter 2. Addition takes *two* `MyReal` arguments, so we must lift through the quotient twice — one nested `Quot.lift` per argument — and prove that `addCauchy` respects `CauchyEquiv` in *each argument separately*. Neither proof is optional: omit one and Lean rejects the definition.

Lean 4

```

-- Respecting the equivalence in the right argument...
theorem addCauchy_respects_right (a b b' : CauchySeq)
  (h : CauchyEquiv b b') :
  CauchyEquiv (addCauchy a b) (addCauchy a b') := by
  intro eps heps
  obtain ⟨N, ⟩hN := h eps heps
  refine ⟨N, fun n hn => ?⟩_

```

```

have hb := hN n hn
show rabs ((a.seq n + b.seq n) - (a.seq n + b'.seq n)) < eps
have key : (a.seq n + b.seq n) - (a.seq n + b'.seq n)
           = b.seq n - b'.seq n := by grind
rw [key]; exact hb

-- ...and in the left argument.
theorem addCauchy_respects_left (a a' b : CauchySeq)
  (h : CauchyEquiv a a') :
  CauchyEquiv (addCauchy a b) (addCauchy a' b) := by
intro eps heps
obtain ⟨N, ⟩hN := h eps heps
refine ⟨N, fun n hn => ?⟩_
have ha := hN n hn
show rabs ((a.seq n + b.seq n) - (a'.seq n + b.seq n)) < eps
have key : (a.seq n + b.seq n) - (a'.seq n + b.seq n)
           = a.seq n - a'.seq n := by grind
rw [key]; exact ha

-- Addition on MyReal: nested Quot.lift, one well-definedness
-- proof per argument.
def addReal : MyReal → MyReal → MyReal :=
  Quot.lift
    (fun a => Quot.lift (fun b => Quot.mk CauchyEquiv (addCauchy a b))
      (fun b b' h => Quot.sound (addCauchy_respects_right a b b' h)))
    (fun a a' h => by
      funext y
      induction y using Quot.ind with
      | mk b => exact Quot.sound (addCauchy_respects_left a a' b h))

```

5.2 Completeness and Key Properties of $\mathbb{R}$

We built $\mathbb{R}$ specifically to be complete, so the first thing to verify is that we succeeded. Completeness of $\mathbb{R}$ — every Cauchy sequence converges — follows almost by definition: a Cauchy sequence in $\mathbb{R}$ is a sequence of equivalence classes of Cauchy sequences in $\mathbb{Q}$, and we can always “diagonalise” to extract a single Cauchy sequence in $\mathbb{Q}$ that represents the limit.

The **least upper bound property** (every nonempty bounded-above set has a supremum) is equivalent to completeness, and is often taken as the definition of completeness in analysis textbooks. We derive it here from the Cauchy-sequence version using a bisection argument. This property is what powers the proofs of the Intermediate Value Theorem, the Extreme Value Theorem, and the existence of definite integrals — essentially every deep theorem of calculus ultimately rests on it.

The **Archimedean property** says $\mathbb{R}$ has no “infinitely large” natural numbers and no “infinitely small” positive reals. This might seem obvious but it is not a consequence of the ordered field axioms alone — there exist non-Archimedean

ordered fields. In $\mathbb{R}$ it follows from the least upper bound property: if $\mathbb{N}$ were bounded above, it would have a supremum, leading immediately to a contradiction.

Finally, a note on uniqueness: the complete ordered field axioms characterise $\mathbb{R}$ up to isomorphism. Any two complete ordered fields are isomorphic. This means the specific construction we used — Cauchy sequences of rationals — is one of many possible routes to the same destination. Dedekind cuts lead to the same place. So does any other sensible completion of $\mathbb{Q}$. This is reassuring: the real numbers are not an artifact of our particular construction but a mathematical object with an intrinsic identity.

The constructivist objection. Brouwer and his followers reject the least upper bound property as stated, because its proof requires the *law of excluded middle*: either a given rational q is an upper bound for S or it is not. A constructivist demands a *decision procedure* — an algorithm that, given any q , tells you which case applies. For a general subset $S \subseteq \mathbb{R}$ defined by an arbitrary predicate, no such algorithm may exist.

Bishop’s constructive analysis resolves this by weakening the statement: instead of “every bounded-above set has a supremum,” one proves “every *located* set” (one for which the distance from any point can be computed) has a supremum. This is a strictly stronger hypothesis and gives a strictly weaker conclusion, but it is computable. The resulting theory is consistent with classical analysis (every classically valid theorem is also constructively valid if you add enough hypotheses), but proofs are longer.

For our purposes, we work classically throughout. The Lean 4 theorems in this book use `Classical.em` (law of excluded middle) implicitly wherever we appeal to the LUB property or prove existence by contradiction. If you wanted a fully constructive development, you would need to replace our `MyReal` with a type of *Cauchy reals with modulus* — sequences equipped with an explicit convergence rate function — and all existence proofs would need to be replaced by explicit constructions. This is the approach taken in constructive libraries such as CoRN (for the Coq/Rocq prover); Lean’s Mathlib, which we adopt in the next chapter, is unapologetically classical.

Theorem: $\mathbb{R}$ is Complete

Every Cauchy sequence in $\mathbb{R}$ converges to an element of $\mathbb{R}$.

Proof sketch. Let (x_k) be Cauchy in $\mathbb{R}$, where each $x_k = [(a_n^{(k)})_n]$. For each k , choose a “diagonal” rational r_k within $1/k$ of x_k (possible since the Cauchy sequence for x_k eventually stays close to its limit). Then (r_k) is Cauchy in $\mathbb{Q}$, and $[(r_k)] = \lim_{k \rightarrow \infty} x_k$ in $\mathbb{R}$. $\square$

Theorem: The Least Upper Bound Property

Every nonempty subset of $\mathbb{R}$ that is bounded above has a **supremum** (least upper bound) in $\mathbb{R}$.

Proof sketch. This follows from completeness. Given $S \subseteq \mathbb{R}$ bounded above by M : pick any $a_0 \in S$ and set $b_0 = M$. Inductively, let $m_k = (a_k + b_k)/2$. If some element

of S exceeds m_k , set $a_{k+1} = m_k$, $b_{k+1} = b_k$; otherwise $a_{k+1} = a_k$, $b_{k+1} = m_k$. The a_k form a Cauchy sequence whose limit is $\sup S$. $\square$

Remark

Classical Logic and the Least Upper Bound

The completeness of $\mathbb{R}$ —often stated as “every Cauchy sequence converges” or equivalently “every bounded-above set has a least upper bound”—is *not* a theorem but an axiom that distinguishes $\mathbb{R}$ from $\mathbb{Q}$. We have *constructed* $\mathbb{R}$ to have this property. The remarkable fact is that this property, together with the ordered field axioms, uniquely determines $\mathbb{R}$ up to isomorphism.

Theorem: The Archimedean Property of $\mathbb{R}$

For any $x \in \mathbb{R}$ with $x > 0$, there exists $n \in \mathbb{N}$ with $1/n < x$. Equivalently, $\mathbb{N}$ is not bounded above in $\mathbb{R}$.

Proof. If $\mathbb{N}$ were bounded above, by the LUB property it would have a supremum s . But $s - 1/2$ is not an upper bound (since s is the least), so some $n > s - 1/2$, giving $n + 1 > s$ —contradicting s being an upper bound. $\square$

Exercises

1. Prove that $\mathbb{R}$ is uncountable (Cantor’s diagonal argument).
2. Prove the **nested interval theorem**: if $[a_n, b_n]$ is a nested sequence of closed intervals with $b_n - a_n \rightarrow 0$, then $\bigcap [a_n, b_n]$ is a single point.
3. Show the ordering on $\mathbb{R}$ is well-defined: $[(a_n)] > 0$ iff there exists $\varepsilon \in \mathbb{Q}$, $\varepsilon > 0$, and N such that $a_n > \varepsilon$ for all $n > N$.
4. Prove that $\mathbb{Q}$ is dense in $\mathbb{R}$: for any $x < y$ in $\mathbb{R}$, there exists $q \in \mathbb{Q}$ with $x < q < y$.
5. Implement `mulReal` in Lean 4, proving well-definedness with respect to `CauchyEquiv`.

Part II

Topology of $\mathbb{R}$ and Limits

Chapter 6

Sequences and Convergence

With $\mathbb{R}$ in hand, we can give rigorous meaning to “approaching a limit.” This chapter establishes the language of sequences and convergence that underlies all of calculus.

Remark

Historical Note: The Arithmetisation of Analysis

For most of the eighteenth century, analysis was built on intuitive notions of “infinitely small” quantities. Newton called them *fluxions*; Leibniz called them *differentials*. Both inventors knew these objects were problematic — Newton hedged by speaking of “ultimate ratios” and “first and last quantities,” Leibniz by treating dx as a formal symbol that obeyed algebraic rules. Bishop Berkeley savaged both approaches in 1734 in *The Analyst*, mocking infinitesimals as “the ghosts of departed quantities.”

The response, over the following century, was the **arithmetisation of analysis**: the project of rebuilding calculus entirely in terms of numbers and their properties, without appealing to geometric intuition or infinitesimal quantities. The heroes of this programme are three: **Bolzano** (1817), who first stated the rigorous definition of convergence and continuity; **Cauchy** (1821–1823), whose three textbooks — *Cours d’analyse*, *Résumé des leçons*, and *Leçons sur le calcul différentiel* — gave the first systematic ε - δ treatment of limits, continuity, and derivatives; and **Weierstrass** (1860s–1880s), whose Berlin lectures completed the programme by filling the gaps Cauchy had left (uniform convergence, the pathological functions, and the rigorous construction of $\mathbb{R}$). The ε - N definition we use in this chapter is essentially Cauchy’s, rendered in modern notation by Weierstrass. The notation ε for the “error bound” and δ for the input tolerance appear to have been introduced by Weierstrass; earlier writers used prose descriptions. The symbol $\lim$ for limit is due to Simon L’Huillier (1786), though it was not universally adopted until the early twentieth century.

An important subtlety: Cauchy’s definition of limit was slightly different from ours. He defined $\lim_{n \rightarrow \infty} a_n = L$ to mean “ $a_n - L$ becomes infinitely small,” which required a separate (and never fully rigorous) account of what “infinitely small” meant. Weierstrass replaced this with the purely quantitative $\forall \varepsilon > 0, \exists N, \forall n > N, |a_n - L| < \varepsilon$, which contains no undefined terms at all. This is the version in our Definition box below.

6.1 From `MyReal` to `Mathlib`

Before the first definition of this chapter, an important and deliberate change of gear.

Retiring `MyReal`. The `MyReal` we built in Chapter 5 has served its purpose. You have now seen, in full detail, that the real numbers are not handed down from anywhere: they can be constructed — pairs, fractions, Cauchy sequences, three quotients — from `zero` and `succ`, with every well-definedness debt paid. But pushing *analysis* through that construction would mean months of API-building: every $|\cdot|$ estimate, every field identity, every completeness argument re-proved by hand before we could even state the Intermediate Value Theorem. That work has already been done, once, by a community of hundreds — so from this chapter onward we retire `MyReal` and write `Real` to mean **Mathlib’s** real numbers. This is not a retreat: Mathlib’s `Real` is *the same construction you just carried out* — equivalence classes of Cauchy sequences of rationals (see `Mathlib.Data.Real.Basic`; the quotient lives in `CauSeq.Completion`) — built with more generality and a vastly larger lemma library. You built the engine once by hand; now you get to drive the production model.

What `Mathlib` is. Mathlib is the Lean community’s monolithic mathematics library: over a million lines of machine-checked mathematics, from linear algebra through measure theory to schemes, maintained at <https://github.com/leanprover-community/mathlib4>. Importing it gives us two things. First, *definitions and theorems*: `Real`, `Real.sqrt`, `Real.exp`, `Real.sin`, the absolute-value notation $|x|$, `Finset` sums, and thousands of named lemmas (`abs_add_le`, `abs_mul`, ...). Second, *tactics*: powerful proof-automation such as `linarith` (linear arithmetic), `nlinarith`, `norm_num`, `positivity`, `ring`, `field_simp`, and `push_neg`, none of which exist in core Lean. Each is described in Appendix A the first time we use it.

Setting up a `Mathlib` project. Lake, Lean’s build tool, fetches Mathlib as a dependency. Create a project directory containing a file `lean-toolchain` with the single line `leanprover/lean4:v4.31.0` and a `lakefile.toml` like this:

lakefile.toml

```
name = "verifiedcalculus"
defaultTargets = ["VerifiedCalculus"]

[[require]]
name = "mathlib"
scope = "leanprover-community"
rev = "v4.31.0"

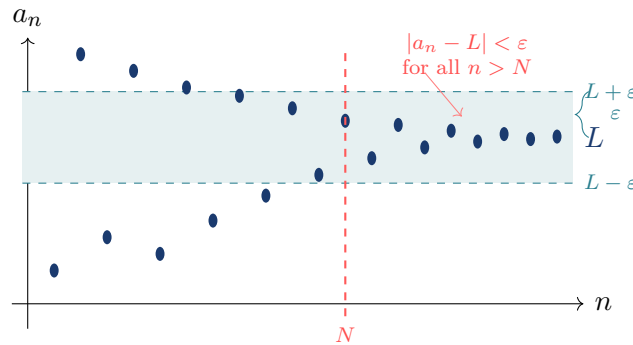
[[lean_lib]]
name = "VerifiedCalculus"
```

Then run `lake update` to resolve the dependency and `lake exe cache get` to download prebuilt binaries — do not skip this step, or your machine will spend hours compiling Mathlib from source. All fetched packages and build artifacts (including the compiled `.olean` files) live under the `.lake/` directory of your project. Start your Lean files with `import Mathlib` and everything in this chapter compiles.

The ground rules from here on. We still *define our own* analysis: `SeqLimit`, `FuncLimit`, `ContinuousAt`, `HasDerivAt` below are ours, written out in ε - δ form, not Mathlib’s versions — the point of the book is to build calculus, not to import it. What we take from Mathlib is the substrate: the real numbers and their ordered-field API, plus the tactics. (One exception is flagged loudly when it happens: in Chapter 16 we state the Fundamental Theorem against Mathlib’s own integral.) When you have finished the book, comparing our definitions with Mathlib’s `Filter`-based ones is a highly instructive exercise.

6.2 Convergence of Sequences

With $\mathbb{R}$ constructed and its completeness established, we can now give rigorous meaning to the phrase “approaching a value.” Informally, a sequence converges to L if the terms eventually stay as close to L as we like. The ε - N definition formalises this: ε is the precision we demand, and N is the threshold beyond which the sequence delivers it. The quantifier order matters: $\forall \varepsilon, \exists N$ means the threshold N is allowed to depend on how tight a precision we ask for. The smaller the ε , the larger N may need to be.



Definition: Convergence of a Sequence

A sequence $(a_n)_{n=1}^{\infty}$ in $\mathbb{R}$ **converges** to $L \in \mathbb{R}$, written $a_n \rightarrow L$ or $\lim_{n \rightarrow \infty} a_n = L$, if:

$$\forall \varepsilon > 0, \exists N \in \mathbb{N}, \forall n > N, |a_n - L| < \varepsilon.$$

A sequence that converges to some L is **convergent**; otherwise **divergent**.

The definition unwinds as: “for every desired precision ε , there is a threshold N beyond which the sequence stays within ε of L .” The key point is that N may depend on ε —smaller ε may require larger N .

Theorem: Limits Are Unique

If $a_n \rightarrow L$ and $a_n \rightarrow M$, then $L = M$.

Proof. Suppose $L \neq M$; set $\varepsilon = |L - M|/2 > 0$. Choose N_1 so $|a_n - L| < \varepsilon$ for $n > N_1$ and N_2 so $|a_n - M| < \varepsilon$ for $n > N_2$. For $n > \max(N_1, N_2)$:

$$|L - M| \leq |L - a_n| + |a_n - M| < \varepsilon + \varepsilon = |L - M|.$$

Contradiction. □

Lean 4

```
def SeqLimit (a : Nat → Real) (L : Real) : Prop :=
  ∀ eps : Real, eps > 0 →
    ∃ N : Nat, ∀ n : Nat, n > N → |a n - L| < eps

-- Limits are unique. The proof mirrors the prose exactly:
-- eps = |L - M|/2, pick an index beyond both thresholds, and let
-- the triangle inequality produce |L - M| < |L - M|.
theorem limit_unique (a : Nat → Real) (L M : Real)
  (hL : SeqLimit a L) (hM : SeqLimit a M) : L = M := by
  by_contra hne
  have heps : |L - M| / 2 > 0 := by
    have h : L - M ≠ 0 := sub_ne_zero.mpr hne
    positivity
  obtain ⟨N1,⟩ hN1 := hL (|L - M| / 2) heps
  obtain ⟨N2,⟩ hN2 := hM (|L - M| / 2) heps
  have h1 := hN1 (max N1 N2 + 1) (by omega)
  have h2 := hN2 (max N1 N2 + 1) (by omega)
  have key : |L - M|
    ≤ |a (max N1 N2 + 1) - L| + |a (max N1 N2 + 1) - M| := by
  calc |L - M|
    = |(L - a (max N1 N2 + 1)) + (a (max N1 N2 + 1) - M)| := by
      ring_nf
    _ ≤ |L - a (max N1 N2 + 1)| + |a (max N1 N2 + 1) - M| :=
      abs_add_le _ _
    _ = |a (max N1 N2 + 1) - L| + |a (max N1 N2 + 1) - M| := by
      rw [abs_sub_comm]
  linarith
```

Three Mathlib tactics make their debut here: `by_contra` turns the goal $L = M$ into a hypothesis $L \neq M$ with goal `False`; `positivity` dispatches the “obviously positive” side-goal; and `linarith` finishes by deriving a contradiction from the linear inequalities in scope ($|L - M| \leq h_1 + h_2$, $h_1 < |L - M|/2$, $h_2 < |L - M|/2$). Appendix A covers all three.

6.3 Algebra of Limits

Once we know individual sequences converge, we want to know whether building new sequences from old ones preserves convergence. The algebra of limits says yes: limits interact well with the four arithmetic operations. This is the machinery that will let us compute derivatives and integrals without returning to ε - N arguments every time — we reduce everything to known limits by algebraic manipulation.

Theorem: Algebra of Limits

If $a_n \rightarrow L$ and $b_n \rightarrow M$ in $\mathbb{R}$, then:

1. $a_n + b_n \rightarrow L + M$
2. $c \cdot a_n \rightarrow cL$ for any constant c
3. $a_n \cdot b_n \rightarrow L \cdot M$
4. $a_n/b_n \rightarrow L/M$ provided $M \neq 0$ (and $b_n \neq 0$ eventually)

Proof. **(1):** Given $\varepsilon > 0$, choose N_1, N_2 so that $|a_n - L| < \varepsilon/2$ and $|b_n - M| < \varepsilon/2$ for $n > N_1, N_2$ respectively. For $n > \max(N_1, N_2)$:

$$|(a_n + b_n) - (L + M)| \leq |a_n - L| + |b_n - M| < \varepsilon.$$

(3): Write $a_n b_n - LM = (a_n - L)b_n + L(b_n - M)$. Use boundedness of (b_n) and convergence of both sequences.

(4): Reduce to showing $1/b_n \rightarrow 1/M$. Since $M \neq 0$, for large n we have $|b_n| \geq |M|/2 > 0$, and:

$$\left| \frac{1}{b_n} - \frac{1}{M} \right| = \frac{|b_n - M|}{|b_n||M|} \leq \frac{2}{|M|^2} |b_n - M|.$$

□

6.4 Important Limit Theorems

Three theorems deserve special attention because they do not follow directly from the algebra of limits but are used constantly. The Squeeze Theorem handles limits we cannot compute directly by trapping the sequence between two simpler ones. The Monotone Convergence Theorem is the sequential counterpart of the least upper bound property — it is the reason that bounded increasing sequences always converge in $\mathbb{R}$ but not in $\mathbb{Q}$. Bolzano–Weierstrass is the compactness statement for sequences: any bounded sequence, however wildly it oscillates, must have a convergent subsequence hiding inside it.

Theorem: Squeeze Theorem

If $a_n \leq c_n \leq b_n$ for all n and $a_n \rightarrow L$, $b_n \rightarrow L$, then $c_n \rightarrow L$.

Proof. Given $\varepsilon > 0$, choose N so $|a_n - L| < \varepsilon$ and $|b_n - L| < \varepsilon$ for $n > N$. Then $L - \varepsilon < a_n \leq c_n \leq b_n < L + \varepsilon$, so $|c_n - L| < \varepsilon$. □

Theorem: Monotone Convergence Theorem

A bounded, monotone sequence in $\mathbb{R}$ converges. Specifically:

- An increasing sequence bounded above converges to its supremum.
- A decreasing sequence bounded below converges to its infimum.

Proof. Let (a_n) be increasing and bounded above. By the LUB property, $L := \sup\{a_n\}$ exists in $\mathbb{R}$. Given $\varepsilon > 0$, since $L - \varepsilon$ is not an upper bound, there exists N with $a_N > L - \varepsilon$. For $n > N$: $L - \varepsilon < a_N \leq a_n \leq L$, so $|a_n - L| < \varepsilon$. $\square$

Theorem: Bolzano–Weierstrass Theorem

Every bounded sequence in $\mathbb{R}$ has a convergent subsequence.

Proof. Let (a_n) be bounded by $[m, M]$. Use the bisection method: at each step, one half of the current interval contains infinitely many terms of the sequence; pick any one as the next subsequence element. This produces a Cauchy subsequence, which converges by completeness. $\square$

Exercises

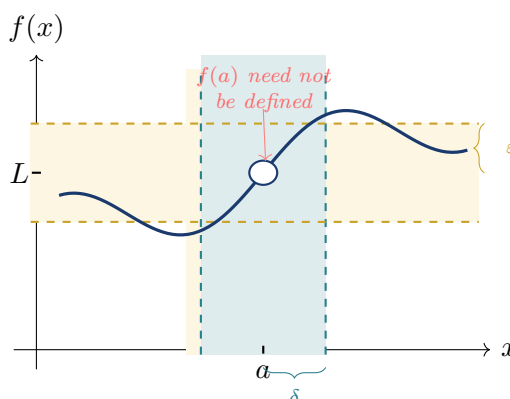
1. Prove parts (2) and (4) of the Algebra of Limits theorem.
2. Show that $1/n^2 \rightarrow 0$ directly from the definition.
3. Prove that any convergent sequence is bounded.
4. Use the Squeeze Theorem to show $\lim_{n \rightarrow \infty} \sin(n)/n = 0$.
5. Prove the Bolzano–Weierstrass theorem in full detail (the bisection argument sketched above).
6. Formalize the Monotone Convergence Theorem in Lean 4 using [SeqLimit](#).

Chapter 7

Limits of Functions

7.1 The ε - δ Definition

Sequential limits described convergence of infinite lists of numbers. Functional limits describe what happens to $f(x)$ as x moves continuously toward a point a . The definition is deliberately agnostic about the value of f at a itself — limits capture approach, not arrival. This separation between the limit and the function value is what makes the concept useful: it lets us study where a function is *heading* even at points where it is undefined or discontinuous.



Definition: Limit of a Function at a Point

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and $a, L \in \mathbb{R}$. We say $\lim_{x \rightarrow a} f(x) = L$ if:

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}, 0 < |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon.$$

Note that $f(a)$ need not be defined, and even if it is, the limit may differ from $f(a)$.

Lean 4

```
def Funclimit (f : Real → Real) (a L : Real) : Prop :=  
  ∀ eps : Real, eps > 0 →  
    ∃ delta : Real, delta > 0 ∧
```

```


$$\forall x : \text{Real}, 0 < |x - a| \rightarrow |x - a| < \delta \rightarrow |f x - L| < \epsilon$$


-- Example:  $\lim_{x \rightarrow 2} (x^2) = 4$ 
-- We need: for  $|x - 2| < \delta$ ,  $|x^2 - 4| < \epsilon$ 
--  $|x^2 - 4| = |x+2||x-2|$ . If  $|x-2| < 1$  then  $|x+2| < 5$ .
-- So take  $\delta = \min(1, \epsilon/5)$ .
example : FuncLimit (fun x => x * x) 2 4 := by
  intro eps heps
  refine (min 1 (eps / 5), by positivity, fun x _ hxd => ?)
  have hx1 : |x - 2| < 1 := lt_of_lt_of_le hxd (min_le_left _)
  have hx5 : |x - 2| < eps/5 := lt_of_lt_of_le hxd (min_le_right _)
  have hbound : |x + 2| < 5 := by
    have h := abs_add_le (x - 2) 4
    have e : x - 2 + 4 = x + 2 := by ring
    rw [e] at h
    have h4 : |(4 : Real)| = 4 := by norm_num
    linarith
  calc |x * x - 4| = |x - 2| * |x + 2| := by rw <[ abs_mul ]; ring_nf
  _ < (eps / 5) * 5 :=
    mul_lt_mul'' hx5 hbound (abs_nonneg _) (abs_nonneg _)
  _ = eps := by ring
    
```

7.2 One-sided Limits and Limits at Infinity

The standard limit requires x to approach a from both sides simultaneously. Sometimes a function behaves differently from the left and the right — the absolute value $|x|/x$ is -1 for $x < 0$ and $+1$ for $x > 0$, so its one-sided limits at 0 differ. Limits at infinity formalise the notion of a horizontal asymptote: $f(x) \rightarrow L$ as $x \rightarrow \infty$ means the graph of f flattens out toward the line $y = L$.

Definition: One-sided Limits

$\lim_{x \rightarrow a^+} f(x) = L$ (right-hand limit): replace $0 < |x - a| < \delta$ by $0 < x - a < \delta$.

$\lim_{x \rightarrow a^-} f(x) = L$ (left-hand limit): replace by $0 < a - x < \delta$.

$\lim_{x \rightarrow \infty} f(x) = L$: $\forall \epsilon > 0, \exists M, \forall x > M, |f(x) - L| < \epsilon$.

Theorem: Sequential Characterization of Limits

$\lim_{x \rightarrow a} f(x) = L$ if and only if for every sequence (x_n) with $x_n \rightarrow a$ and $x_n \neq a$ for all n , we have $f(x_n) \rightarrow L$.

Proof. ($\Rightarrow$): If $f(x) \rightarrow L$ and $x_n \rightarrow a$ ($x_n \neq a$), given $\epsilon > 0$ choose δ for the limit, then N so $|x_n - a| < \delta$ for $n > N$; then $|f(x_n) - L| < \epsilon$.

($\Leftarrow$): Suppose the limit does not hold. Then there exists $\epsilon > 0$ such that for every $\delta > 0$, there exists x with $0 < |x - a| < \delta$ and $|f(x) - L| \geq \epsilon$. Taking

$\delta = 1/n$ gives a sequence $x_n \rightarrow a$ with $|f(x_n) - L| \geq \varepsilon$ for all n —contradicting $f(x_n) \rightarrow L$. $\square$

7.3 Algebra of Functional Limits

The algebra of functional limits is a direct corollary of the sequential version: we proved limits behave well under arithmetic for sequences, and the sequential characterisation of functional limits (the next theorem) transfers this immediately. The key bridge theorem below is worth internalising: it converts every question about functional limits into a question about sequential limits, making the two notions equivalent and letting us move freely between them.

Theorem: Algebra of Functional Limits

If $\lim_{x \rightarrow a} f(x) = L$ and $\lim_{x \rightarrow a} g(x) = M$, then: $(f+g)(x) \rightarrow L+M$, $(fg)(x) \rightarrow LM$, $(f/g)(x) \rightarrow L/M$ (if $M \neq 0$), all as $x \rightarrow a$.

Proof. Reduce to the sequential characterization and apply the Algebra of Sequential Limits from Chapter 6. $\square$

Exercises

1. Prove from the definition: $\lim_{x \rightarrow 3} (2x + 1) = 7$.
2. Prove that $\lim_{x \rightarrow 0} \sin(x)/x = 1$ using the squeeze theorem (hint: geometric argument gives $\cos \theta \leq \sin \theta / \theta \leq 1$ for $\theta \in (0, \pi/2)$).
3. Prove the sequential characterization of limits in full detail.
4. Show that $\lim_{x \rightarrow 0} |x|/x$ does not exist by finding sequences approaching 0 from different sides.
5. Prove: if $\lim_{x \rightarrow a} f(x) = L > 0$, then $f(x) > 0$ for all x sufficiently close to a .

Chapter 8

Continuity

Remark

Historical Note: Continuity and the Pathological Functions

The nineteenth century produced a series of increasingly disturbing examples that forced mathematicians to rethink their intuitions about continuity.

Fourier (1822) showed that a discontinuous function — one with jumps — could be represented as an infinite sum of continuous sine and cosine functions. This raised an immediate question: if the *limit* of continuous functions is not necessarily continuous, what exactly does “continuous” mean? Cauchy claimed (incorrectly) that the *pointwise* limit of continuous functions is continuous. **Abel** found a counterexample in 1826; **Seidel** and **Stokes** independently clarified the issue in 1847–48 by distinguishing uniform from pointwise convergence: it is the *uniform* limit of continuous functions that is guaranteed continuous (the uniform-limit theorem), while pointwise limits can jump.

The next shock came from **Dirichlet** (1829), who defined the function that is 1 on rationals and 0 on irrationals. This function is nowhere continuous — the rationals and irrationals are both dense — yet it is perfectly well-defined pointwise. It is not Riemann integrable (Riemann’s criterion requires the oscillation to be controllable on each subinterval, which fails everywhere for Dirichlet’s function). It *is* Lebesgue integrable, with integral 0, since the rationals form a set of measure zero.

The most spectacular example came from **Weierstrass** in 1872 (announced to the Berlin Academy; published 1875): a function $f(x) = \sum_{n=0}^{\infty} a^n \cos(b^n \pi x)$ where $0 < a < 1$, b is a positive odd integer, and $ab > 1 + 3\pi/2$. This function is *continuous everywhere* but *differentiable nowhere*. Weierstrass’s construction demolished the intuition that a continuous curve must have a tangent “almost everywhere.” Hermite called it “a lamentable scourge” (*un fléau déplorable*). Poincaré complained that such functions were “invented to show that our fathers’ reasoning was at fault.” Today we know that, in a precise sense, the “typical” continuous function is nowhere differentiable — the differentiable functions are a small exception.

These examples are what forced the rigorous ε - δ definitions in this chapter. Without them, you cannot even *state* precisely what it means to be continuous, let alone prove that Dirichlet’s function is discontinuous everywhere or that

Weierstrass's function is continuous everywhere but nowhere differentiable.

8.1 Continuous Functions

A function is continuous at a when the limit and the function value agree — when there is no jump, hole, or oscillation at a . Informally, you can draw the graph without lifting your pen. Formally, continuity is the condition that f *preserves* the notion of closeness: points near a map to points near $f(a)$. This is the natural condition for a function to interact well with limits, and nearly every theorem in analysis will require continuity as a hypothesis.

Definition: Continuity at a Point

$f : \mathbb{R} \rightarrow \mathbb{R}$ is **continuous at** $a \in \mathbb{R}$ if $\lim_{x \rightarrow a} f(x) = f(a)$, i.e.:

$$\forall \varepsilon > 0, \exists \delta > 0, |x - a| < \delta \Rightarrow |f(x) - f(a)| < \varepsilon.$$

f is **continuous on** $S \subseteq \mathbb{R}$ if it is continuous at every point of S .

The three conditions the definition packages together: $f(a)$ is defined, $\lim_{x \rightarrow a} f(x)$ exists, and they are equal.

Lean 4

```
def ContinuousAt (f : Real → Real) (a : Real) : Prop :=
  ∀ eps : Real, eps > 0 →
    ∃ delta : Real, delta > 0 ∧
      ∀ x : Real, |x - a| < delta → |f x - f a| < eps

-- Polynomials are continuous: x^2 is continuous at every a
theorem sq_continuous_at (a : Real) : ContinuousAt (fun x => x * x) a :=
  by
    intro eps heps
    -- |x^2 - a^2| = |x-a||x+a|. If |x-a| < 1, |x+a| ≤ |x-a| + 2|a| < 2|a| + 1
    refine (min 1 (eps / (2 * |a| + 2))), by positivity, fun x hx => ?_
    have hdl : |x - a| < 1 := lt_of_lt_of_le hx (min_le_left _ _)
    have hde : |x - a| < eps / (2 * |a| + 2) :=
      lt_of_lt_of_le hx (min_le_right _ _)
    have hxa : |x + a| < 2 * |a| + 1 := by
      have h : |x + a| ≤ |x - a| + 2 * |a| := by
        calc |x + a| = |(x - a) + 2 * a| := by ring_nf
        _ ≤ |x - a| + |2 * a| := abs_add_le _ _
        _ = |x - a| + 2 * |a| := by rw [abs_mul]; norm_num
      linarith
    calc |x * x - a * a|
      = |x - a| * |x + a| := by rw <[ abs_mul ]; ring_nf
      _ < (eps / (2 * |a| + 2)) * (2 * |a| + 2) :=
```

```
mul_lt_mul'' hde (by linarith) (abs_nonneg _) (abs_nonneg _)
_ = eps := ⋈div_mul_cancel eps (by positivity)
```

8.2 Properties of Continuous Functions

Continuous functions are closed under the basic constructions of analysis: sums, products, quotients, and compositions. This means that once we know a handful of basic functions are continuous — polynomials, $\sin$, $\cos$, e^x — the entire zoo of functions built from them by algebra and composition is automatically continuous wherever it is defined. We never need to return to the ε - δ definition for a composed function.

Theorem: Algebra of Continuous Functions

If f and g are continuous at a , then $f + g$, fg , cf (for $c \in \mathbb{R}$), and f/g (when $g(a) \neq 0$) are continuous at a .

Theorem: Composition of Continuous Functions

If g is continuous at a and f is continuous at $g(a)$, then $f \circ g$ is continuous at a .

Proof. Given $\varepsilon > 0$, choose $\delta_1 > 0$ so $|y - g(a)| < \delta_1 \Rightarrow |f(y) - f(g(a))| < \varepsilon$. Choose $\delta > 0$ so $|x - a| < \delta \Rightarrow |g(x) - g(a)| < \delta_1$. Then $|x - a| < \delta \Rightarrow |f(g(x)) - f(g(a))| < \varepsilon$. $\square$

8.3 The Intermediate Value Theorem

The Intermediate Value Theorem captures the intuition that a continuous function cannot skip values: if it starts at $f(a)$ and ends at $f(b)$, it must pass through every value in between. This is a purely topological fact — it follows from the connectedness of $[a, b]$ — and it is one of the deepest consequences of the completeness of $\mathbb{R}$. The same theorem fails for $\mathbb{Q}$: the function $f(q) = q^2 - 2$ satisfies $f(1) < 0 < f(2)$, yet $f(q) = 0$ has no rational solution.

Theorem: Intermediate Value Theorem (IVT)

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous and $f(a) < v < f(b)$ (or $f(b) < v < f(a)$), then there exists $c \in (a, b)$ with $f(c) = v$.

Proof. Assume $f(a) < v < f(b)$. Consider $S = \{x \in [a, b] : f(x) \leq v\}$. S is nonempty (contains a) and bounded above by b , so $c := \sup S$ exists. We claim $f(c) = v$.

If $f(c) < v$: by continuity, $f(x) < v$ for x near c , so some $c' > c$ has $f(c') < v$, contradicting $c = \sup S$.

If $f(c) > v$: similarly, $f(x) > v$ near c , so $c - \delta$ is an upper bound for S for small δ —contradicting $c = \sup S$.

Hence $f(c) = v$. $\square$

Corollary: Existence of n -th Roots

For any $y > 0$ and any integer $n \geq 1$, $y^{1/n}$ exists as a real number.

Proof. Apply IVT to $f(x) = x^n$ on $[0, 1+y]$: $f(0) = 0 < y \leq (1+y)^n = f(1+y)$. $\square$

8.4 Uniform Continuity

Ordinary continuity says the tolerance δ for a given precision ε may depend on the point x we are near. Uniform continuity says δ can be chosen to work *everywhere on the domain at once*. This is a strictly stronger condition: $f(x) = 1/x$ is continuous on $(0, 1)$ but not uniformly so, because near 0 you need a much smaller δ than near 1. Uniform continuity is what integration needs: to approximate $\int_a^b f$ by Riemann sums, we need the oscillation of f on each subinterval to be uniformly small, not just pointwise small.

Definition: Uniform Continuity

f is **uniformly continuous** on S if the δ in the continuity definition can be chosen independently of x :

$$\forall \varepsilon > 0, \exists \delta > 0, \forall x, y \in S, |x - y| < \delta \Rightarrow |f(x) - f(y)| < \varepsilon.$$

Theorem: Continuous Functions on Closed Intervals Are Uniformly Continuous

If f is continuous on $[a, b]$, it is uniformly continuous on $[a, b]$.

Proof. Suppose not. Then there exists $\varepsilon > 0$ and sequences x_n, y_n in $[a, b]$ with $|x_n - y_n| < 1/n$ but $|f(x_n) - f(y_n)| \geq \varepsilon$. By Bolzano–Weierstrass, (x_n) has a convergent subsequence $x_{n_k} \rightarrow c \in [a, b]$. Then $y_{n_k} \rightarrow c$ as well. By continuity at c , $f(x_{n_k}) \rightarrow f(c)$ and $f(y_{n_k}) \rightarrow f(c)$, so $|f(x_{n_k}) - f(y_{n_k})| \rightarrow 0$ —contradicting $\geq \varepsilon$. $\square$

8.5 Pathological Examples

Not every function is continuous. Two canonical examples:

Example 8.1 (Dirichlet’s Function). $f(x) = \begin{cases} 1 & x \in \mathbb{Q} \\ 0 & x \notin \mathbb{Q} \end{cases}$ is continuous nowhere.

For any $a \in \mathbb{R}$ and $\delta > 0$, the interval $(a - \delta, a + \delta)$ contains both rationals and irrationals, so f oscillates between 0 and 1.

Example 8.2 (The Ruler Function). $f(x) = \begin{cases} 1/q & x = p/q \text{ in lowest terms} \\ 0 & x \notin \mathbb{Q} \end{cases}$ is continuous at every irrational, discontinuous at every rational.

Exercises

1. Prove that $f(x) = \sqrt{x}$ is uniformly continuous on $[0, \infty)$.
2. Prove the Extreme Value Theorem: if f is continuous on $[a, b]$, then f attains a maximum and minimum value.
3. Show that $f(x) = 1/x$ is continuous but not uniformly continuous on $(0, 1)$.
4. Use the IVT to prove that every odd-degree polynomial with real coefficients has at least one real root.
5. Formalize continuity of $f(x) = x^3$ at an arbitrary point a in Lean 4.
6. Prove the fixed-point theorem: if $f : [0, 1] \rightarrow [0, 1]$ is continuous, then there exists c with $f(c) = c$.

Chapter 9

Topology of $\mathbb{R}$

The theorems of the previous chapter (IVT, EVT, uniform continuity) all relied on the completeness of $\mathbb{R}$ and properties of $[a, b]$ as a domain. This chapter makes those structural properties precise via the language of point-set topology.

9.1 Open and Closed Sets

The theorems of the previous chapter relied on specific properties of $[a, b]$: it is bounded, it contains its endpoints, and it is connected. Point-set topology gives us precise language for these properties. An open set is one where every point has breathing room — a small neighbourhood entirely inside the set. A closed set contains all the limit points it could ever accumulate. These two notions are dual, and together they provide the vocabulary for stating the deepest theorems of analysis in their most general form.

Definition: Open and Closed Subsets of $\mathbb{R}$

A set $U \subseteq \mathbb{R}$ is **open** if every point has a neighborhood contained in U :

$$\forall x \in U, \exists \varepsilon > 0, (x - \varepsilon, x + \varepsilon) \subseteq U.$$

A set $F \subseteq \mathbb{R}$ is **closed** if its complement $\mathbb{R} \setminus F$ is open, equivalently if F contains all its limit points.

Proposition: Basic Topological Properties

1. $\emptyset$ and $\mathbb{R}$ are both open and closed.
2. Arbitrary unions of open sets are open.
3. Finite intersections of open sets are open.
4. Arbitrary intersections of closed sets are closed.
5. Finite unions of closed sets are closed.

9.2 Compactness

Compactness is the topological abstraction of “closed and bounded,” and it is the property that makes analysis work on finite-length intervals. The key feature of compact sets is that every open cover has a *finite* subcover: you cannot cover $[a, b]$ with infinitely many open sets without finitely many of them already doing the job. This finiteness is what lets us extract uniform bounds and guarantees from pointwise ones — it is the engine behind the Extreme Value Theorem, uniform continuity, and the completeness of the Riemann integral.

Definition: Compactness

A set $K \subseteq \mathbb{R}$ is **compact** if every open cover of K has a finite subcover: if $K \subseteq \bigcup_{\alpha} U_{\alpha}$ (each U_{α} open), then $K \subseteq U_{\alpha_1} \cup \cdots \cup U_{\alpha_n}$ for finitely many α_i . Equivalently (in $\mathbb{R}$): K is compact iff K is **closed and bounded** (Heine–Borel theorem).

Theorem: Heine–Borel Theorem

A subset $K \subseteq \mathbb{R}$ is compact if and only if it is closed and bounded.

Proof. ($\Rightarrow$): Compact sets are closed and bounded (the argument works in any metric space — any set with a distance function, of which $\mathbb{R}$ with $d(x, y) = |x - y|$ is the basic example). Boundedness: the cover $\{(-n, n)\}_{n \geq 1}$ has a finite subcover. Closure: if x is a limit point not in K , the sets $\mathbb{R} \setminus \{y : |y - x| \leq 1/n\}$ cover K but no finite subcollection does.

($\Leftarrow$): Closed and bounded in $\mathbb{R}$ implies compact. The key lemma (proved by the bisection method, using completeness) is that $[a, b]$ is compact; a closed subset of a compact set is compact. $\square$

Theorem: Extreme Value Theorem

If $f : K \rightarrow \mathbb{R}$ is continuous and K is compact, then f attains its maximum and minimum on K .

Proof. The image $f(K)$ is compact (continuous images of compact sets are compact). A compact subset of $\mathbb{R}$ is closed and bounded, hence has a maximum and minimum, both achieved. $\square$

9.3 Connectedness

A connected set cannot be split into two separated open pieces. Intuitively, it is “in one piece.” In $\mathbb{R}$, the connected sets are exactly the intervals — a fact that is not obvious from the definition but is a theorem. Connectedness is what powers the Intermediate Value Theorem: the image of a connected set under a continuous function is connected, so it must itself be an interval, and therefore contains all values between its endpoints.

Definition: Connectedness

A set $S \subseteq \mathbb{R}$ is **connected** if it cannot be written as a disjoint union $S = A \cup B$ of two nonempty sets that are *open in S* — that is, $A = S \cap U$ and $B = S \cap V$ for open sets $U, V \subseteq \mathbb{R}$ (this is the **subspace topology** on S). The relativisation matters: $S = [0, 1] \cup [2, 3]$ is visibly disconnected, yet neither piece is open in $\mathbb{R}$; each *is* open in S (take $U = (-1, 1.5)$ and $V = (1.5, 4)$). The connected subsets of $\mathbb{R}$ are exactly the intervals (including rays and $\mathbb{R}$ itself).

Theorem: Continuous Images of Connected Sets Are Connected

If f is continuous and S is connected, then $f(S)$ is connected.

Proof. Suppose $f(S) = A \cup B$ with A, B nonempty, disjoint, and open in $f(S)$, say $A = f(S) \cap U$ and $B = f(S) \cap V$ with U, V open. Then $S = (S \cap f^{-1}(U)) \cup (S \cap f^{-1}(V))$, a disjoint union of nonempty sets open in S (preimages of open sets are open, by continuity of f). This contradicts the connectedness of S . $\square$

Corollary: Intermediate Value Theorem (Topological Proof)

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous and v is between $f(a)$ and $f(b)$, then $v \in f([a, b])$.

Proof. $[a, b]$ is connected, so $f([a, b])$ is connected, hence an interval. Since $f(a)$ and $f(b)$ are in this interval and v is between them, v is in the interval. $\square$

Exercises

1. Prove that (a, b) is open and $[a, b]$ is closed using the definitions.
2. Prove that an infinite intersection of open sets need not be open (example: $\bigcap_{n=1}^{\infty} (-1/n, 1/n) = \{0\}$).
3. Prove that compact subsets of $\mathbb{R}$ are closed and bounded.
4. Prove: a continuous image of a compact set is compact.
5. Show that $[0, 1]$ and $(0, 1)$ are not homeomorphic (hint: consider what happens when you remove a point).
6. Prove that every continuous $f : [a, b] \rightarrow [a, b]$ has a fixed point (Brouwer in 1D).

Part III

Differential Calculus

Chapter 10

The Derivative

Remark

Historical Note: Newton, Leibniz, and the Priority Dispute

The invention of calculus is one of the most famous priority disputes in the history of science. **Isaac Newton** developed his method of “fluxions” in 1665–66, during the plague years when Cambridge University was closed and he retreated to his family farm in Woolsthorpe. He did not publish his results until decades later. **Gottfried Wilhelm Leibniz** independently developed his calculus in 1675–76, invented the notation dy/dx and $\int$, and published his results in 1684 and 1686 — decades before Newton’s publication.

Newton’s priority over Leibniz in *discovery* is not disputed: his notebooks are dated. But Leibniz’s priority in *publication* and the superiority of his *notation* are equally clear. The dispute, which consumed both men’s later years and poisoned English and Continental mathematics for a century, was fuelled by nationalist pride and the accusation (almost certainly false) that Leibniz had seen Newton’s unpublished manuscripts during a visit to London in 1676. The Royal Society investigated in 1712 and, in a verdict widely regarded as dishonest, found for Newton — largely because Newton, as President of the Royal Society, effectively controlled the investigation.

The mathematical cost was severe: British mathematicians clung to Newton’s cumbersome “dot” notation ($\dot{x}$, $\ddot{x}$) out of loyalty, while the Continent used Leibniz’s far more flexible dy/dx notation. This notational disadvantage held British mathematics back for over a century. The Cambridge Analytical Society, founded in 1812 by Babbage, Herschel, and Peacock, explicitly aimed to replace the “d-ism of Leibniz” for the “dot-age of Newton” — their pun, not ours.

The notation we use in this chapter is Lagrange’s: $f'(x)$ for the derivative. Lagrange introduced this in his 1797 *Théorie des fonctions analytiques*, where he also introduced the term “derivative” (*fonction dérivée*). Lagrange’s approach was to define the derivative via Taylor series rather than limits, which is logically backwards (you need the derivative to define the Taylor series) but has the virtue of being computationally clean.

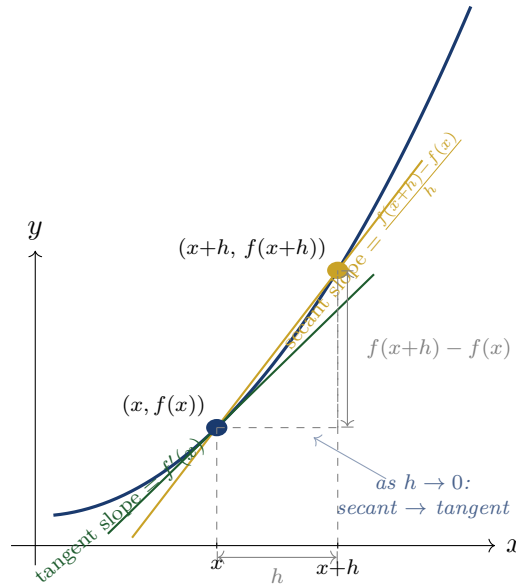
The derivative formalizes the notion of instantaneous rate of change. Geometrically, it is the slope of the tangent line to a curve at a point; analytically, it is a

limit of difference quotients.

10.1 The Difference Quotient and the Derivative

The derivative formalises instantaneous rate of change. The idea is to approximate the rate of change on a small interval $[x, x+h]$ by the slope $(f(x+h) - f(x))/h$ of the secant line, then take the limit as $h \rightarrow 0$. If this limit exists, the function has a well-defined instantaneous slope at x — its tangent line slope.

The “local linearity” formulation $f(x+h) \approx f(x) + f'(x)h$ is in some ways more fundamental: it says differentiability is equivalent to the existence of a good linear approximation. This is the form that generalises cleanly to $\mathbb{R}^n$, where the derivative becomes a linear map rather than a number.



Definition: The Derivative at a Point

Let $f : (a, b) \rightarrow \mathbb{R}$ and $x \in (a, b)$. The **derivative** of f at x is:

$$f'(x) := \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h},$$

provided this limit exists. If it does, f is **differentiable at x** .

Equivalently, $f'(x) = L$ if $\lim_{h \rightarrow 0} \frac{f(x+h) - f(x) - Lh}{h} = 0$, i.e., f is “locally linear” with slope L .

This “local linearity” formulation will generalize cleanly to $\mathbb{R}^n$. We record it as a separate characterization:

Proposition: Local Linearity Characterization

$f'(x) = L$ if and only if $f(x+h) = f(x) + Lh + o(h)$ as $h \rightarrow 0$, where $o(h)$ means the error divided by h goes to zero.

Theorem: Differentiability Implies Continuity

If f is differentiable at x , then f is continuous at x .

Proof. $f(x + h) - f(x) = \frac{f(x+h)-f(x)}{h} \cdot h \rightarrow f'(x) \cdot 0 = 0$ as $h \rightarrow 0$. $\square$

Remark

The converse fails: $f(x) = |x|$ is continuous at 0 but not differentiable there (left derivative = -1 , right derivative = 1). Even more dramatically, Weierstrass constructed a function continuous everywhere but differentiable nowhere.

Lean 4

```
def HasDerivAt (f : Real → Real) (x L : Real) : Prop :=
  ∀ eps : Real, eps > 0 →
    ∃ delta : Real, delta > 0 ∧
      ∀ h : Real, |h| < delta → h ≠ 0 →
        |(f (x + h) - f x) / h - L| < eps

-- The derivative of a constant is zero: the difference quotient
-- (c - c)/h is literally 0, so simp closes everything.
theorem const_deriv (c x : Real) : HasDerivAt (fun _ => c) x 0 := by
  intro eps heps
  refine ⟨1, one_pos, fun h _ _ => ?⟩_
  simp using heps

-- The derivative of x is 1: the quotient is h/h = 1 (h ≠ 0!).
theorem id_deriv (x : Real) : HasDerivAt id x 1 := by
  intro eps heps
  refine ⟨1, one_pos, fun h _ hne => ?⟩_
  simp [div_self hne] using heps

-- Differentiability implies continuity
theorem differentiable_implies_continuous (f : Real → Real) (x L : Real)
  (hf : HasDerivAt f x L) : ContinuousAt f x := by
  intro eps heps
  obtain ⟨delta, hd, hfnc := hf (eps / (|L| + 1))
    (div_pos heps (by linarith [abs_nonneg L]))⟩
  refine ⟨min delta 1, by positivity, fun y hy => ?⟩_
  sorry -- full proof uses: f(x+h) - f(x) = h * ((f(x+h)-f(x))/h) → 0
```

10.2 Derivatives of Elementary Functions

We compute the derivatives of the basic functions, which will become the foundation of the differentiation engine in Part VII.

Theorem: Derivatives of Power Functions

For $n \in \mathbb{N}$, $(x^n)' = nx^{n-1}$, with the convention $x^0 = 1$.

Proof. By the binomial theorem:

$$\frac{(x+h)^n - x^n}{h} = \frac{\sum_{k=0}^n \binom{n}{k} x^k h^{n-k} - x^n}{h} = nx^{n-1} + \binom{n}{2} x^{n-2} h + \dots \rightarrow nx^{n-1}.$$

□

Theorem: Derivatives of Trigonometric Functions

$(\sin x)' = \cos x$ and $(\cos x)' = -\sin x$.

Proof. Using the angle-addition formula and the limits $\lim_{h \rightarrow 0} \sin(h)/h = 1$ and $\lim_{h \rightarrow 0} (1 - \cos h)/h = 0$:

$$\frac{\sin(x+h) - \sin x}{h} = \cos x \cdot \frac{\sin h}{h} - \sin x \cdot \frac{1 - \cos h}{h} \rightarrow \cos x.$$

□

Theorem: Derivative of e^x

$(e^x)' = e^x$.

Proof. $(e^{x+h} - e^x)/h = e^x(e^h - 1)/h \rightarrow e^x \cdot 1$, where $\lim_{h \rightarrow 0} (e^h - 1)/h = 1$ follows from the power series $e^h = 1 + h + h^2/2! + \dots$. □

Remark

An honest accounting: where do $\sin$, $\cos$, and e^x come from?

Notice that the last two proofs *consume* facts we have not yet established: the angle-addition formula, the limits $\sin(h)/h \rightarrow 1$ and $(e^h - 1)/h \rightarrow 1$, indeed the very definitions of $\sin$, $\cos$, and e^x . This book does not construct these functions from first principles. The standard rigorous route — which we sketch but do not complete — is to *define* them by their power series ($e^x = \sum x^n/n!$, etc.), prove the series converge everywhere (Chapter 20), and derive the derivative rules by term-by-term differentiation (Chapters 20–21). In Lean, this is exactly what Mathlib does: `Real.exp`, `Real.sin`, and `Real.cos` are defined via power series (over $\mathbb{C}$, then restricted), and the rules above are library theorems (`Real.hasDerivAt_sin` and friends). When the capstone's differentiation engine applies $(\sin u)' = \cos(u) u'$, it is these theorems that back the rule.

Exercises

1. Prove $(\ln x)' = 1/x$ for $x > 0$.
2. Show directly from the definition that $(|x|)' = \text{sgn}(x)$ for $x \neq 0$ and that $|x|$ is not differentiable at 0.
3. Prove $(x^n)' = nx^{n-1}$ for negative integers n (where $x \neq 0$).
4. Prove $(\tan x)' = \sec^2 x$ using $\tan x = \sin x / \cos x$ and the quotient rule from the next chapter.
5. Formalize `HasDerivAt (fun x => x * x * x) a (3 * a * a)` in Lean 4.

Chapter 11

Differentiation Rules

11.1 The Four Fundamental Rules

Every differentiation rule is a theorem — a consequence of the limit definition. We do not assume products differentiate by the product rule; we prove it. These four rules (sum, product, quotient, chain) form a complete toolkit: any expression built from elementary functions by these four operations can be differentiated algorithmically by pattern-matching against the rules. This is precisely what the differentiation engine in Part VII will do.

Theorem: Sum Rule

If f and g are differentiable at x , then $(f + g)'(x) = f'(x) + g'(x)$.

Proof. $\frac{(f(x+h)+g(x+h))-(f(x)+g(x))}{h} = \frac{f(x+h)-f(x)}{h} + \frac{g(x+h)-g(x)}{h} \rightarrow f'(x) + g'(x)$. $\square$

Theorem: Product Rule

$(fg)'(x) = f'(x)g(x) + f(x)g'(x)$.

Proof.

$$\frac{f(x+h)g(x+h) - f(x)g(x)}{h} = \frac{f(x+h) - f(x)}{h} \cdot g(x+h) + f(x) \cdot \frac{g(x+h) - g(x)}{h}.$$

As $h \rightarrow 0$: first factor $\rightarrow f'(x)$, $g(x+h) \rightarrow g(x)$ (by continuity), second $\rightarrow f(x)g'(x)$. $\square$

Theorem: Quotient Rule

If $g(x) \neq 0$, then $(f/g)'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}$.

Proof. Write $f/g = f \cdot (1/g)$ and apply the product rule, after computing $(1/g)'(x) = -g'(x)/g(x)^2$. $\square$

Theorem: Chain Rule

If g is differentiable at x and f is differentiable at $g(x)$, then:

$$(f \circ g)'(x) = f'(g(x)) \cdot g'(x).$$

Proof. The naive proof (multiplying and dividing by $g(x+h) - g(x)$) fails when $g(x+h) = g(x)$. The correct proof uses the local linearity characterization: write $f(g(x) + k) = f(g(x)) + f'(g(x))k + \varepsilon(k)k$ where $\varepsilon(k) \rightarrow 0$. Setting $k = g(x+h) - g(x) = g'(x)h + \eta(h)h$:

$$f(g(x+h)) - f(g(x)) = f'(g(x))(g'(x)h + \eta h) + \varepsilon(k)(g'(x)h + \eta h).$$

Dividing by h and taking $h \rightarrow 0$ gives $f'(g(x))g'(x)$. □

Lean 4

```
-- Product rule
theorem product_rule (f g : Real → Real) (x Lf Lg : Real)
  (hf : HasDerivAt f x Lf) (hg : HasDerivAt g x Lg) :
  HasDerivAt (fun x => f x * g x) x (Lf * g x + f x * Lg) := by
  intro eps heps
  -- Need: |(f(x+h)g(x+h) - f(x)g(x))/h - (Lf * g(x) + f(x) * Lg)| < eps
  -- Use: f(x+h)g(x+h) - f(x)g(x) = (f(x+h)-f(x))g(x+h) + f(x)(g(x+h)-g(x))
  sorry -- Full proof requires bounding g near x

-- Chain rule
theorem chain_rule (f g : Real → Real) (x Lf Lg : Real)
  (hg : HasDerivAt g x Lg) (hf : HasDerivAt f (g x) Lf) :
  HasDerivAt (f ∘ g) x (Lf * Lg) := by
  sorry -- Requires the local linearity formulation
```

11.2 Implicit and Logarithmic Differentiation

Sometimes a function is defined implicitly by a relation $F(x, y) = 0$ rather than by an explicit formula $y = f(x)$.

Example 11.1 (Implicit Differentiation). The unit circle $x^2 + y^2 = 1$ defines y implicitly as a function of x near any point with $y \neq 0$. Differentiating both sides with respect to x : $2x + 2y dy/dx = 0$, so $dy/dx = -x/y$.

Example 11.2 (Logarithmic Differentiation). To differentiate $f(x) = x^x$ for $x > 0$: let $y = x^x$, so $\ln y = x \ln x$. Differentiating: $y'/y = \ln x + 1$, so $y' = x^x(\ln x + 1)$.

Theorem: Inverse Function Theorem (1D)

If f is differentiable and strictly monotone on (a, b) with $f'(c) \neq 0$, then f^{-1} is differentiable at $f(c)$ with:

$$(f^{-1})'(f(c)) = \frac{1}{f'(c)}.$$

Proof. Let $y = f(c)$, $k = f(c + h) - f(c)$. As $h \rightarrow 0$, $k \rightarrow 0$ (by continuity). Then:

$$(f^{-1})'(y) = \lim_{k \rightarrow 0} \frac{f^{-1}(y + k) - f^{-1}(y)}{k} = \lim_{h \rightarrow 0} \frac{h}{f(c + h) - f(c)} = \frac{1}{f'(c)}.$$

□

Exercises

1. Use the inverse function theorem to prove $(\arcsin x)' = 1/\sqrt{1 - x^2}$.
2. Prove $(a^x)' = a^x \ln a$ for $a > 0$ using logarithmic differentiation.
3. Prove the general power rule $(x^r)' = rx^{r-1}$ for real r (write $x^r = e^{r \ln x}$).
4. Differentiate $y = (\sin x)^{\cos x}$ using logarithmic differentiation.
5. Formalize the sum rule in Lean 4: if `HasDerivAt f x Lf` and `HasDerivAt g x Lg`, then `HasDerivAt (fun x => f x + g x) x (Lf + Lg)`.

Chapter 12

Mean Value Theorems

12.1 Rolle's Theorem and the MVT

The Mean Value Theorem is one of the most useful theorems in calculus: it says that the average rate of change over an interval $[a, b]$ is achieved as an instantaneous rate at some interior point. Geometrically, the secant line from $(a, f(a))$ to $(b, f(b))$ has the same slope as some tangent line. Rolle's theorem is the special case where the endpoints have equal values, and the MVT follows from it by a linear correction.

Theorem: Rolle's Theorem

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous, differentiable on (a, b) , and $f(a) = f(b)$, then there exists $c \in (a, b)$ with $f'(c) = 0$.

Proof. By EVT, f attains its maximum M and minimum m on $[a, b]$. If $M = m$, f is constant and $f' \equiv 0$. Otherwise, at least one extremum is attained at an interior point $c \in (a, b)$. At an interior maximum, $f'(c) \geq 0$ (from the left) and $f'(c) \leq 0$ (from the right), so $f'(c) = 0$. $\square$

Theorem: Mean Value Theorem (MVT)

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous and differentiable on (a, b) , then there exists $c \in (a, b)$ with:

$$f'(c) = \frac{f(b) - f(a)}{b - a}.$$

Proof. Define $g(x) = f(x) - \frac{f(b)-f(a)}{b-a}(x-a)$. Then $g(a) = f(a) = g(b)$. By Rolle's theorem, $g'(c) = 0$ for some c , i.e., $f'(c) = \frac{f(b)-f(a)}{b-a}$. $\square$

Corollary: Monotonicity via the Derivative

If $f'(x) > 0$ on (a, b) , then f is strictly increasing. If $f'(x) < 0$, strictly decreasing. If $f'(x) = 0$ everywhere on (a, b) , then f is constant.

Proof. For $a < x < y < b$: by MVT, $f(y) - f(x) = f'(c)(y - x)$ for some $c \in (x, y)$. The sign of $f'(c)$ determines whether $f(y) > f(x)$ or $f(y) < f(x)$. $\square$

12.2 L'Hôpital's Rule

L'Hôpital's rule resolves indeterminate forms like $0/0$ or ∞/∞ by replacing the ratio of functions with the ratio of their derivatives. The intuition: near the point where both numerator and denominator vanish, the dominant behaviour is determined by first-order Taylor terms, whose ratio is the ratio of derivatives. The rule is powerful but requires care: it applies only to genuine indeterminate forms, and $\lim f'/g'$ must actually exist.

Theorem: L'Hôpital's Rule

Suppose $\lim_{x \rightarrow a} f(x) = \lim_{x \rightarrow a} g(x) = 0$ (or both $\pm\infty$), and $g'(x) \neq 0$ near a . If $\lim_{x \rightarrow a} f'(x)/g'(x) = L$, then $\lim_{x \rightarrow a} f(x)/g(x) = L$.

Proof. In the $0/0$ case: define $f(a) = g(a) = 0$. For x near a , by Cauchy's MVT there exists c between a and x with $\frac{f(x)}{g(x)} = \frac{f(x)-f(a)}{g(x)-g(a)} = \frac{f'(c)}{g'(c)}$. As $x \rightarrow a$, $c \rightarrow a$, so the ratio approaches L . $\square$

Exercises

1. Use L'Hôpital to evaluate $\lim_{x \rightarrow 0} (\sin x)/x$ and $\lim_{x \rightarrow 0} (e^x - 1 - x)/x^2$.
2. Prove: if $f'(x) \geq 0$ on (a, b) and f' is not identically zero, then f is strictly increasing.
3. Use the MVT to prove the AM-GM inequality: for $a, b > 0$, $\sqrt{ab} \leq (a + b)/2$.
4. Prove Cauchy's MVT: if f, g are continuous on $[a, b]$, differentiable on (a, b) , and $g'(x) \neq 0$ on (a, b) , then there exists c with $f'(c)/g'(c) = (f(b) - f(a))/(g(b) - g(a))$.

Chapter 13

Higher Derivatives and Taylor's Theorem

Remark

Historical Note: Taylor, Maclaurin, and the Power Series Debate

Brook Taylor published his series in 1715, claiming (as was common at the time) more generality than he actually proved. The series had been known earlier: James Gregory had used it in 1671, and Leibniz had the expansion for $\arctan$ in 1673. What Taylor contributed was a systematic account treating the series as a general tool rather than a collection of special cases.

Colin Maclaurin, whose name is attached to the special case $a = 0$, never claimed to have discovered it — he explicitly credited Taylor. The Maclaurin series appears in his 1742 *Treatise of Fluxions*, written partly to defend Newton's calculus against Berkeley's philosophical attacks, and the special case became associated with his name through the accident of emphasis rather than discovery.

The question of *when* a smooth function equals its Taylor series — the issue of *analyticity* versus mere smoothness — was not clearly understood until the nineteenth century. Cauchy constructed the now-standard counterexample: $f(x) = e^{-1/x^2}$ for $x \neq 0$ and $f(0) = 0$ is infinitely differentiable everywhere, yet its Taylor series at 0 is identically zero, so the series converges but not to f . This function is smooth (C^∞) but not analytic at 0.

The distinction between smooth and analytic functions is fundamental in complex analysis: every holomorphic (complex-differentiable) function is automatically analytic, which is one of the remarkable rigidity properties of complex analysis. In real analysis, smoothness is far weaker than analyticity, and the gap between them is inhabited by important functions like bump functions and cutoff functions used in differential geometry and partial differential equations. The Lagrange remainder formula we prove — bounding $|R_n(x)|$ by $M|x - a|^{n+1}/(n+1)!$ — was Lagrange's main contribution to Taylor series theory. It turned Taylor series from a formal tool into a theorem with an error bound, making numerical approximations rigorous.

13.1 Higher-Order Derivatives

If the derivative f' is itself differentiable, we get the second derivative $f'' = (f')'$, measuring how the rate of change is itself changing. For a position function, f' is velocity and f'' is acceleration. Iterating gives higher-order derivatives. The smoothness class C^k records how many times a function can be differentiated and still have a continuous result. C^∞ (infinitely differentiable) is the natural domain for Taylor series.

Definition: Higher-Order Derivatives

The n -th derivative $f^{(n)}$ is defined inductively: $f^{(0)} = f$ and $f^{(n)} = (f^{(n-1)})'$. f is of class C^k on an interval if $f^{(k)}$ exists and is continuous. f is **smooth** (C^∞) if all derivatives exist.

13.2 Taylor's Theorem

Taylor's theorem gives the best polynomial approximation to a smooth function near a point, together with a precise bound on the error. It explains why tables of sines and logarithms work: $\sin x \approx x - x^3/6$ for small x because the error is bounded by $|x|^5/120$. The Lagrange remainder turns an approximation into a theorem with a quantitative error bound, and it will be our main tool for proving that power series converge to the functions they represent.

Theorem: Taylor's Theorem with Lagrange Remainder

If f is $n+1$ times differentiable on $(a-r, a+r)$, then for each x in this interval:

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(a)}{k!} (x-a)^k + R_n(x),$$

where the **Lagrange remainder** satisfies $R_n(x) = \frac{f^{(n+1)}(c)}{(n+1)!} (x-a)^{n+1}$ for some c between a and x .

Proof. Define $g(t) = f(x) - \sum_{k=0}^n \frac{f^{(k)}(t)}{k!} (x-t)^k$. Then $g(x) = 0$ and $g'(t) = -\frac{f^{(n+1)}(t)}{n!} (x-t)^n$. Apply the MVT to g on $[a, x]$ using the auxiliary function $h(t) = (x-t)^{n+1}$ to obtain Cauchy's MVT form of the remainder. $\square$

Example 13.1. The Taylor polynomial for $\sin x$ around 0 is:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots + (-1)^n \frac{x^{2n+1}}{(2n+1)!} + R_{2n+1}(x).$$

The remainder satisfies $|R_{2n+1}(x)| \leq |x|^{2n+3}/(2n+3)!$, which $\rightarrow 0$ for any fixed x as $n \rightarrow \infty$.

Exercises

1. Compute the Taylor polynomial of degree 4 for $\cos x$ around 0.
2. Use Taylor's theorem to prove that $e^x \geq 1 + x$ for all $x \in \mathbb{R}$.
3. Prove the second derivative test: if $f'(a) = 0$ and $f''(a) > 0$, then a is a local minimum.
4. Show that the function $f(x) = e^{-1/x^2}$ (for $x \neq 0$) and $f(0) = 0$ is C^∞ but its Taylor series at 0 is identically zero, failing to converge to f . This shows smooth $\neq$ analytic.
5. Prove the general binomial theorem: $(1 + x)^\alpha = \sum_{k=0}^{\infty} \binom{\alpha}{k} x^k$ for $|x| < 1$.

Chapter 14

Applications of Differentiation

14.1 Optimization

One of the most immediate applications of the derivative is finding maxima and minima. At a local extremum in the interior of the domain, the tangent line must be horizontal — so the derivative must be zero. This reduces the infinite search for a maximum to a finite computation: find all critical points, classify them, and compare values. The second derivative test provides the classification by measuring whether the function curves upward (minimum) or downward (maximum) at the critical point.

Definition: Critical Points

A point c in the domain of f is a **critical point** if $f'(c) = 0$ or $f'(c)$ does not exist.

Theorem: First Derivative Test

Suppose $f'(c) = 0$.

- If f' changes from positive to negative at c , then c is a local maximum.
- If f' changes from negative to positive at c , then c is a local minimum.
- If f' does not change sign, c is neither.

Theorem: Second Derivative Test

If $f'(c) = 0$ and $f''(c)$ exists:

- $f''(c) > 0 \Rightarrow$ local minimum at c .
- $f''(c) < 0 \Rightarrow$ local maximum at c .
- $f''(c) = 0 \Rightarrow$ inconclusive.

14.2 Newton's Method

Newton's method finds roots of $f(x) = 0$ using the iteration:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Theorem: Convergence of Newton's Method

If f is twice continuously differentiable near a simple root r (with $f(r) = 0$, $f'(r) \neq 0$) and x_0 is sufficiently close to r , then Newton's method converges quadratically:

$$|x_{n+1} - r| \leq C|x_n - r|^2$$

for some constant C .

Proof. By Taylor's theorem: $0 = f(r) = f(x_n) + f'(x_n)(r - x_n) + \frac{f''(\xi)}{2}(r - x_n)^2$ for some ξ between r and x_n . Dividing by $f'(x_n)$: $r - x_{n+1} = r - x_n + \frac{f(x_n)}{f'(x_n)} = -\frac{f''(\xi)}{2f'(x_n)}(r - x_n)^2$. $\square$

Lean 4

```
-- Newton's method as a recursive function. Note the type: Float,
-- not Real. Mathlib's Real is noncomputable (a Real is an
-- equivalence class of Cauchy sequences -- there is nothing to
-- "run"), so a Real-valued newton could be defined but never
-- #eval'd. Float is the computational stand-in.
def newton (f df : Float → Float) : Nat → Float → Float
| 0,      x => x
| n + 1, x => newton f df n (x - f x / df x)

-- Example: computing sqrt(2) via Newton's method for f(x) = x^2 - 2
def sqrtNewton (n : Nat) (x0 : Float) : Float :=
  newton (fun x => x * x - 2) (fun x => 2 * x) n x0

-- Starting from x0 = 1.5:
-- 1.5 → 1.4166... → 1.41421356... (converges to sqrt(2))
#eval sqrtNewton 5 1.5 -- prints 1.414214 (Float's default precision)
```

Exercises

1. Find all local extrema of $f(x) = x^4 - 4x^3 + 4x^2$.
2. Prove that $e^x > 1 + x + x^2/2$ for all $x > 0$ using the MVT applied twice.
3. Apply Newton's method to compute $\sqrt[3]{7}$ to 5 decimal places by hand (first two iterations).
4. Prove formally that a cubic polynomial always has at least one real critical point.

5. Prove the AM-GM inequality for n numbers using calculus (minimize $x_1 + \cdots + x_n$ subject to $x_1 \cdots x_n = 1$).

Part IV

Integral Calculus

Chapter 15

The Riemann Integral

Remark

Historical Note: From Cavalieri to Lebesgue

Integration has a longer history than differentiation. **Archimedes** computed areas by the “method of exhaustion” — approximating curved regions by polygons and taking limits — in the third century BCE. This is recognisably the idea behind Riemann sums, though Archimedes had no notion of a function or a limit in the modern sense.

The connection between integration (area) and differentiation (slope) was recognised before either was fully formalised. **Barrow** (Newton’s teacher) proved a geometric version of the Fundamental Theorem around 1665. Newton and Leibniz made it the cornerstone of calculus, though they understood integration primarily as “anti-differentiation” — working backwards from derivatives to areas — rather than as a limit of sums.

The rigorous limit-of-sums definition was given by **Riemann** in his 1854 Habilitation thesis, submitted to qualify as a lecturer at Göttingen. Riemann’s definition — tagged partitions, Riemann sums, the limit as the mesh goes to zero — is the one most calculus textbooks still use. In the same paper, Riemann showed that a function is integrable (by his definition) if and only if its set of discontinuities has “measure zero” in an informal sense. He exhibited functions with discontinuities dense in $[0, 1]$ that are nonetheless Riemann integrable, and functions that are not.

The Darboux reformulation — upper and lower sums, with integrability equivalent to their infimum and supremum coinciding — was given by **Darboux** in 1875. It is logically equivalent to Riemann’s definition but much easier to work with for proving theorems. We follow Darboux.

The limitations of Riemann’s integral became clear in the early twentieth century. Dirichlet’s function is not Riemann integrable. The limit of a uniformly bounded sequence of Riemann-integrable functions need not be Riemann integrable. **Lebesgue** (1901–1902) introduced a completely new approach: instead of partitioning the *domain*, partition the *range*, and measure the preimage of each range interval. This gives an integral that handles Dirichlet’s function easily (integral = 0), allows far more general limit theorems (the Dominated Convergence Theorem), and extends naturally to abstract mea-

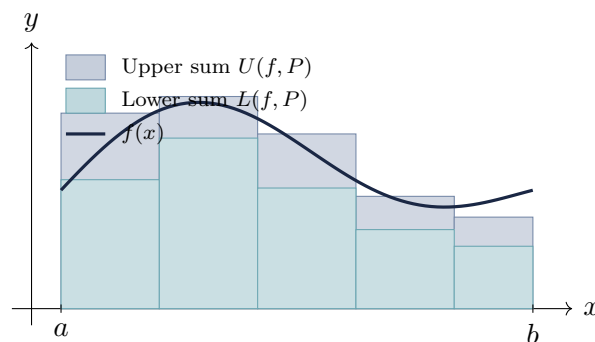
sure spaces far beyond $\mathbb{R}$. Lebesgue integration is the foundation of modern probability theory, functional analysis, and the L^p spaces used in partial differential equations.

For this book we develop Riemann integration, which suffices for all the calculus of Parts III–V. The Lebesgue integral would require a separate book on measure theory, though the connection appears briefly in the exercises.

We now develop a rigorous theory of integration, answering the question: what does it mean to sum up infinitely many infinitesimally thin rectangles?

15.1 Partitions and Riemann Sums

The integral captures the notion of “signed area under a curve.” Our approach is via Darboux sums, which bound the integral from above and below using the supremum and infimum of the function on each subinterval. A function is integrable precisely when the upper and lower bounds can be made to agree arbitrarily closely by refining the partition. This is more technically convenient than the tagged-partition approach of Riemann, and it produces the same integral for all functions that matter in practice.



Definition: Partitions and Riemann Sums

A **partition** of $[a, b]$ is a finite sequence $P = \{a = x_0 < x_1 < \cdots < x_n = b\}$.

The **mesh** of P is $\|P\| = \max_i (x_i - x_{i-1})$.

For $f : [a, b] \rightarrow \mathbb{R}$ **bounded** (boundedness is what guarantees the suprema and infima below exist), the **upper Darboux sum** and **lower Darboux sum** are:

$$U(f, P) = \sum_{i=1}^n M_i (x_i - x_{i-1}), \quad L(f, P) = \sum_{i=1}^n m_i (x_i - x_{i-1}),$$

where $M_i = \sup_{x \in [x_{i-1}, x_i]} f(x)$ and $m_i = \inf_{x \in [x_{i-1}, x_i]} f(x)$.

Definition: Riemann Integrability

A bounded f is **Riemann integrable** on $[a, b]$ if:

$$\sup_P L(f, P) = \inf_P U(f, P).$$

This common value is the **Riemann integral** $\int_a^b f(x) dx$.

Theorem: Darboux's Criterion

f is integrable on $[a, b]$ if and only if for every $\varepsilon > 0$ there exists a partition P with $U(f, P) - L(f, P) < \varepsilon$.

Theorem: Continuous Functions Are Integrable

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous, then f is Riemann integrable.

Proof. By EVT, f is bounded. Since $[a, b]$ is compact, f is uniformly continuous. Given $\varepsilon > 0$, choose δ so $|x - y| < \delta \Rightarrow |f(x) - f(y)| < \varepsilon/(b - a)$. Take a partition with mesh $< \delta$. Then $M_i - m_i < \varepsilon/(b - a)$ on each subinterval, so $U - L < \varepsilon$. $\square$

Theorem: Monotone Functions Are Integrable

If $f : [a, b] \rightarrow \mathbb{R}$ is monotone, then f is Riemann integrable.

Proof. For increasing f and partition P with equal subintervals of width $(b - a)/n$: $M_i = f(x_i)$, $m_i = f(x_{i-1})$, so $U - L = \frac{b-a}{n}(f(b) - f(a)) \rightarrow 0$. $\square$

Lean 4

```
-- A partition of [a, b]
structure Partition (a b : Real) where
  points : List Real
  sorted : points.Pairwise (· ≤ ·)
  starts : points.head? = some a
  ends   : points.getLast? = some b

-- Subintervals from a partition
def subintervals {a b : Real} (P : Partition a b) : List (Real × Real)
  :=
  P.points.zip P.points.tail

-- Upper and lower Darboux sums (sketch: the two sorries stand for
-- the sup and inf of f on a subinterval, which exist because f is
-- bounded; constructing them is this chapter's project exercise).
noncomputable def upperSum {a b : Real} (f : Real → Real)
  (P : Partition a b) : Real :=
  (subintervals P).foldl (fun acc (p : Real × Real) =>
```

```

let Mi : Real := sorry -- sup of f on [p.1, p.2]
acc + Mi * (p.2 - p.1)) 0

noncomputable def lowerSum {a b : Real} (f : Real → Real)
(P : Partition a b) : Real :=
(subintervals P).foldl (fun acc (p : Real × Real) =>
let mi : Real := sorry -- inf of f on [p.1, p.2]
acc + mi * (p.2 - p.1)) 0

-- Riemann integrability (for bounded f)
def IsIntegrable (f : Real → Real) (a b : Real) : Prop :=
∃ I : Real,
  ∀ eps : Real, eps > 0 →
    ∃ P : Partition a b,
      upperSum f P - I < eps ∧ I - lowerSum f P < eps

```

15.2 Properties of the Integral

The four basic properties below — linearity, monotonicity, additivity, and boundness — are used in nearly every integral calculation. Linearity says integration is a linear operation: you can pull out constants and split sums. Monotonicity says the integral respects the ordering of functions. Additivity lets us split an integral at any intermediate point, which is essential for handling piecewise-defined functions and improper integrals.

Theorem: Basic Properties of the Riemann Integral

If f, g are integrable on $[a, b]$:

1. **Linearity:** $\int_a^b (cf + g) = c \int_a^b f + \int_a^b g$.
2. **Monotonicity:** $f \leq g \Rightarrow \int_a^b f \leq \int_a^b g$.
3. **Additivity:** $\int_a^b f = \int_a^c f + \int_c^b f$ for $a \leq c \leq b$.
4. **Boundedness:** $\left| \int_a^b f \right| \leq \int_a^b |f| \leq M(b - a)$ where $M = \sup |f|$.

Exercises

1. Compute $\int_0^1 x^2 dx$ directly from the definition, using partitions $P_n = \{0, 1/n, 2/n, \dots, 1\}$.
2. Prove that if f is integrable and $f = g$ everywhere except finitely many points, then g is integrable with $\int f = \int g$.
3. Show that Dirichlet's function $f(x) = 1$ for $x \in \mathbb{Q}$ and 0 otherwise is not Riemann integrable on $[0, 1]$.

4. Prove the triangle inequality for integrals: $\left| \int_a^b f \right| \leq \int_a^b |f|$.
5. Complete the formalization of `IsIntegrable` over Mathlib's `Real` type: replace the two `sorry`s in the sketch above with genuine suprema and infima (Mathlib's `sSup`/`sInf` of `f` ' ' `Set.Icc p.1 p.2`), adding the boundedness hypothesis the definition needs.

Chapter 16

The Fundamental Theorem of Calculus

Remark

Historical Note: The Fundamental Theorem and Its Discovery

The Fundamental Theorem of Calculus is the hinge on which the entire subject turns, yet its precise statement and proof were slow to emerge. **Isaac Barrow** proved a geometric version in his *Lectiones Geometricae* (1670): if you know the area function $A(t) = \int_a^t f(x) dx$, then its tangent slope is $f(t)$. This is Part 1 of our theorem.

Newton understood Part 2 — that the definite integral can be evaluated by finding an antiderivative — and used it constantly in his *Method of Fluxions* (written 1671, published 1736). For Newton, this was practically the *definition* of the integral: integration *was* anti-differentiation, and the connection to areas was a consequence. Leibniz, by contrast, thought of $\int$ as a summing operation from the start, and the connection to derivatives was the theorem.

The first completely rigorous proof was given by **Cauchy** in his 1823 *Résumé*, using the definition of the integral as a limit of sums that he had just introduced. Cauchy proved Part 1 by estimating the difference quotient $(F(x+h) - F(x))/h$ using the intermediate value theorem for f , essentially the argument we give. This required continuity of f , a hypothesis that Cauchy was careful to state.

The generalisation to discontinuous functions — asking exactly which f can be integrated and which integrated functions can be differentiated — occupied analysis for the rest of the nineteenth century, culminating in Lebesgue's theory. The most general version of the FTC (in the Lebesgue setting) states that an absolutely continuous function is differentiable almost everywhere, and the derivative, integrated, returns the original function. This precise formulation required the full machinery of measure theory.

The Fundamental Theorem is the central result of calculus: it reveals that differentiation and integration are inverse operations, connecting the two branches of the subject.

16.1 FTC Part 1: The Integral as an Antiderivative

The first part of the Fundamental Theorem reveals that differentiation and integration are inverse operations: if you integrate a function f from a fixed base point a to a variable upper limit x , the resulting function $F(x) = \int_a^x f(t) dt$ is an antiderivative of f . This is a deep fact, not a tautology. The integral is defined as a limit of sums; the derivative is a limit of difference quotients. That these two limiting operations undo each other is the central miracle of calculus.

Definition: The Integral Function

For f integrable on $[a, b]$, define the **integral function**:

$$F(x) := \int_a^x f(t) dt, \quad x \in [a, b].$$

Theorem: Fundamental Theorem of Calculus, Part 1

If f is continuous on $[a, b]$, then F is differentiable on (a, b) and $F'(x) = f(x)$.

Proof. For $h \neq 0$:

$$\frac{F(x+h) - F(x)}{h} = \frac{1}{h} \int_x^{x+h} f(t) dt.$$

Since f is continuous at x , for any $\varepsilon > 0$ there exists $\delta > 0$ so $|f(t) - f(x)| < \varepsilon$ when $|t - x| < \delta$. For $|h| < \delta$:

$$\left| \frac{1}{h} \int_x^{x+h} f(t) dt - f(x) \right| = \left| \frac{1}{h} \int_x^{x+h} (f(t) - f(x)) dt \right| \leq \frac{|h|\varepsilon}{|h|} = \varepsilon.$$

□

16.2 FTC Part 2: The Evaluation Theorem

Part 2 is what makes integration computable in practice. Rather than computing a limit of Riemann sums (which is laborious even for simple functions), we simply find any antiderivative G and evaluate $G(b) - G(a)$. The two parts together mean: *to integrate, anti-differentiate*. This transforms the problem of integration into the problem of recognising derivatives — which is exactly what the integration engine in Part VII formalises as a pattern-matching algorithm.

Theorem: Fundamental Theorem of Calculus, Part 2

If f is continuous on $[a, b]$ and G is any antiderivative of f (i.e., $G' = f$), then:

$$\int_a^b f(x) dx = G(b) - G(a).$$

Proof. Let $F(x) = \int_a^x f(t) dt$. By Part 1, $F' = f = G'$, so $G - F$ is constant (by MVT, $(G - F)' = 0 \Rightarrow G - F = c$). Evaluating the constant at $x = a$: $c = G(a) - F(a) = G(a)$, since $F(a) = 0$. Hence $F(x) = G(x) - G(a)$ for all x , and in particular $\int_a^b f = F(b) = G(b) - G(a)$. $\square$

The power of FTC Part 2 cannot be overstated: instead of computing limits of Riemann sums, we need only find an antiderivative. This is why the integration techniques of Chapter 17 matter.

Formalising FTC honestly requires an *actual integral*, and our Chapter 15 sketch stopped short of one (its Darboux sums still contain [sorrys](#) for the suprema). Rather than state a theorem about an integral we never finished defining, this is the one place in the book where we lean on Mathlib’s analysis library itself: Mathlib’s interval integral `∫ x in a..b, f x`, its own derivative predicate (written `_root_.HasDerivAt` below to distinguish it from our ε - δ version — the underscore prefix says “the top-level one, not ours”), and its integrability predicate `IntervalIntegrable`. The reward for stating it in Mathlib’s terms: the proof is a *one-liner*, because Mathlib has already done the work of this whole chapter, in full generality.

Lean 4

```
-- FTC Part 2, stated against Mathlib's interval integral and
-- proved by Mathlib's own FTC. The hypothesis asks G to have
-- derivative f(x) at every x between a and b (Set.uIcc a b is
-- the closed interval between them, whichever is larger).
theorem ftc_part2 (f G : Real → Real) (a b : Real)
  (hderiv : ∀ x ∈ Set.uIcc a b, _root_.HasDerivAt G (f x) x)
  (hint : IntervalIntegrable f MeasureTheory.volume a b) :
  ∫ x in a..b, f x = G b - G a :=
  intervalIntegral.integral_eq_sub_of_hasDerivAt hderiv hint
```

Completing the story with our own definitions — proving that a continuous f satisfies `IsIntegrable` (Chapter 15) and that the resulting value equals $G(b) - G(a)$, following the paper proof above — is a substantial but rewarding project; the sketch in the paper proof is the roadmap.

Exercises

1. Use FTC Part 2 to evaluate $\int_0^\pi \sin x \, dx$, $\int_1^e (1/x) \, dx$, and $\int_0^1 e^x \, dx$.
2. Prove that $\frac{d}{dx} \int_a^{g(x)} f(t) \, dt = f(g(x)) \cdot g'(x)$ (chain rule + FTC Part 1).
3. Prove: if $F' = f$ everywhere and $f \geq 0$, then F is non-decreasing.
4. Show: FTC Part 1 can fail if f is not continuous. (Hint: consider $f = 1_{\mathbb{Q}}$ on $[0, 1]$.)
5. Formalize FTC Part 1 in Lean 4 assuming the necessary lemmas.

Chapter 17

Techniques of Integration

Each integration technique is a theorem—a rule that transforms one integral into another, simpler one. This chapter is also the direct mathematical foundation for the integration engine in Part VII.

17.1 Substitution (Change of Variables)

U -substitution is the integration counterpart of the chain rule: just as $(f \circ g)' = (f' \circ g) \cdot g'$, integrating $f(g(x))g'(x)$ reduces to integrating $f(u)$ after the substitution $u = g(x)$. Geometrically, substitution rescales the axis: the factor $g'(x)$ accounts for how the substitution stretches or compresses the x -axis. In formal terms, it is the change-of-variables theorem applied to a one-dimensional integral.

Theorem: Integration by Substitution

If $g : [a, b] \rightarrow \mathbb{R}$ is differentiable with continuous g' , and f is continuous on the range of g , then:

$$\int_a^b f(g(x)) g'(x) dx = \int_{g(a)}^{g(b)} f(u) du.$$

Proof. Let F be an antiderivative of f . By the chain rule, $\frac{d}{dx}F(g(x)) = f(g(x))g'(x)$. By FTC Part 2: LHS = $F(g(b)) - F(g(a))$ = RHS. $\square$

Example 17.1. $\int_0^1 2xe^{x^2} dx$. Let $u = x^2$, $du = 2x dx$. When $x = 0$, $u = 0$; when $x = 1$, $u = 1$: $\int_0^1 e^u du = e^1 - e^0 = e - 1$.

17.2 Integration by Parts

Integration by parts is the integration counterpart of the product rule. It trades one integral for another: $\int u dv = uv - \int v du$. The art is in choosing u and dv so the new integral $\int v du$ is simpler than the original. The LIATE heuristic (Logarithm, Inverse trig, Algebraic, Trig, Exponential) ranks which factor should be u — it encodes centuries of experience about which choices tend to reduce complexity.

Theorem: Integration by Parts

If f and g are differentiable with continuous derivatives on $[a, b]$:

$$\int_a^b f(x)g'(x) dx = [f(x)g(x)]_a^b - \int_a^b f'(x)g(x) dx.$$

Proof. By the product rule: $(fg)' = f'g + fg'$. Integrate both sides from a to b and apply FTC. $\square$

Example 17.2. $\int xe^x dx$. Let $f = x$, $g' = e^x$, so $f' = 1$, $g = e^x$: $= xe^x - \int e^x dx = xe^x - e^x + C = (x - 1)e^x + C$.

17.3 Partial Fraction Decomposition

Rational functions $p(x)/q(x)$ can be integrated by decomposing into partial fractions.

Theorem: Partial Fraction Decomposition

Every rational function $p(x)/q(x)$ (with $\deg p < \deg q$) over $\mathbb{R}$ can be written as a sum of terms of the form $\frac{A}{(x-a)^k}$ and $\frac{Bx+C}{(x^2+bx+c)^k}$ (for irreducible quadratics $x^2 + bx + c$).

The proof uses the factorization of $q(x)$ over $\mathbb{R}$ and the algebraic uniqueness of the decomposition.

Example 17.3. $\int \frac{1}{x^2-1} dx = \int \frac{1}{(x-1)(x+1)} dx = \int \left(\frac{1/2}{x-1} - \frac{1/2}{x+1} \right) dx = \frac{1}{2} \ln|x-1| - \frac{1}{2} \ln|x+1| + C$.

17.4 Trigonometric Substitutions

For integrands involving $\sqrt{a^2 - x^2}$, $\sqrt{a^2 + x^2}$, or $\sqrt{x^2 - a^2}$, use the substitutions $x = a \sin \theta$, $x = a \tan \theta$, $x = a \sec \theta$ respectively to remove the square root.

17.5 Trigonometric Integrals

Integrals of the form $\int \sin^m x \cos^n x dx$ are handled by:

- If m or n is odd: save one factor and use $\sin^2 + \cos^2 = 1$ to convert.
- If both even: use double-angle formulas $\sin^2 x = (1 - \cos 2x)/2$ and $\cos^2 x = (1 + \cos 2x)/2$.

Exercises

1. Evaluate $\int_0^{\pi/2} \sin^3 x \cos^2 x dx$ using the odd-power strategy.
2. Evaluate $\int \frac{x^2+1}{x^3-x} dx$ using partial fractions.

3. Evaluate $\int \sqrt{1-x^2} dx$ using trigonometric substitution.
4. Derive the reduction formula: $\int \sin^n x dx = -\frac{\sin^{n-1} x \cos x}{n} + \frac{n-1}{n} \int \sin^{n-2} x dx.$
5. Use integration by parts repeatedly to show $\int x^n e^x dx$ satisfies a recursive formula.
6. Show $\int_0^\infty x^{n-1} e^{-x} dx = (n-1)!$ using integration by parts (this is $\Gamma(n)$).

Chapter 18

Improper Integrals

Definition: Improper Integrals of Type I and II

Type I (infinite limits): $\int_a^\infty f(x) dx := \lim_{b \rightarrow \infty} \int_a^b f(x) dx$ (if the limit exists).

Type II (unbounded integrand): $\int_a^b f(x) dx := \lim_{c \rightarrow a^+} \int_c^b f(x) dx$ (if f is unbounded near a).

An improper integral **converges** if the limit is finite; otherwise it **diverges**.

Theorem: Comparison Test for Improper Integrals

If $0 \leq f(x) \leq g(x)$ for $x \geq a$:

- If $\int_a^\infty g$ converges, so does $\int_a^\infty f$.
- If $\int_a^\infty f$ diverges, so does $\int_a^\infty g$.

Theorem: The p -integral

$\int_1^\infty \frac{1}{x^p} dx$ converges iff $p > 1$. Similarly, $\int_0^1 \frac{1}{x^p} dx$ converges iff $p < 1$.

Definition: The Gamma Function

For $n > 0$: $\Gamma(n) := \int_0^\infty x^{n-1} e^{-x} dx$.

Theorem: $\Gamma(n) = (n-1)!$ for $n \in \mathbb{N}$

Proof. $\Gamma(1) = \int_0^\infty e^{-x} dx = [-e^{-x}]_0^\infty = 1$. Integration by parts with $u = x^n$, $dv = e^{-x} dx$ gives $\Gamma(n+1) = n\Gamma(n)$. By induction $\Gamma(n) = (n-1)!$ for all $n \in \mathbb{N}$. $\square$

Exercises

1. Determine convergence: $\int_1^\infty e^{-x^2} dx$, $\int_0^1 \ln x dx$, $\int_0^\infty \frac{\sin x}{x} dx$.

2. Prove $\int_0^\infty e^{-x^2} dx = \sqrt{\pi}/2$ (Gaussian integral, using the double-integral technique).
3. Prove the functional equation $\Gamma(n+1) = n\Gamma(n)$ for all $n > 0$.
4. Show $\int_{-\infty}^\infty \frac{1}{1+x^2} dx = \pi$ and interpret it probabilistically.

Part V

Sequences and Series

Chapter 19

Infinite Series

Remark

Historical Note: The Paradoxes of the Infinite and the Birth of Convergence Theory

Infinite series were used long before they were understood. **Archimedes** summed the geometric series $\sum (1/4)^n = 4/3$ to compute the area of a parabolic segment. Medieval scholars debated Zeno's paradox of Achilles and the tortoise — which is really just the statement that $\sum_{n=1}^{\infty} (1/2)^n = 1$ — without having the tools to resolve it rigorously.

The danger of infinite series became apparent when **Grandi** (1703) noted the series $1 - 1 + 1 - 1 + \cdots$. Grouping as $(1 - 1) + (1 - 1) + \cdots$ gives 0; grouping as $1 + (-1 + 1) + (-1 + 1) + \cdots$ gives 1. Leibniz argued, by probability, the “true” sum was $1/2$ (the average of 0 and 1). Euler, who was cavalier about convergence throughout his career, evaluated the series $1 + 2 + 3 + 4 + \cdots = -1/12$ using his divergent series techniques. (This is not merely wrong: in the context of *analytic continuation* of the Riemann zeta function, it is a meaningful statement. But treating it as a convergent series in the ordinary sense is incorrect.)

Cauchy imposed order in 1821 by insisting on a precise definition of convergence via partial sums, as we give below. He proved that a series converges if and only if its partial sums form a Cauchy sequence (the Cauchy criterion for series), and derived the ratio and root tests. **Abel** (1826) proved the theorem bearing his name: if a power series $\sum a_n x^n$ converges at $x = R$, then it converges uniformly on $[0, R]$. Abel used this to correct Cauchy's erroneous claim that a convergent series of continuous functions is always continuous — continuity at the endpoint R requires uniform convergence, not just pointwise. The distinction between absolute and conditional convergence was clarified by **Dirichlet** (1837), who proved that a conditionally convergent series can be rearranged to converge to any value — the **Riemann rearrangement theorem** we prove in this chapter (Riemann gave a complete proof in 1854). This is one of the most counterintuitive results in analysis: the infinite analogue of addition is not commutative for conditionally convergent series.

Definition: Convergence of a Series

Given $(a_n)_{n=1}^{\infty}$, define the partial sums $S_n = \sum_{k=1}^n a_k$. The series $\sum_{n=1}^{\infty} a_n$ **converges** to S if $S_n \rightarrow S$; otherwise it **diverges**.

Theorem: Divergence Test

If $\sum a_n$ converges, then $a_n \rightarrow 0$. Equivalently, if $a_n \not\rightarrow 0$, the series diverges.

Theorem: Geometric Series

$\sum_{n=0}^{\infty} r^n = \frac{1}{1-r}$ for $|r| < 1$, and diverges for $|r| \geq 1$.

Proof. $S_n = 1 + r + \cdots + r^n = \frac{1-r^{n+1}}{1-r}$. For $|r| < 1$, $r^{n+1} \rightarrow 0$, so $S_n \rightarrow \frac{1}{1-r}$. $\square$

19.1 Convergence Tests

Deciding whether an infinite series converges is often harder than evaluating it. The convergence tests below are organised by the type of information they extract: the ratio and root tests measure the asymptotic growth rate of individual terms; the integral test connects series to improper integrals; the alternating series test exploits sign cancellation. No single test works for everything — part of the skill of analysis is recognising which test applies to which series.

Theorem: Tests for Convergence

For a series $\sum a_n$ with $a_n \geq 0$:

1. **Comparison:** If $a_n \leq b_n$ and $\sum b_n$ converges, so does $\sum a_n$.
2. **Ratio Test:** If $\lim |a_{n+1}/a_n| = L$: converges if $L < 1$, diverges if $L > 1$.
3. **Root Test:** If $\lim |a_n|^{1/n} = L$: same conclusion.
4. **Integral Test:** If $f(n) = a_n$ with f positive, decreasing: $\sum a_n$ and $\int_1^{\infty} f$ either both converge or both diverge.
5. **Alternating Series Test:** If $b_n \searrow 0$, then $\sum (-1)^n b_n$ converges.

Definition: Absolute vs. Conditional Convergence

$\sum a_n$ **converges absolutely** if $\sum |a_n|$ converges. Absolute convergence implies convergence. $\sum a_n$ **converges conditionally** if it converges but not absolutely.

Theorem: Riemann Rearrangement Theorem

A conditionally convergent series can be rearranged to converge to any real number L , or to diverge.

Exercises

1. Determine convergence of $\sum_{n=1}^{\infty} \frac{n^2}{2^n}$, $\sum \frac{(-1)^n}{\sqrt{n}}$, $\sum \frac{1}{n \ln n}$.
2. Prove the ratio test from first principles.
3. Show that $\sum 1/n^2 = \pi^2/6$ (Basel problem) using the fact that $\pi^2/6 = \int_0^1 \int_0^1 \frac{1}{1-xy} dx dy$.
4. Construct an explicit rearrangement of $\sum (-1)^n/n$ that converges to 2.
5. Prove: if $\sum a_n$ converges absolutely and (b_n) is bounded, then $\sum a_n b_n$ converges.

Chapter 20

Power Series

The radius-of-convergence formula below uses one piece of vocabulary we have not yet defined: the *limit superior*. A bounded sequence need not converge, but the suprema of its tails always settle down.

Definition: Limit Superior

For a sequence (a_n) of reals, the **limit superior** is

$$\limsup_{n \rightarrow \infty} a_n := \lim_{n \rightarrow \infty} \left(\sup_{k \geq n} a_k \right),$$

i.e. the limit of the tail-suprema $s_n = \sup\{a_k : k \geq n\}$. The s_n are non-increasing, so by the Monotone Convergence Theorem the limit always exists when (a_n) is bounded above (we set $\limsup a_n = +\infty$ otherwise, and $-\infty$ if $a_n \rightarrow -\infty$). Equivalently, $\limsup a_n$ is the largest limit of any convergent subsequence. For a convergent sequence, $\limsup a_n = \lim a_n$; the point of $\limsup$ is that it exists even when the plain limit does not — e.g. $\limsup (-1)^n = 1$.

Definition: Power Series and Radius of Convergence

A **power series** centered at a is $\sum_{n=0}^{\infty} c_n(x-a)^n$.

Its **radius of convergence** is $R = 1/\limsup_{n \rightarrow \infty} |c_n|^{1/n}$ (the Cauchy–Hadamard formula, with the conventions $1/0 = \infty$ and $1/\infty = 0$). The series converges absolutely for $|x-a| < R$ and diverges for $|x-a| > R$.

Theorem: Term-by-Term Differentiation and Integration

If $f(x) = \sum_{n=0}^{\infty} c_n(x-a)^n$ has radius of convergence $R > 0$, then for $|x-a| < R$:

$$f'(x) = \sum_{n=1}^{\infty} n c_n (x-a)^{n-1}, \quad \text{also with radius of convergence } R.$$

$$\int_a^x f(t) dt = \sum_{n=0}^{\infty} \frac{c_n}{n+1} (x-a)^{n+1}.$$

Proof. The key lemma is that the derived series has the same radius of convergence (since $\lim n^{1/n} = 1$). The equality $f' = \sum n c_n (x-a)^{n-1}$ is proved by showing uniform convergence on compact subsets and passing the limit through the derivative. $\square$

Theorem: Uniqueness of Power Series

If $f(x) = \sum_{n=0}^{\infty} c_n (x-a)^n$ for $|x-a| < R$, then $c_n = f^{(n)}(a)/n!$.

Exercises

1. Find the radius of convergence of $\sum n!x^n$, $\sum x^n/n!$, $\sum x^n/n$.
2. Use the geometric series $1/(1-x) = \sum x^n$ and term-by-term integration to derive the series for $\ln(1+x)$.
3. Derive the series for $\arctan x$ from $1/(1+x^2) = \sum (-1)^n x^{2n}$.
4. Prove that $\sum x^n/n!$ converges for all $x \in \mathbb{R}$ and defines a differentiable function equal to its own derivative.

Chapter 21

Taylor and Maclaurin Series

Definition: Taylor Series

If f is infinitely differentiable at a , its **Taylor series** is:

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n.$$

A function is **analytic** at a if it equals its Taylor series in some neighborhood of a .

The standard Taylor series (all centered at 0, valid for the indicated x):

$$\begin{aligned} e^x &= \sum_{n=0}^{\infty} \frac{x^n}{n!} && \text{all } x \\ \sin x &= \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} && \text{all } x \\ \cos x &= \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} && \text{all } x \\ \ln(1+x) &= \sum_{n=1}^{\infty} \frac{(-1)^{n-1} x^n}{n} && |x| < 1 \\ (1+x)^\alpha &= \sum_{n=0}^{\infty} \binom{\alpha}{n} x^n && |x| < 1 \end{aligned}$$

Remark

Not every C^∞ function is analytic. The function $f(x) = e^{-1/x^2}$ (extended by $f(0) = 0$) is C^∞ everywhere, yet its Taylor series at 0 is identically 0 $\neq f(x)$ for $x \neq 0$. Analytic functions form a strictly smaller class than smooth functions.

Lean 4

```
-- We can define e^x via its Taylor series and prove properties.
-- Taylor polynomials for e^x (noncomputable: Real division is)
noncomputable def expTaylor (n : Nat) (x : Real) : Real :=
  Finset.sum (Finset.range (n + 1)) (fun k =>
    x ^ k / (Nat.factorial k : Real))

-- The k-th term
noncomputable def expTerm (k : Nat) (x : Real) : Real :=
  x ^ k / (Nat.factorial k : Real)

-- Ratio test: |a_{k+1}/a_k| = |x|/(k+1) → 0 for any fixed x
theorem expTaylor_converges (x : Real) :
  SeqLimit (fun n => expTaylor n x) (Real.exp x) := by
  sorry -- exercise: series convergence machinery from Ch. 19-20

-- e^x satisfies (e^x)' = e^x (our epsilon-delta HasDerivAt)
theorem exp_deriv (x : Real) : HasDerivAt Real.exp x (Real.exp x) := by
  sorry -- exercise: term-by-term differentiation, or transfer
        -- Mathlib's Real.hasDerivAt_exp to our definition
```

Exercises

1. Derive the Taylor series for $\arcsin x$ from $1/\sqrt{1-x^2} = \sum \binom{-1/2}{n}(-x^2)^n$.
2. Use series to evaluate $\int_0^1 e^{-x^2} dx$ to 4 decimal places.
3. Prove $e^{ix} = \cos x + i \sin x$ (Euler's formula) using the Taylor series.
4. Show that the Taylor series for $\ln(1+x)$ at $x = 1$ converges to $\ln 2$ by the alternating series test.
5. Prove the addition formula for exp: $e^{x+y} = e^x \cdot e^y$, using series multiplication.

Part VI

Multivariable Calculus

Chapter 22

$\mathbb{R}^n$ and Its Topology

22.1 Vectors in $\mathbb{R}^n$

Multivariable calculus generalises everything from Part III onward to functions of n real variables. The domain is no longer a line but n -dimensional Euclidean space $\mathbb{R}^n$. The correct notion of distance is the Euclidean norm, and the correct notion of angle is the dot product. Cauchy–Schwarz — the single most useful inequality in analysis — bounds the dot product by the product of norms, and its proof is an elegant one-liner using the non-negativity of a quadratic. In Lean 4, we model $\mathbb{R}^n$ as the function type `Fin n -> Real`: a vector is just a function from n indices to real values.

Definition: Euclidean Space $\mathbb{R}^n$

$\mathbb{R}^n$ is the set of n -tuples $(x_1, \dots, x_n)$ with $x_i \in \mathbb{R}$. In Lean 4 we model it as `Fin n -> Real`.

The **Euclidean norm** is $\|x\| = \sqrt{x_1^2 + \dots + x_n^2}$. The **dot product** is $x \cdot y = \sum_{i=1}^n x_i y_i$. The **Euclidean distance** is $d(x, y) = \|x - y\|$.

Theorem: Cauchy–Schwarz Inequality

For all $x, y \in \mathbb{R}^n$: $|x \cdot y| \leq \|x\| \|y\|$.

Proof. For any $t \in \mathbb{R}$: $0 \leq \|x + ty\|^2 = \|x\|^2 + 2t(x \cdot y) + t^2\|y\|^2$. This quadratic in t is non-negative, so its discriminant satisfies $4(x \cdot y)^2 - 4\|x\|^2\|y\|^2 \leq 0$. $\square$

Theorem: Triangle Inequality in $\mathbb{R}^n$

$\|x + y\| \leq \|x\| + \|y\|$.

Proof. $\|x + y\|^2 = \|x\|^2 + 2x \cdot y + \|y\|^2 \leq \|x\|^2 + 2\|x\|\|y\| + \|y\|^2 = (\|x\| + \|y\|)^2$. $\square$

Lean 4

```

-- R^n as functions from Fin n to Real
def Vec (n : Nat) := Fin n → Real

def dotProduct {n : Nat} (v w : Vec n) : Real :=
  Finset.sum Finset.univ (fun i => v i * w i)

def normSq {n : Nat} (v : Vec n) : Real :=
  dotProduct v v

noncomputable def euclidNorm {n : Nat} (v : Vec n) : Real :=
  Real.sqrt (normSq v)

-- Cauchy-Schwarz
theorem cauchy_schwarz {n : Nat} (v w : Vec n) :
  dotProduct v w ^ 2 ≤ normSq v * normSq w := by
  -- Classic proof via discriminant
  have h : ∀ t : Real,
    0 ≤ normSq (fun i => v i + t * w i) := fun t => by
    unfold normSq dotProduct
    exact Finset.sum_nonneg fun i _ => mul_self_nonneg _
  -- The LHS is a non-negative quadratic in t: ||v||^2 + 2t(v.w) + t^2||w||^2
  -- Discriminant must be ≤ 0
  sorry -- exercise: expand normSq and apply the discriminant argument

```

22.2 Open Sets, Convergence, and Compactness in $\mathbb{R}^n$

All the topological concepts from Chapter 9 carry over to $\mathbb{R}^n$ essentially unchanged, with the absolute value $|x - a|$ replaced by the Euclidean norm $\|x - a\|$. Open balls, open sets, closed sets, convergence, and compactness are all defined by the same pattern. The Heine–Borel theorem — compact iff closed and bounded — holds in $\mathbb{R}^n$ for exactly the same reason as in $\mathbb{R}$: completeness allows the bisection argument to work in each coordinate independently.

Definition: Topology of $\mathbb{R}^n$

An **open ball** in $\mathbb{R}^n$ is $B(a, r) = \{x \in \mathbb{R}^n : \|x - a\| < r\}$. A set $U \subseteq \mathbb{R}^n$ is **open** if every point has a neighborhood ball inside U . A set is **closed** if its complement is open. A sequence (x_k) in $\mathbb{R}^n$ **converges** to L if $\|x_k - L\| \rightarrow 0$.

Theorem: Heine–Borel in $\mathbb{R}^n$

A subset of $\mathbb{R}^n$ is compact iff it is closed and bounded.

The proof follows the same bisection argument as in $\mathbb{R}$, now bisecting an n -dimensional box.

Exercises

1. Prove: $(x_k) \rightarrow L$ in $\mathbb{R}^n$ iff each component $(x_k)_i \rightarrow L_i$ in $\mathbb{R}$.
2. Prove the triangle inequality in $\mathbb{R}^n$ from Cauchy–Schwarz.
3. Show that the closed unit ball $\{x : \|x\| \leq 1\}$ is compact in $\mathbb{R}^n$.
4. Define continuity of $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ at a point and prove compositions of continuous maps are continuous.

Chapter 23

Partial Derivatives and Differentiability

23.1 Partial Derivatives

For a function of n variables, we can differentiate in each coordinate direction separately by holding all other variables constant. The result is called a partial derivative. Partial derivatives are easy to compute — just apply the single-variable rules with all other variables treated as constants — but they carry less information than the total derivative: a function can have all partial derivatives at a point yet still fail to be differentiable there, as the classic counterexample below shows.

Definition: Partial Derivative

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and the i -th standard basis vector e_i :

$$\frac{\partial f}{\partial x_i}(a) := \lim_{h \rightarrow 0} \frac{f(a + he_i) - f(a)}{h}.$$

This is the derivative of f in the direction of the i -th coordinate axis, with all other coordinates fixed.

23.2 Total Differentiability

A function may have all partial derivatives yet still fail to be “differentiable” in a meaningful sense. The right notion is the *total derivative*.

Definition: Total Derivative (Fréchet Derivative)

$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is **differentiable** at a if there exists a linear map $Df(a) : \mathbb{R}^n \rightarrow \mathbb{R}^m$ such that:

$$\lim_{h \rightarrow 0} \frac{\|f(a + h) - f(a) - Df(a)(h)\|}{\|h\|} = 0.$$

$Df(a)$ is the **total derivative** (or Fréchet derivative). Its matrix in standard bases is the **Jacobian** $J_{ij} = \partial f_i / \partial x_j$.

Theorem: Differentiability Implies Existence of Partial Derivatives (But Not Vice Versa)

If f is differentiable at a , then all partial derivatives exist at a and $Df(a)$ is the Jacobian matrix. The converse fails: the function $f(x, y) = xy/(x^2 + y^2)$ (with $f(0, 0) = 0$) has both partial derivatives at the origin but is not differentiable there.

Theorem: Continuous Partial Derivatives $\Rightarrow$ Differentiability

If all partial derivatives of f exist and are continuous at a , then f is differentiable at a .

Proof. By the MVT applied in each coordinate direction. Write $f(a + h) - f(a) = \sum_i [f(a + h_1 e_1 + \dots + h_i e_i) - f(a + h_1 e_1 + \dots + h_{i-1} e_{i-1})]$; apply the MVT to each term; use continuity of the partials to show the error is $o(\|h\|)$. $\square$

Lean 4

```
-- Partial derivative: fix all coordinates except the i-th and
-- take the one-variable derivative of the resulting function.
-- Here we use Mathlib's deriv (which returns 0 when the function
-- is not differentiable -- a "junk value", a standard Mathlib
-- idiom that keeps partialDeriv total).
noncomputable def partialDeriv {n : Nat} (f : Vec n → Real)
  (i : Fin n) (a : Vec n) : Real :=
  deriv (fun t => f (Function.update a i t)) (a i)

-- Total (Frechet) derivative as a linear map
def HasTotalDerivAt {n : Nat} (f : Vec n → Real) (a : Vec n)
  (L : Vec n → Real) : Prop :=
  (∀ v w : Vec n, L (fun i => v i + w i) = L v + L w) ∧
  (∀ c : Real, ∀ v, L (fun i => c * v i) = c * L v) ∧
  ∀ eps : Real, eps > 0 →
    ∃ delta : Real, delta > 0 ∧
      ∀ h : Vec n, euclidNorm h < delta → euclidNorm h > 0 →
        |f (fun i => a i + h i) - f a - L h| < eps * euclidNorm h
```

Exercises

1. Compute all partial derivatives of $f(x, y) = x^2 y + \sin(xy)$.
2. Show that $f(x, y) = xy/\sqrt{x^2 + y^2}$ (for $(x, y) \neq 0$) is not differentiable at the origin despite having partial derivatives there.
3. Prove: if $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable at a , then f is continuous at a .
4. Prove the multivariable chain rule: if $g : \mathbb{R}^m \rightarrow \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}^p$ are differentiable, then $D(f \circ g)(a) = Df(g(a)) \circ Dg(a)$.

Chapter 24

The Gradient and Directional Derivatives

Definition: Directional Derivative and Gradient

The **directional derivative** of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ at a in direction v (with $\|v\| = 1$) is:

$$D_v f(a) := \lim_{t \rightarrow 0} \frac{f(a + tv) - f(a)}{t}.$$

The **gradient** is the vector of partial derivatives:

$$\nabla f(a) = \left(\frac{\partial f}{\partial x_1}(a), \dots, \frac{\partial f}{\partial x_n}(a) \right).$$

Theorem: Gradient and Directional Derivatives

If f is differentiable at a , then $D_v f(a) = \nabla f(a) \cdot v$. The gradient $\nabla f(a)$ points in the direction of steepest ascent.

Proof. By the chain rule: $D_v f(a) = Df(a)(v) = \nabla f(a) \cdot v$. By Cauchy-Schwarz, $|\nabla f(a) \cdot v| \leq \|\nabla f(a)\|$, with equality when $v = \nabla f(a) / \|\nabla f(a)\|$. $\square$

Theorem: Chain Rule in $\mathbb{R}^n$

If $\mathbf{r}(t) : \mathbb{R} \rightarrow \mathbb{R}^n$ is differentiable and $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, then:

$$(f \circ \mathbf{r})'(t) = \nabla f(\mathbf{r}(t)) \cdot \mathbf{r}'(t).$$

Exercises

1. Find the gradient of $f(x, y, z) = x^2 + yz + e^{xz}$.
2. Find the direction of steepest ascent for $f(x, y) = x^2 - y^2$ at $(1, 1)$.
3. Prove that if f is constant on a surface $g(x) = c$, then $\nabla f(a)$ is proportional to $\nabla g(a)$ at any a on the surface (this is the geometric basis for Lagrange multipliers).

4. Compute the directional derivative of $f(x, y) = \sin(x + y)$ at $(0, 0)$ in direction $(1/\sqrt{2}, 1/\sqrt{2})$.

Chapter 25

Higher-Order Partialials

Definition: Mixed Partial Derivatives and the Hessian

The **Hessian matrix** of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ at a is $H_{ij}(a) = \frac{\partial^2 f}{\partial x_i \partial x_j}(a)$.

Theorem: Clairaut's Theorem (Equality of Mixed Partialials)

If $\frac{\partial^2 f}{\partial x_i \partial x_j}$ and $\frac{\partial^2 f}{\partial x_j \partial x_i}$ are both continuous at a , then they are equal there.

Proof. By a careful limit argument involving the mixed difference quotient $f(a + he_i + ke_j) - f(a + he_i) - f(a + ke_j) + f(a)$. Apply the MVT twice. $\square$

The Hessian is symmetric (when mixed partials are continuous): $H = H^T$.

Theorem: Second-Order Taylor Expansion in $\mathbb{R}^n$

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice continuously differentiable at a , then:

$$f(a + h) = f(a) + \nabla f(a) \cdot h + \frac{1}{2} h^T H(a) h + o(\|h\|^2),$$

where $H(a)$ is the Hessian matrix at a .

Proof. Apply the scalar Taylor theorem to $g(t) = f(a + th)$ at $t = 0$ up to second order, then expand using the chain rule twice. The first derivative gives $\nabla f(a) \cdot h$; the second derivative gives $h^T H(a) h$. $\square$

Exercises

1. Compute the Hessian of $f(x, y) = x^3 - 3xy^2$ and verify Clairaut's theorem.
2. Show that $f(x, y) = xy \sin(1/(x^2 + y^2))$ (extended by $f(0, 0) = 0$) has equal mixed partials at the origin despite neither being continuous there.
3. Use the second-order expansion to classify the critical points of $f(x, y) = x^4 + y^4 - 4xy$.

4. Prove: if $H(a)$ is positive definite, then a is a strict local minimum.

Chapter 26

Multivariable Optimization

26.1 Critical Points and the Second Derivative Test

The multivariable second-order analysis mirrors the one-dimensional case, with the second derivative replaced by the Hessian matrix. A critical point is where the gradient vanishes. Classification depends on the definiteness of the Hessian: a positive definite Hessian means the function curves upward in every direction (local minimum), while an indefinite Hessian means the function goes up in some directions and down in others (saddle point). The eigenvalues of the Hessian determine definiteness — a bridge to the linear algebra underlying optimisation.

Definition: Critical Points in $\mathbb{R}^n$

A point $a \in \mathbb{R}^n$ is a **critical point** of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ if $\nabla f(a) = \mathbf{0}$, i.e. all partial derivatives vanish at a .

Theorem: Second Derivative Test in $\mathbb{R}^n$

Let a be a critical point of f and $H = H(a)$ the Hessian at a .

- If H is positive definite ($h^T H h > 0$ for all $h \neq 0$): strict local minimum.
- If H is negative definite: strict local maximum.
- If H is indefinite (positive and negative eigenvalues): saddle point.
- If H is semidefinite: inconclusive.

For $n = 2$ the test reduces to: let $D = f_{xx}f_{yy} - f_{xy}^2$. If $D > 0$ and $f_{xx} > 0$: local min; $D > 0$ and $f_{xx} < 0$: local max; $D < 0$: saddle.

26.2 Lagrange Multipliers

Lagrange multipliers handle optimisation under constraints. The geometric insight is clean: at a constrained extremum, the gradient of the objective must be parallel to the gradient of the constraint (otherwise we could move along the constraint surface and improve the objective). The scalar λ that relates the two gradients

is the Lagrange multiplier; its value measures how much the optimal value would change if we relaxed the constraint slightly. This interpretation makes Lagrange multipliers central to economics (shadow prices) and physics (action principles).

Theorem: Lagrange Multiplier Theorem

Suppose $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ are continuously differentiable and a is a local extremum of f subject to the constraint $g(x) = c$, with $\nabla g(a) \neq \mathbf{0}$. Then there exists $\lambda \in \mathbb{R}$ (the **Lagrange multiplier**) such that $\nabla f(a) = \lambda \nabla g(a)$.

Proof. Near a , the constraint surface $g(x) = c$ is locally a smooth $(n-1)$ -dimensional manifold. Any curve $\mathbf{r}(t)$ on this surface through a satisfies $\nabla g(a) \cdot \mathbf{r}'(0) = 0$. At an extremum, $\nabla f(a) \cdot \mathbf{r}'(0) = 0$ for every such curve. Hence $\nabla f(a)$ is orthogonal to every vector in $T_a(\{g = c\})$, so it is parallel to $\nabla g(a)$. $\square$

Example 26.1. Maximise $f(x, y) = xy$ on the ellipse $x^2/4 + y^2 = 1$. Setting $\nabla f = \lambda \nabla g$: $(y, x) = \lambda(x/2, 2y)$. From $y = \lambda x/2$ and $x = 2\lambda y$: $x = 2\lambda \cdot \lambda x/2 = \lambda^2 x$, so $\lambda = \pm 1$. Taking $\lambda = 1$ gives $y = x/2$; substituting into the constraint, $x^2/4 + x^2/4 = 1$, so $x = \sqrt{2}$, $y = 1/\sqrt{2}$. The maximum is $f = \sqrt{2} \cdot \frac{1}{\sqrt{2}} = 1$, attained at $(\sqrt{2}, 1/\sqrt{2})$ (and at $(-\sqrt{2}, -1/\sqrt{2})$).

Exercises

1. Find all critical points and classify them for $f(x, y) = x^3 + y^3 - 3x - 3y$.
2. Use Lagrange multipliers to find the point on the plane $x + 2y + 3z = 6$ closest to the origin.
3. Prove that the maximum of $\sum x_i^2$ subject to $\sum x_i = 1$ and $x_i \geq 0$ is 1, attained at the vertices.
4. Show that the AM-GM inequality $\sqrt[n]{x_1 \cdots x_n} \leq (x_1 + \cdots + x_n)/n$ follows from maximising $\ln x_1 + \cdots + \ln x_n$ on $\sum x_i = n$.

Chapter 27

Multiple Integrals

27.1 Double Integrals and Iterated Integration

Integrating a function of two variables over a region D means summing up infinitely many infinitely thin volume slices. Fubini's theorem — the central result of this section — says we can compute this double integral by iterating two single-variable integrals, integrating in one variable at a time. The order does not matter for continuous functions. This is a deep theorem: it is not obvious that integrating y first then x gives the same answer as x first then y , and the theorem can fail for non-integrable functions.

Definition: Double Integral over a Rectangle

For $f : [a, b] \times [c, d] \rightarrow \mathbb{R}$ bounded, the **double integral** $\iint_R f \, dA$ is defined as the limit of double Riemann sums $\sum_{i,j} f(x_i^*, y_j^*) \Delta x_i \Delta y_j$ as the mesh of both partitions tends to zero.

Theorem: Fubini's Theorem

If $f : [a, b] \times [c, d] \rightarrow \mathbb{R}$ is continuous, then:

$$\iint_R f \, dA = \int_a^b \int_c^d f(x, y) \, dy \, dx = \int_c^d \int_a^b f(x, y) \, dx \, dy.$$

Both iterated integrals equal the double integral; in particular they are equal to each other.

Proof. Define $F(x) = \int_c^d f(x, y) \, dy$. By uniform continuity of f on the compact rectangle, F is continuous in x . Show that $\int_a^b F(x) \, dx$ equals the double Riemann sums in the limit using the one-variable integrability of $f(\cdot, y)$ for each y , then exchange the limits. $\square$

27.2 Change of Variables in $\mathbb{R}^n$

The change of variables theorem is the multidimensional counterpart of u -substitution. When we transform coordinates via $\mathbf{x} = \mathbf{G}(\mathbf{u})$, the volume element transforms by the absolute Jacobian determinant $|\det D\mathbf{G}|$. This determinant measures how much the transformation stretches or compresses volumes locally. Polar coordinates are the canonical example: the factor r in $dA = r dr d\theta$ accounts for the fact that rings at larger radius are wider for the same angular increment.

Theorem: Change of Variables (General)

Let $\mathbf{G} : U \rightarrow V$ be a C^1 bijection between open sets in $\mathbb{R}^n$, with Jacobian $J_{\mathbf{G}}(\mathbf{u}) = \det D\mathbf{G}(\mathbf{u}) \neq 0$. Then for any integrable $f : V \rightarrow \mathbb{R}$:

$$\int_V f(\mathbf{x}) d\mathbf{x} = \int_U f(\mathbf{G}(\mathbf{u})) |J_{\mathbf{G}}(\mathbf{u})| d\mathbf{u}.$$

Proof sketch. Partition U into small boxes. $\mathbf{G}$ maps each box approximately to a parallelotope whose volume is $|\det D\mathbf{G}| \cdot (\text{box volume})$ by the linear approximation. Summing and passing to the limit yields the formula. $\square$

The three classical coordinate systems are instances:

- **Polar** (r, θ) : $x = r \cos \theta$, $y = r \sin \theta$, Jacobian $= r$.
- **Cylindrical** (r, θ, z) : Jacobian $= r$.
- **Spherical** (ρ, ϕ, θ) : $x = \rho \sin \phi \cos \theta$, $y = \rho \sin \phi \sin \theta$, $z = \rho \cos \phi$, Jacobian $= \rho^2 \sin \phi$.

Example 27.1. $\iint_{x^2+y^2 \leq 1} e^{-(x^2+y^2)} dA$. In polar coordinates: $\int_0^{2\pi} \int_0^1 e^{-r^2} r dr d\theta = 2\pi \cdot \frac{1}{2}(1 - e^{-1}) = \pi(1 - e^{-1})$.

Exercises

1. Use Fubini's theorem to evaluate $\int_0^1 \int_0^1 (x+y)^2 dx dy$.
2. Compute the volume of the unit ball $B^3 = \{(x, y, z) : x^2 + y^2 + z^2 \leq 1\}$ using spherical coordinates.
3. Prove the Gaussian integral $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$ by computing $\left(\int_{-\infty}^{\infty} e^{-x^2} dx\right)^2$ in polar coordinates.
4. Show that Fubini's theorem can fail if f is not integrable (Tonelli's counterexample: $f(x, y) = (x^2 - y^2)/(x^2 + y^2)^2$ on $[0, 1]^2$).

Chapter 28

Vector Fields and Line Integrals

Definition: Vector Fields and Line Integrals

A **vector field** on $U \subseteq \mathbb{R}^n$ is a function $\mathbf{F} : U \rightarrow \mathbb{R}^n$.

For a smooth curve $\mathbf{r} : [a, b] \rightarrow \mathbb{R}^n$, the **line integral** of $\mathbf{F}$ along $\mathbf{r}$ is:

$$\int_C \mathbf{F} \cdot d\mathbf{r} := \int_a^b \mathbf{F}(\mathbf{r}(t)) \cdot \mathbf{r}'(t) dt.$$

This measures the total work done by $\mathbf{F}$ along the path.

Definition: Conservative Fields and Potential Functions

$\mathbf{F}$ is **conservative** if $\mathbf{F} = \nabla\phi$ for some scalar function ϕ (called the **potential**).

Theorem: Fundamental Theorem for Line Integrals

If $\mathbf{F} = \nabla\phi$ on U , then for any curve C from A to B in U :

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \phi(B) - \phi(A).$$

In particular, line integrals of conservative fields are path-independent.

Proof. $\int_a^b \nabla\phi(\mathbf{r}(t)) \cdot \mathbf{r}'(t) dt = \int_a^b \frac{d}{dt}\phi(\mathbf{r}(t)) dt = \phi(\mathbf{r}(b)) - \phi(\mathbf{r}(a))$ by the chain rule and FTC. $\square$

Theorem: Characterisation of Conservative Fields

On a simply connected domain, $\mathbf{F}$ is conservative iff $\nabla \times \mathbf{F} = \mathbf{0}$ (the curl vanishes).

Exercises

1. Compute $\int_C \mathbf{F} \cdot d\mathbf{r}$ for $\mathbf{F} = (y, x)$ along the parabola $y = x^2$ from $(0, 0)$ to $(1, 1)$.
2. Show $\mathbf{F} = (2xy + z^2, x^2, 2xz)$ is conservative and find a potential.
3. Prove: on a simply connected domain, $\oint_C \mathbf{F} \cdot d\mathbf{r} = 0$ for every closed curve iff $\mathbf{F}$ is conservative.
4. Compute the work done by gravity $\mathbf{F} = (0, 0, -mg)$ on a particle moving from $(0, 0, h_1)$ to $(0, 0, h_2)$ along any path.

Chapter 29

Surface Integrals

Definition: Parametric Surfaces and Surface Area

A **parametric surface** is a map $\mathbf{r} : D \subseteq \mathbb{R}^2 \rightarrow \mathbb{R}^3$. The **surface area element** is $d\mathbf{S} = (\mathbf{r}_u \times \mathbf{r}_v) du dv$. The **surface area** is $\iint_D |\mathbf{r}_u \times \mathbf{r}_v| du dv$. The **flux integral** (surface integral of a vector field) is:

$$\iint_S \mathbf{F} \cdot d\mathbf{S} = \iint_D \mathbf{F}(\mathbf{r}(u, v)) \cdot (\mathbf{r}_u \times \mathbf{r}_v) du dv.$$

Example 29.1. The flux of $\mathbf{F} = (0, 0, z)$ upward through the paraboloid $z = 1 - x^2 - y^2$ over $D = \{x^2 + y^2 \leq 1\}$. In polar: $\mathbf{r} = (r \cos \theta, r \sin \theta, 1 - r^2)$, $\mathbf{r}_u \times \mathbf{r}_v = (2r^2 \cos \theta, 2r^2 \sin \theta, r)$, $\mathbf{F} \cdot d\mathbf{S} = (1 - r^2)r dr d\theta$. Integrating: $\int_0^{2\pi} \int_0^1 (r - r^3) dr d\theta = 2\pi(1/2 - 1/4) = \pi/2$.

Exercises

1. Compute the surface area of the sphere $x^2 + y^2 + z^2 = R^2$.
2. Find the flux of $\mathbf{F} = (x, y, z)$ outward through the unit sphere.
3. Show the surface area element for a graph $z = f(x, y)$ is $\sqrt{1 + f_x^2 + f_y^2} dx dy$.

Chapter 30

The Three Great Theorems

Remark

Historical Note: Green, Stokes, Gauss, and the Theorems Named After Them

The three theorems of this chapter have a tangled history of credit and misattribution.

Green's theorem was published by George Green in 1828 in a privately printed pamphlet, *An Essay on the Application of Mathematical Analysis to the Theories of Electricity and Magnetism*, sold by subscription in Nottingham. Green was a self-taught miller's son with almost no formal education. He distributed roughly fifty copies; the pamphlet was essentially unknown until William Thomson (later Lord Kelvin) rediscovered it in 1845 and recognised its importance. By then Green had died (1841), having finally been admitted to Cambridge in 1833 at age 40. Gauss had proved the same result independently for three dimensions (the Divergence Theorem) around 1813, but his proof was also not widely known.

Stokes' theorem appears, in modern form, in a letter by Kelvin to Stokes in 1850. Stokes set it as a problem on the Smith's Prize examination at Cambridge in 1854 — an examination sat by Maxwell, among others. The theorem thus bears Stokes' name partly by examination accident. Earlier versions were known to Ampère and to Hankel, and the full proof in the modern sense was given by Brioschi in 1854.

The **Divergence Theorem** is variously attributed to Gauss, Green, Ostrogradsky (who published a proof in 1831 and is given credit in Russian textbooks), and Stokes. The pattern is typical of nineteenth-century mathematics: ideas were discovered repeatedly and independently, publications were slow and distribution limited, and national pride played a large role in determining who got credit in which country's textbooks.

The **unification** under the generalised Stokes' theorem on differential forms is due to **Cartan** (1899–1945), who developed the modern language of differential forms and exterior derivatives in the early twentieth century. Cartan's formulation shows that Green's, Stokes', the Divergence theorem, and the Fundamental Theorem of Calculus are all the same theorem $\int_M d\omega = \int_{\partial M} \omega$, differing only in dimension and the choice of differential form. This unification

was one of the great conceptual achievements of modern mathematics.

The three great theorems of vector calculus — Green’s, Stokes’, and the Divergence theorem — all say the same thing at different levels of generality: *an integral over a region equals an integral over its boundary*.

30.1 Green’s Theorem

Green’s theorem relates a line integral around a closed curve to a double integral over the enclosed region. It is the two-dimensional Fundamental Theorem of Calculus: the integral of a “derivative-like” combination $(Q_x - P_y)$ over a region equals the integral of (P, Q) around the boundary. The theorem has an immediate payoff: it lets us compute areas using line integrals and circulation using area integrals, switching representations as convenient.

Theorem: Green’s Theorem

Let C be a positively oriented, piecewise smooth, simple closed curve bounding a region D in $\mathbb{R}^2$. For continuously differentiable P, Q :

$$\oint_C (P dx + Q dy) = \iint_D \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \right) dA.$$

Proof. It suffices to prove $\oint_C P dx = -\iint_D P_y dA$ and $\oint_C Q dy = \iint_D Q_x dA$ separately. For a region $D = \{(x, y) : a \leq x \leq b, g_1(x) \leq y \leq g_2(x)\}$:

$$\iint_D P_y dA = \int_a^b [P(x, g_2(x)) - P(x, g_1(x))] dx = -\oint_C P dx,$$

where the last step parametrises the boundary in two pieces and applies FTC. $\square$

Corollary: Area via Line Integral

The area of region D with boundary C is $A = \frac{1}{2} \oint_C (x dy - y dx)$.

30.2 Stokes’ Theorem

Stokes’ theorem extends Green’s theorem from flat planar regions to curved surfaces in $\mathbb{R}^3$. The curl $\nabla \times \mathbf{F}$ measures the local rotation of a vector field; Stokes says the total rotation through a surface equals the circulation around its boundary curve. This is the mathematical core of Faraday’s law in electromagnetism and of the Kelvin–Stokes theorem in fluid mechanics.

Theorem: Stokes' Theorem

Let S be an oriented surface in $\mathbb{R}^3$ with positively oriented boundary curve ∂S . For a C^1 vector field $\mathbf{F}$:

$$\iint_S (\nabla \times \mathbf{F}) \cdot d\mathbf{S} = \oint_{\partial S} \mathbf{F} \cdot d\mathbf{r}.$$

Proof. Write $\mathbf{F} = (P, Q, R)$ and S as a parametric surface. Expand both sides in coordinates; after a careful calculation using Green's theorem on each coordinate patch, the two sides match. $\square$

Green's theorem is the special case where S is a planar region.

30.3 The Divergence Theorem

The Divergence theorem relates the outward flux through a closed surface to the total source strength (divergence) inside the enclosed volume. If $\mathbf{F}$ is a fluid velocity field, it says: the net flow out through the surface equals the total rate of fluid production inside. This is the mathematical statement of conservation laws, and it underlies Gauss's law in electrostatics as well as the continuity equations of fluid dynamics.

Theorem: Divergence Theorem (Gauss's Theorem)

Let E be a solid region in $\mathbb{R}^3$ with smooth closed boundary ∂E (oriented outward). For a C^1 vector field $\mathbf{F}$:

$$\iint_{\partial E} \mathbf{F} \cdot d\mathbf{S} = \iiint_E \nabla \cdot \mathbf{F} dV.$$

Proof. Prove the component identity $\iint_{\partial E} R dA_z = \iiint_E R_z dV$ (where dA_z is the z -component of $d\mathbf{S}$) by writing E as a region between two graphs and applying FTC. The full theorem follows by adding all three components. $\square$

30.4 Unification: Generalised Stokes' Theorem

All three theorems are special cases of one statement on differential forms:

Theorem: Generalised Stokes' Theorem

Let ω be a smooth $(n-1)$ -form on an oriented n -dimensional manifold M with boundary ∂M . Then:

$$\int_M d\omega = \int_{\partial M} \omega.$$

Here d is the exterior derivative. The dictionary is:

- Green's theorem: M is a 2D region, $\omega = P dx + Q dy$, $d\omega = (Q_x - P_y) dx \wedge dy$.
- Stokes' theorem: M is a surface in $\mathbb{R}^3$, $\omega = \mathbf{F} \cdot d\mathbf{r}$, $d\omega = (\nabla \times \mathbf{F}) \cdot d\mathbf{S}$.
- Divergence theorem: M is a 3D region, $\omega = \mathbf{F} \cdot d\mathbf{S}$, $d\omega = \nabla \cdot \mathbf{F} dV$.
- FTC: $M = [a, b]$, $\omega = f$, $d\omega = f' dx$, $\partial M = \{b\} - \{a\}$.

Exercises

1. Use Green's theorem to compute the area enclosed by the ellipse $x^2/a^2 + y^2/b^2 = 1$.
2. Use Stokes' theorem to evaluate $\oint_C (y^2 dx + z dy + x dz)$ where C is the unit circle in the plane $z = 0$.
3. Use the Divergence theorem to compute the flux of $\mathbf{F} = (x^3, y^3, z^3)$ outward through the unit sphere.
4. Show that $\nabla \cdot (\nabla \times \mathbf{F}) = 0$ and $\nabla \times (\nabla f) = \mathbf{0}$ algebraically, then interpret them via the generalised Stokes' theorem.
5. Derive the heat equation $u_t = \nabla^2 u$ from conservation of energy and the divergence theorem.

Part VII

The Capstone: *Symboliculus*

The previous six parts built the real numbers from scratch and then developed calculus over them, proving its core theorems. Now we build something with those theorems: **Symboliculus**, a command-line symbolic calculus solver in Lean 4 that computes derivatives and integrals with step-by-step traces — just like Symbolab, but with every rule traced back to the mathematics we proved. The capstone is once again plain core Lean — it computes over `Float` and needs no Mathlib to build or run; only the correctness theorems of Chapter 36 bring Mathlib back.

Chapter 31

Expression Trees

31.1 The `Expr` Inductive Type

A symbolic expression is a tree. Each node is either a leaf (a constant or variable) or an operator applied to one or two sub-expressions.

Lean 4

```
-- symbolculus/Expr.lean
namespace Symbolculus

inductive Expr where
| Const  : Float → Expr           -- numeric constant
| Var    : String → Expr         -- variable (e.g. "x")
| Add    : Expr → Expr → Expr
| Sub    : Expr → Expr → Expr
| Mul    : Expr → Expr → Expr
| Div    : Expr → Expr → Expr
| Pow    : Expr → Expr → Expr    -- e1 ^ e2
| Neg    : Expr → Expr
| Sin    : Expr → Expr
| Cos    : Expr → Expr
| Tan    : Expr → Expr
| Exp    : Expr → Expr          -- e^(...)
| Ln     : Expr → Expr
| Abs    : Expr → Expr
deriving Repr, BEq

end Symbolculus
```

31.2 Evaluation

Once we have the syntax tree, evaluation gives it meaning: given an assignment of numerical values to variables, `eval` traverses the tree and computes the corresponding `Float`. This is the **denotational semantics** of our expression language — the

same idea by which programming-language theory assigns mathematical meaning to programs, here applied to a concrete arithmetic language. Note that `eval` is total: every well-formed `Expr` has a value (though it may be `Float.nan` or `Float.inf` for divisions by zero).

Lean 4

```
-- Evaluate an expression given a variable assignment
def Expr.eval (env : String → Float) : Expr → Float
| Const c      ⇒ c
| Var x        ⇒ env x
| Add e1 e2    ⇒ e1.eval env + e2.eval env
| Sub e1 e2    ⇒ e1.eval env - e2.eval env
| Mul e1 e2    ⇒ e1.eval env * e2.eval env
| Div e1 e2    ⇒ e1.eval env / e2.eval env
| Pow e1 e2    ⇒ Float.pow (e1.eval env) (e2.eval env)
| Neg e        ⇒ -(e.eval env)
| Sin e        ⇒ Float.sin (e.eval env)
| Cos e        ⇒ Float.cos (e.eval env)
| Tan e        ⇒ Float.tan (e.eval env)
| Exp e        ⇒ Float.exp (e.eval env)
| Ln e        ⇒ Float.log (e.eval env)
| Abs e        ⇒ Float.abs (e.eval env)
```

31.3 Pretty-Printing

Pretty-printing is the inverse of parsing: it turns an `Expr` tree back into a human-readable string. The challenge is parenthesisation: we want $x + y \times z$ not $(x + y) \times z$, but we need parentheses where the tree structure would otherwise be ambiguous. The precedence function assigns a priority to each operator, and the `paren` helper wraps a subexpression in parentheses only when its operator binds less tightly than the surrounding context requires.

Lean 4

```
-- Pretty-print with minimal parentheses
def Expr.precedence : Expr → Nat
| Add _ _ | Sub _ _ ⇒ 1
| Mul _ _ | Div _ _ ⇒ 2
| Neg _      ⇒ 3
| Pow _ _    ⇒ 4
| _          ⇒ 10

def Expr.pp : Expr → String
| Const c      ⇒ if c == Float.floor c then
  toString c.toInt64 -- print 2.0 as "2"
  else toString c
| Var x        ⇒ x
```



```

| Add e1 e2  ⇒ s! "{e1.pp} + {e2.pp}"
| Sub e1 e2  ⇒ s! "{e1.pp} - {paren e2 2}"
| Mul e1 e2  ⇒ s! "{paren e1 2} * {paren e2 3}"
| Div e1 e2  ⇒ s! "{paren e1 2} / {paren e2 3}"
| Pow e1 e2  ⇒ s! "{paren e1 5}^{paren e2 5}"
| Neg e      ⇒ s! "-{paren e 4}"
| Sin e      ⇒ s! "sin({e.pp})"
| Cos e      ⇒ s! "cos({e.pp})"
| Tan e      ⇒ s! "tan({e.pp})"
| Exp e      ⇒ s! "exp({e.pp})"
| Ln e       ⇒ s! "ln({e.pp})"
| Abs e      ⇒ s! "|{e.pp}|"
where
  paren (e : Expr) (minPrec : Nat) :=
    if e.precedence < minPrec then s! "{e.pp}" else e.pp

```

31.4 Parsing

The parser turns a string like `"x^2 * sin(x)"` into an `Expr`. We implement a simple recursive-descent parser with operator precedence.

Lean 4

```

-- Tokeniser
inductive Token where
| TNum   : Float → Token
| TVar   : String → Token
| TPlus  | TMinus | TStar | TSlash | TCaret
| TLParen | TRParen | TEOF
deriving Repr, BEq

def tokenise (s : String) : List Token := sorry

-- Recursive-descent parser (Pratt / precedence climbing)
structure Parser where
  tokens : Array Token
  pos     : Nat

def Parser.peek (p : Parser) : Token :=
  p.tokens.getD p.pos Token.TEOF

def Parser.advance (p : Parser) : Parser :=
  { p with pos := p.pos + 1 }

-- parseExpr, parseTerm, parseFactor defined by mutual recursion
-- (full implementation in the Lake project)
def parseExpr : Parser → Except String (Expr × Parser) := sorry

```

```

def parseTerm : Parser → Except String (Expr × Parser) := sorry
def parseFactor: Parser → Except String (Expr × Parser) := sorry

def parseStr (s : String) : Except String Expr :=
  match parseExpr { tokens := (tokenise s).toArray, pos := 0 } with
  | .ok (e, _) ⇒ .ok e
  | .error msg ⇒ .error msg

```

31.5 Algebraic Simplification

Before and after differentiation or integration we run a simplification pass to collapse trivial expressions.

One Lean subtlety shapes the code: you cannot pattern-match on `Float literals`. A pattern like `Add e (Const 0)` is rejected, because `0 : Float` is not a constructor — it elaborates to a function call (`Float.ofScientific ...`), and match patterns must be built from constructors. So the identity rules match `Const c` and test `c == 0` with an ordinary `if` guard on the right-hand side.

Lean 4

```

def Expr.simplify : Expr → Expr
  -- Arithmetic on constants
  | Add (Const a) (Const b) ⇒ Const (a + b)
  | Sub (Const a) (Const b) ⇒ Const (a - b)
  | Mul (Const a) (Const b) ⇒ Const (a * b)
  | Div (Const a) (Const b) ⇒
    if b != 0 then Const (a / b) else Div (Const a) (Const b)
  -- Additive identity
  | Add e (Const c) ⇒
    if c == 0 then e.simplify else Add e.simplify (Const c)
  | Add (Const c) e ⇒
    if c == 0 then e.simplify else Add (Const c) e.simplify
  -- Multiplicative identity / zero
  | Mul e (Const c) ⇒
    if c == 1 then e.simplify
    else if c == 0 then Const 0
    else Mul e.simplify (Const c)
  | Mul (Const c) e ⇒
    if c == 1 then e.simplify
    else if c == 0 then Const 0
    else Mul (Const c) e.simplify
  -- Power rules
  | Pow e (Const c) ⇒
    if c == 1 then e.simplify
    else if c == 0 then Const 1
    else if e == Const 1 then Const 1
    else Pow e.simplify (Const c)

```

```

| Pow (Const c) e      ⇒
  if c == 1 then Const 1 else Pow (Const c) e.simplify
-- Double negation
| Neg (Neg e)          ⇒ e.simplify
-- Recurse
| Add e1 e2            ⇒ Add e1.simplify e2.simplify
| Sub e1 e2            ⇒ Sub e1.simplify e2.simplify
| Mul e1 e2            ⇒ Mul e1.simplify e2.simplify
| Div e1 e2            ⇒ Div e1.simplify e2.simplify
| Pow e1 e2            ⇒ Pow e1.simplify e2.simplify
| Neg e                ⇒ Neg e.simplify
| Sin e                ⇒ Sin e.simplify
| Cos e                ⇒ Cos e.simplify
| Tan e                ⇒ Tan e.simplify
| Exp e                ⇒ Exp e.simplify
| Ln e                 ⇒ Ln e.simplify
| Abs e                ⇒ Abs e.simplify
| e                    ⇒ e

-- Repeat until fixed point
def Expr.fullSimplify (e : Expr) (fuel : Nat := 20) : Expr :=
  match fuel with
  | 0      ⇒ e
  | n + 1 ⇒
    let e' := e.simplify
    if e' == e then e else e'.fullSimplify n

```

Exercises

1. Implement structural equality for `Expr` (two expressions are equal iff they have the same tree shape and constants).
2. Add `Arcsin`, `Arccos`, `Arctan` constructors and extend `eval` and `pp` accordingly.
3. Extend `fullSimplify` to handle $e^{\ln x} = x$ and $\ln(e^x) = x$.
4. Prove in Lean 4: for any `e : Expr` and environment `env`, `e.simplify.eval env = e.eval env`.

Chapter 32

The Differentiation Engine

32.1 The `DiffStep` Trace Type

Each call to the differentiation engine returns not just the result but a *proof trace* recording which rule was applied and to which subexpression.

Lean 4

```
-- symbolculus/Diff.lean
inductive DiffStep where
| ConstRule    : DiffStep          -- d/dx c = 0
| VarRule      : DiffStep          -- d/dx x = 1
| VarOther     : DiffStep          -- d/dx y = 0 (y !=
  x)
| SumRule      : DiffStep → DiffStep → DiffStep -- (u+v)' = u'+v'
| SubRule      : DiffStep → DiffStep → DiffStep
| ProductRule  : DiffStep → DiffStep → DiffStep -- (uv)' = u'v+uv'
| QuotRule     : DiffStep → DiffStep → DiffStep -- (u/v)' = (u'v-uv')/
  v^2
| ChainRule    : String  → DiffStep → DiffStep -- outer fn, inner
  deriv
| PowerRule    : DiffStep          -- d/dx x^n = nx^{n
  -1}
| NegRule      : DiffStep → DiffStep
deriving Repr
```

32.2 The `diff` Function

The differentiation engine is a structural recursion over `Expr`. Each constructor of `Expr` corresponds to exactly one differentiation rule from Part III, and the Lean pattern match makes this correspondence explicit. Crucially, each recursive call produces not just the derivative expression but also a `DiffStep` that records which rule was applied — this is what will become the step-by-step trace the user sees. After computing the derivative, we run `fullSimplify` to collapse trivial expressions before returning.

A syntax note: because `diff` is not declared inside the `Expr` namespace, the *patterns* use the dot-prefixed constructor names (`.Add`, `.Mul`, ...), which Lean resolves against the type being matched. Writing the bare names under an `open Expr` would collide with the arithmetic type classes `Add`, `Mul`, `Pow`, `Neg` that core Lean defines — Lean rejects such patterns as ambiguous. On the right-hand sides, where the expected type disambiguates, the bare names (via `open Expr`) are fine.

Lean 4

```
open Expr

-- diff x e returns (derivative, trace)
def diff (x : String) : Expr → Expr × DiffStep
  -- Constants
  | .Const _    ⇒ (Const 0, .ConstRule)

  -- Variables
  | .Var y      ⇒ if y == x
                  then (Const 1, .VarRule)
                  else (Const 0, .VarOther)

  -- Negation: (-u)' = -u'
  | .Neg u      ⇒
    let (u', s) := diff x u
    (Neg u', .NegRule s)

  -- Sum rule: (u + v)' = u' + v'
  | .Add u v    ⇒
    let (u', su) := diff x u
    let (v', sv) := diff x v
    (Add u' v' |>.fullSimplify, .SumRule su sv)

  -- Sub rule: (u - v)' = u' - v'
  | .Sub u v    ⇒
    let (u', su) := diff x u
    let (v', sv) := diff x v
    (Sub u' v' |>.fullSimplify, .SubRule su sv)

  -- Product rule: (uv)' = u'v + uv'
  | .Mul u v    ⇒
    let (u', su) := diff x u
    let (v', sv) := diff x v
    ((Add (Mul u' v) (Mul u v')).fullSimplify, .ProductRule su sv)

  -- Quotient rule: (u/v)' = (u'v - uv') / v^2
  | .Div u v    ⇒
    let (u', su) := diff x u
    let (v', sv) := diff x v
    ((Div (Sub (Mul u' v) (Mul u v')) (Pow v (Const 2))).fullSimplify,
```

```

    .QuotRule su sv)

-- Power rule: (u^n)' = n * u^(n-1) * u' (general, via chain rule;
-- note the trace is the LEAF .PowerRule -- the inner derivation su
-- is used for the derivative but not recorded in the trace)
| .Pow u (Const n) =>
    let (u', su) := diff x u
    ((Mul (Mul (Const n) (Pow u (Const (n-1)))) u').fullSimplify,
     .PowerRule)

-- Trig: sin(u)' = cos(u) * u'
| .Sin u =>
    let (u', su) := diff x u
    ((Mul (Cos u) u').fullSimplify, .ChainRule "sin" su)

-- Trig: cos(u)' = -sin(u) * u'
| .Cos u =>
    let (u', su) := diff x u
    ((Neg (Mul (Sin u) u')).fullSimplify, .ChainRule "cos" su)

-- Tan: (tan u)' = sec^2(u) * u' = u' / cos^2(u)
| .Tan u =>
    let (u', su) := diff x u
    ((Div u' (Pow (Cos u) (Const 2))).fullSimplify, .ChainRule "tan"
     su)

-- Exp: (e^u)' = e^u * u'
| .Exp u =>
    let (u', su) := diff x u
    ((Mul (Exp u) u').fullSimplify, .ChainRule "exp" su)

-- Ln: (ln u)' = u' / u
| .Ln u =>
    let (u', su) := diff x u
    ((Div u' u).fullSimplify, .ChainRule "ln" su)

-- General Pow: (u^v)' = u^v * (v'*ln(u) + v*u'/u)
| .Pow u v =>
    let (u', su) := diff x u
    let (v', sv) := diff x v
    let result := Mul (Pow u v)
                      (Add (Mul v' (Ln u)) (Div (Mul v u') u))
    (result.fullSimplify, .ChainRule "pow" (.ProductRule su sv))

| .Abs _ => (Const 0, .ConstRule) -- simplified; full version
    uses sign

```

32.3 Rendering the Trace

The `DiffStep` tree records the *structure* of the derivation. Rendering turns that structure into text the user can read. The renderer is also a structural recursion: each `DiffStep` constructor maps to a human-readable description of the rule used, optionally recursing into sub-derivations. This is the piece that makes Symbolicus feel like Symbolab — without the trace renderer, the user would just see the final answer with no explanation of how it was reached.

Lean 4

```
def DiffStep.render (orig result : Expr) : DiffStep → String
| .ConstRule    ⇒ s!"d/dx({orig.pp}) = 0  [constant rule]"
| .VarRule      ⇒ s!"d/dx({orig.pp}) = 1  [variable rule]"
| .VarOther     ⇒ s!"d/dx({orig.pp}) = 0  [independent variable]"
| .NegRule s    ⇒
  s!"Negate: d/dx(-u) = -(d/dx u)\n  " ++ s.render orig result
| .SumRule s t  ⇒
  s!"Sum rule: d/dx(u + v) = d/dx(u) + d/dx(v)"
| .ProductRule s t ⇒
  s!"Product rule: d/dx(u*v) = u'*v + u*v'"
| .QuotRule s t ⇒
  s!"Quotient rule: d/dx(u/v) = (u'*v - uv') / v^2"
| .ChainRule fn s ⇒
  s!"Chain rule on {fn}: d/dx({fn}(u)) = {fn}'(u) * d/dx(u)"
| .PowerRule    ⇒
  s!"Power rule: d/dx(u^n) = n * u^(n-1) * u'"
| _             ⇒ s!"[step applied to {orig.pp}]"
```

Exercises

1. Trace through `diff "x" (Mul (Var "x") (Sin (Var "x")))` by hand, showing the `DiffStep` tree produced and the rendered output.
2. Extend `diff` to handle `Arctan` with the rule $(\arctan u)' = u'/(1 + u^2)$.
3. Prove in Lean 4: for `diff "x" (Add u v)`, the result equals `Add (diff "x" u).1 (diff "x" v).1` (sum rule is structurally correct).
4. Add a `DiffStep.renderFull` that recursively renders the entire trace tree as a numbered list of steps.

Chapter 33

The Integration Engine

Integration is harder than differentiation: no single algorithm handles all cases, and some elementary functions have no elementary antiderivative (Risch's theorem, Chapter 36). We model the engine as a collection of *strategies*, each a function that either succeeds with a trace or fails, allowing the engine to try the next approach.

33.1 Strategies and the `IntStep` Trace

Integration is fundamentally different from differentiation: there is no single algorithm that handles all cases. Instead, we model the engine as a collection of *strategies*, each specialised to a class of integrands. A strategy is a function that either succeeds — returning an antiderivative and a trace — or fails, signalling that the next strategy should try. This is the classic strategy pattern from software engineering: the dispatcher runs down an ordered list and short-circuits on the first *some*, exactly what `List.findSome?` does.

Lean 4

```
-- symbolicalculus/Integrate.lean

inductive IntStep where
| DirectLookup : String → IntStep -- "integral of sin(x) is -cos(x)"
| ConstMul     : IntStep → IntStep -- pull out constant
| SumRule      : IntStep → IntStep → IntStep
| USub         : Expr → Expr → IntStep → IntStep
               -- u-substitution: u=..., du=...
| IBP          : Expr → Expr → IntStep → IntStep → IntStep
               -- integration by parts: u=..., dv=...
| PartialFrac  : List (Expr × Expr) → List IntStep → IntStep
               -- partial fractions: [(factor, coeff)]
| TrigSub      : String → Expr → IntStep → IntStep
               -- trig substitution: x = a*sin(t) etc.
| TrigReduce   : IntStep → IntStep -- trig power reduction
| Failed       : IntStep           -- no strategy worked
deriving Repr
```



```
-- A strategy is a function that maybe integrates e w.r.t. x
def Strategy := String → Expr → Option (Expr × IntStep)
```

33.2 Direct Lookup Table

The first strategy is the simplest: pattern-match the integrand against a table of known antiderivatives. This handles atomic expressions directly — constants, the identity, $\sin x$, $\cos x$, e^x , and power functions x^n for $n \neq -1$. When this strategy succeeds, the trace is a `DirectLookup` leaf with a human-readable description of the formula used. When it fails, the dispatcher moves on to more sophisticated strategies.

Lean 4

```
-- Antiderivatives of atomic expressions
def directLookup (x : String) : Expr → Option (Expr × IntStep)
| .Const c      ⇒
  some (Mul (Const c) (Var x), .DirectLookup s!"{c} dx = {c}x")
| .Var y        ⇒
  if y == x
  then some (Div (Pow (Var x) (Const 2)) (Const 2),
    .DirectLookup "∫x dx = x²/2")
  else none
| .Sin (Var y)  ⇒
  if y == x
  then some (Neg (Cos (Var x)), .DirectLookup "∫sin(x) dx = -cos(x)")
  else none
| .Cos (Var y)  ⇒
  if y == x
  then some (Sin (Var x), .DirectLookup "∫cos(x) dx = sin(x)")
  else none
| .Exp (Var y)  ⇒
  if y == x
  then some (Exp (Var x), .DirectLookup "∫e dx = e")
  else none
| .Pow (Var y) (Const n) ⇒
  if y == x && n != -1
  then some (Div (Pow (Var x) (Const (n+1))) (Const (n+1)),
    .DirectLookup s!"∫x^n dx = x^{n+1}/(n+1)")
  else none
| _            ⇒ none
```

33.3 U-Substitution Detector

The u -substitution strategy looks for an integrand of the form $f(g(x)) \cdot g'(x)$. It tries common choices for u and checks whether differentiating u (and dividing) simplifies the integrand. It needs three small syntactic helpers first: the list of variables occurring in an expression, and two substitution functions (rename a variable, and replace a variable by an expression). Each is the same mechanical structural recursion over the fourteen constructors. Note also `innerFunctions`: a `Pow` node contributes only its *base* as a candidate — a second alternative `Pow _ u` would be rejected by Lean as redundant, since the first pattern already matches every `Pow`.

Lean 4

```
-- Collect the innermost composite functions as u candidates
def innerFunctions : Expr → List Expr
| .Sin u | .Cos u | .Tan u | .Exp u | .Ln u ⇒ [u]
| .Pow u _ ⇒ [u]
| .Mul u v | .Add u v ⇒ innerFunctions u ++ innerFunctions v
| _ ⇒ []

-- All variables occurring in an expression
def varsOf : Expr → List String
| .Const _ ⇒ []
| .Var y ⇒ [y]
| .Add a b | .Sub a b | .Mul a b | .Div a b | .Pow a b ⇒
  varsOf a ++ varsOf b
| .Neg a | .Sin a | .Cos a | .Tan a | .Exp a | .Ln a | .Abs a ⇒
  varsOf a

-- Rename variable x to u throughout
def substVar (x u : String) : Expr → Expr
| .Const c ⇒ Const c
| .Var y ⇒ if y == x then Var u else Var y
| .Add a b ⇒ Add (substVar x u a) (substVar x u b)
| .Sub a b ⇒ Sub (substVar x u a) (substVar x u b)
| .Mul a b ⇒ Mul (substVar x u a) (substVar x u b)
| .Div a b ⇒ Div (substVar x u a) (substVar x u b)
| .Pow a b ⇒ Pow (substVar x u a) (substVar x u b)
| .Neg a ⇒ Neg (substVar x u a)
| .Sin a ⇒ Sin (substVar x u a)
| .Cos a ⇒ Cos (substVar x u a)
| .Tan a ⇒ Tan (substVar x u a)
| .Exp a ⇒ Exp (substVar x u a)
| .Ln a ⇒ Ln (substVar x u a)
| .Abs a ⇒ Abs (substVar x u a)

-- Replace variable u by the expression repl: identical shape to
-- substVar, with the Var case reading
-- | .Var y ⇒ if y == u then repl else Var y
```

```

def substExpr (u : String) (repl : Expr) : Expr → Expr
| .Const c    ⇒ Const c
| .Var y      ⇒ if y == u then repl else Var y
| .Add a b    ⇒ Add (substExpr u repl a) (substExpr u repl b)
| .Sub a b    ⇒ Sub (substExpr u repl a) (substExpr u repl b)
| .Mul a b    ⇒ Mul (substExpr u repl a) (substExpr u repl b)
| .Div a b    ⇒ Div (substExpr u repl a) (substExpr u repl b)
| .Pow a b    ⇒ Pow (substExpr u repl a) (substExpr u repl b)
| .Neg a      ⇒ Neg (substExpr u repl a)
| .Sin a      ⇒ Sin (substExpr u repl a)
| .Cos a      ⇒ Cos (substExpr u repl a)
| .Tan a      ⇒ Tan (substExpr u repl a)
| .Exp a      ⇒ Exp (substExpr u repl a)
| .Ln a       ⇒ Ln (substExpr u repl a)
| .Abs a      ⇒ Abs (substExpr u repl a)

-- "e is expressed in terms of u": every variable of e occurs in u
def isInTermsOf (u e : Expr) : Bool :=
  (varsOf e).all (fun v ⇒ (varsOf u).contains v)

-- Try u = inner function, check if integrand / du simplifies
def tryUSub (x : String) (e : Expr) : Option (Expr × IntStep) :=
  -- Collect candidate u-substitutions from the expression tree
  let candidates := innerFunctions e
  candidates.findSome? fun u ⇒
    let du := (diff x u).1 -- du/dx
    -- Check if e / (du/dx) depends only on u (not x directly)
    let reduced := (Div e du).fullSimplify
    if isInTermsOf u reduced then
      -- Integrate the reduced expression in terms of u
      match directLookup "u" (substVar x "u" reduced) with
      | some (antideriv, step) ⇒
        -- Back-substitute x for u
        some ((substExpr "u" u antideriv).fullSimplify,
              .USub u du step)
      | none ⇒ none
    else none

```

A frank engineering note: this detector is deliberately naive. Because `fullSimplify` only folds constants and identities — it cannot cancel a symbolic factor $g'(x)$ from a quotient — `tryUSub` fires only on the easiest shapes. Strengthening the simplifier until $\int 2x e^{x^2} dx$ goes through is one of this chapter's exercises, and a genuinely instructive one.

33.4 Integration by Parts Heuristic

The LIATE rule (Logarithm, Inverse trig, Algebraic, Trig, Exponential) guides which factor to assign to u and which to dv . Two implementation notes. First, our `Expr`

has no inverse-trig constructors (adding `Arctan` and `Arcsin`, with priority 1 here, is Exercise 2 of Chapter 32), so the printed table skips that tier. Second, the “algebraic” tier must include the bare variable: x is the polynomial x^1 , and if `Var` fell through to the default priority it would rank *below* `Exp` — the engine would then pick $u = e^x$, $dv = x dx$ for $\int x e^x dx$ and fail. (In an earlier draft it did exactly that. Heuristics deserve tests.)

Lean 4

```
-- LIATE priority for choosing u (lower = more eager to be u)
def liatePriority : Expr → Nat
| .Ln _           ⇒ 0 -- highest: logarithm
| .Pow (Var _) _   ⇒ 2 -- algebraic (polynomial x^n)
| .Var _          ⇒ 2 -- algebraic (bare x)
| .Sin _ | .Cos _  ⇒ 3 -- trigonometric
| .Exp _          ⇒ 4 -- exponential (lowest priority)
| _               ⇒ 5
```

The IBP strategy itself calls the top-level `integrate` on the remainder integral $\int v du$ — so `tryIBP` and `integrate` are *mutually recursive*, and the recursion is not structurally decreasing (the remainder is a freshly built expression, not a subterm). Lean therefore cannot verify termination, and we must declare the whole cluster inside a `mutual` block with `partial def`. This is an honest trade: `partial` functions can be compiled and run but are opaque to the proof system, and any claim that the engine terminates on all inputs is exactly that — a claim, unproven. For well-founded inputs it terminates in practice because each remainder is strictly simpler; making that precise (and provable) is a hard exercise.

Lean 4

```
mutual

partial def tryIBP (x : String) (e : Expr) : Option (Expr × IntStep) :=
  match e with
| .Mul u v ⇒
  -- Assign u and dv by LIATE priority
  let (uPart, dvPart) :=
    if liatePriority u ≤ liatePriority v
    then (u, v)
    else (v, u)
  let uPrime := (diff x uPart).1
  -- Try to integrate dvPart directly
  match directLookup x dvPart with
| some (vPart, _) ⇒
  -- IBP: ∫ u dv = uv - ∫ v du
  let remaining := (Mul vPart uPrime).fullSimplify
  match integrate x remaining with
| some (intRemainder, step) ⇒
  let result := Sub (Mul uPart vPart) intRemainder
```

```

        some (result.fullSimplify,
              .IBP uPart dvPart (.DirectLookup "") step)
      | none ⇒ none
    | none ⇒ none
  | _ ⇒ none

```

33.5 Top-Level `integrate` Dispatcher

The dispatcher tries each strategy in a fixed priority order and returns the first success. The order reflects the complexity hierarchy of integration techniques: we try the cheapest checks first (direct lookup, constant multiple, sum) before invoking the more expensive searches (substitution, parts, partial fractions). This greedy first-success approach is sound but not complete: some integrands require trying multiple strategies in combination, which a more sophisticated planner could handle.

Two of the strategies in the dispatch list — `partialFracStrategy` and `trigReduceStrategy` — are left for you to implement (Exercises 2 and 3 below). So that the engine compiles and runs meanwhile, we declare them as stubs that always decline; they sit *before* the `mutual` block, since they are not (yet) recursive. The remaining strategies continue the `mutual` block begun with `tryIBP` above — `sumStrategy` and `constMulStrategy` also call back into `integrate` — and the block closes with `end`.

Lean 4

```

-- Stubs for the two exercise strategies (place BEFORE the mutual
-- block): always decline, so the dispatcher moves on.
def partialFracStrategy : Strategy := fun _ _ => none
def trigReduceStrategy  : Strategy := fun _ _ => none

-- ... continuing the mutual block from tryIBP:

partial def integrate (x : String) (e : Expr) :
  Option (Expr × IntStep) :=
  let strategies : List Strategy :=
    [ directLookup,
      constMulStrategy,
      sumStrategy,
      tryUSub,
      tryIBP,
      partialFracStrategy,
      trigReduceStrategy ]
  strategies.findSome? (· x e)

-- Sum rule strategy
partial def sumStrategy (x : String) : Expr → Option (Expr × IntStep)
| .Add u v =>
  match integrate x u, integrate x v with
  | some (U, su), some (V, sv) =>

```

```

        some ((Add U V).fullSimplify, .SumRule su sv)
      | _, _ => none
    | _ => none

-- Constant multiple strategy
partial def constMulStrategy (x : String) :
  Expr → Option (Expr × IntStep)
  | .Mul (Const c) e =>
    match integrate x e with
    | some (E, s) => some ((Mul (Const c) E).fullSimplify, .ConstMul s)
    | none       => none
  | .Mul e (Const c) =>
    match integrate x e with
    | some (E, s) => some ((Mul (Const c) E).fullSimplify, .ConstMul s)
    | none       => none
  | _ => none

end -- of the mutual block

```

Exercises

1. Trace through `integrate "x" (Mul (Var "x") (Exp (Var "x")))` by hand, showing which strategy fires and the resulting `IntStep` tree.
2. Implement `partialFracStrategy` for the case of two distinct linear factors: $\int \frac{1}{(x-a)(x-b)} dx$ for $a \neq b$.
3. Implement `trigReduceStrategy` for $\int \sin^2(x) dx$ using the half-angle formula.
4. Extend `directLookup` with antiderivatives for $1/x$, $\tan x$, $\arctan x$, and $1/\sqrt{1-x^2}$.

Chapter 34

Step-by-Step Trace Rendering

The most important user-facing feature is the step-by-step trace. This is exactly what Symbolab renders: each computation step labelled with the rule used.

34.1 The `SolveTrace` Rose Tree

A `SolveTrace` is a rose tree: each node carries the name of the rule applied, the resulting expression, and a list of child traces for the sub-problems that were solved recursively. Leaf nodes represent atomic steps (a direct lookup, a base-case derivative); internal nodes represent compound rules (product rule, integration by parts) whose proofs depend on sub-derivations. The tree structure mirrors the proof tree of the derivation — rendering it is literally rendering a proof.

Lean 4

```
-- A rose tree of computation steps
inductive SolveTrace where
| Leaf    : String → Expr → SolveTrace      -- rule name, result
| Node    : String → Expr → List SolveTrace → SolveTrace

-- Convert DiffStep to SolveTrace. The match must be exhaustive:
-- forget the SubRule case and Lean rejects the definition with
-- "missing cases".
def DiffStep.toTrace (e result : Expr) : DiffStep → SolveTrace
| .ConstRule    ⇒
  .Leaf "Constant Rule" (Const 0)
| .VarRule       ⇒
  .Leaf "Variable Rule" (Const 1)
| .VarOther      ⇒
  .Leaf "Independent Variable" (Const 0)
| .SumRule s t   ⇒
  .Node "Sum Rule" result [s.toTrace e result, t.toTrace e result]
| .SubRule s t   ⇒
  .Node "Difference Rule" result
    [s.toTrace e result, t.toTrace e result]
| .ProductRule s t ⇒
```

```

      .Node "Product Rule" result [s.toTrace e result, t.toTrace e
result]
| .QuotRule s t =>
      .Node "Quotient Rule" result [s.toTrace e result, t.toTrace e
result]
| .ChainRule fn s =>
      .Node s!"Chain Rule ({fn})" result [s.toTrace e result]
| .PowerRule =>
      .Leaf "Power Rule" result
| .NegRule s =>
      .Node "Negation Rule" result [s.toTrace e result]

```

Observe an honest limitation before we render anything: `DiffStep` records which *rules* fired, but not the intermediate *results* — so `toTrace` has nothing better to thread into a compound child node than the top-level `result`. Compound sub-steps will therefore repeat the final expression rather than show their own sub-derivative, and `PowerRule`, being a leaf, shows no inner derivation at all. Upgrading `DiffStep` to carry the result of each sub-derivation (and `diff`'s `Pow` case to keep its inner trace) is Exercise 3 — the payoff is a trace that reads like Symbolab's.

34.2 Rendering to Text

The text renderer performs an in-order traversal of the `SolveTrace` tree, printing each node with indentation proportional to its depth. Each level of indentation corresponds to one level of sub-problem decomposition. The result is a hierarchical, numbered derivation — the same format Symbolab uses in its step-by-step display, where main steps appear at the top level and substeps are indented beneath them.

Lean 4

```

-- Render a SolveTrace as an indented list. "".pushn ' ' n builds
-- the n-space indent; List.flatMap concatenates the per-child
-- line lists.
def SolveTrace.renderLines (depth : Nat := 0) : SolveTrace → List String
| .Leaf rule result =>
  ["".pushn ' ' (depth * 2) ++ s!"• {rule}: {result.pp}"]
| .Node rule result children =>
  let header := "".pushn ' ' (depth * 2)
    ++ s!"▶ {rule} → {result.pp}"
  let childLines := children.flatMap (·.renderLines (depth + 1))
  header :: childLines

def SolveTrace.render (t : SolveTrace) : String :=
  "\n".intercalate t.renderLines

-- Full solve-and-render for differentiation
def solveDerivative (x : String) (e : Expr) : String :=
  let (result, step) := diff x e

```



```

let trace := step.toTrace e result
s!"d/dx({e.pp}) = {result.pp}\n\nSteps:\n{trace.render}"

-- Full solve-and-render for integration
def solveIntegral (x : String) (e : Expr) : String :=
  match integrate x e with
  | some (result, step) =>
    s!"∫({e.pp}) dx = {result.pp} + C\n\n(step trace available)"
  | none =>
    s!"∫({e.pp}) dx – no elementary antiderivative found"

```

Example 34.1 (Sample Output). Running `solveDerivative "x" (Mul (Var "x") (Sin (Var "x")))` produces:

$d/dx(x * \sin(x)) = \sin(x) + x * \cos(x)$

Steps:

- Product Rule $\rightarrow \sin(x) + x * \cos(x)$
 - Variable Rule: 1
- Chain Rule (`sin`) $\rightarrow \sin(x) + x * \cos(x)$
 - Variable Rule: 1

The derivative is right, and the rule skeleton is right — product rule at the root, with the variable rule and a chain-rule-on-sin beneath it. The blemish is the one flagged above: the Chain Rule line repeats the final answer instead of showing its own contribution $\cos(x)$, because `DiffStep` never recorded it. Exercise 3 fixes this.

Exercises

1. Implement `IntStep.toTrace` following the same pattern as `DiffStep.toTrace`.
2. Add an HTML renderer that wraps each step in `<div class="step">` tags for browser display.
3. Extend the trace to record the original sub-expression at each node, not just the rule and result.
4. Run `solveIntegral "x" (Mul (Var "x") (Exp (Var "x")))` and verify the output is $x * \exp(x) - \exp(x) + C$ — which equals $(x-1)e^x + C$, though the engine does not factor. Bonus: add a factoring rule to `simplify` so it does.

Chapter 35

The CLI

35.1 Lake Project Structure

Lake is Lean 4’s package manager and build system. A Lake project specifies its modules (libraries) and executables in a `lakefile.lean` (or `lakefile.toml`, as in the Mathlib setup of Chapter 6) at the project root. Our project has one library (`Symboliculus`, containing the expression, differentiation, integration, and trace modules) and one executable (`symboliculus`, the CLI entry point). Running `lake build` compiles everything and places all build artifacts under the `.lake/` directory — the binary lands at `.lake/build/bin/symboliculus`. Note that no Mathlib dependency is declared: the engine is plain core Lean.

Lean 4

```
-- lakefile.lean
import Lake
open Lake DSL

package symboliculus

lean_lib Symboliculus where
  srcDir := "Symboliculus"

@[default_target]
lean_exe symboliculus where
  root := `Main
```

Lean 4

```
-- Symboliculus/
-- Expr.lean      -- Expr type, eval, pp, simplify, parser
-- Diff.lean      -- diff, DiffStep, trace rendering
-- Integrate.lean  -- integrate, IntStep, strategies
-- Trace.lean     -- SolveTrace, render
-- Main.lean
```

35.2 Main.lean — Argument Parsing and Entry Point

The entry point parses command-line arguments, delegates to the appropriate engine (differentiation or integration), and prints the result. The `--steps` flag controls whether the full trace tree is rendered or just the final answer. Error handling covers both parse failures (the input string is not a valid expression) and unknown arguments. The design keeps `Main.lean` thin: all the mathematical logic lives in the library modules, and `main` is purely plumbing. One ordering constraint to notice: Lean processes a file top to bottom, so `Mode` must be declared *before* the `Args` structure that uses it.

Lean 4

```
-- Main.lean
import Symboliculus.Expr
import Symboliculus.Diff
import Symboliculus.Integrate
import Symboliculus.Trace

open Symboliculus

def usage : String :=
  "symboliculus [--diff | --integral] --expr <expr> --var <var> [--steps
  ]"

inductive Mode where | Diff | Integral

structure Args where
  mode  : Mode := .Diff
  expr  : String := ""
  var   : String := "x"
  steps : Bool  := false

def parseArgs : List String → Except String Args
| []      ⇒ .ok {}
| "--diff"  :: rest ⇒ do let a ← parseArgs rest; .ok { a with mode := .Diff }
| "--integral" :: rest ⇒ do let a ← parseArgs rest; .ok { a with mode := .Integral }
| "--expr"  :: e :: rest ⇒
  do let a ← parseArgs rest; .ok { a with expr := e }
| "--var"   :: v :: rest ⇒
  do let a ← parseArgs rest; .ok { a with var := v }
| "--steps" :: rest ⇒
  do let a ← parseArgs rest; .ok { a with steps := true }
| s :: _ ⇒ .error s!"Unknown argument: {s}\n{usage}"

def main (args : List String) : IO Unit := do
  match parseArgs args with
```

```

| .error msg => IO.eprintln msg
| .ok a =>
  if a.expr == "" then
    IO.eprintln s!"No expression given.\n{usage}"
  else
    match parseStr a.expr with
    | .error msg => IO.eprintln s!"Parse error: {msg}"
    | .ok e =>
      match a.mode with
      | .Diff =>
        let output := if a.steps
          then solveDerivative a.var e
          else
            let (result, _) := diff a.var e
            s!"d/d{a.var}({e.pp}) = {result.pp}"
        IO.println output
      | .Integral =>
        let output := if a.steps
          then solveIntegral a.var e
          else
            match integrate a.var e with
            | some (result, _) =>
              s!"∫({e.pp}) d{a.var} = {result.pp} + C"
            | none =>
              s!"∫({e.pp}) d{a.var} = (no elementary
form found)"
        IO.println output

```

35.3 Building and Running

Below are the canonical invocations for the two main use cases: differentiation with step trace, and integration. The outputs shown are the engine's *actual* output, so you can diff your build against them character for character. For the derivative trace, recall the Chapter 34 caveats: compound sub-steps repeat the final expression, and the power rule prints as a leaf (Exercise 3 of Chapter 34 upgrades both). For the integral, note the engine returns the unfactored $x * \exp(x) - \exp(x)$ — mathematically $(x - 1)e^x$ — and, since `IntStep.toTrace` is still Exercise 1 of Chapter 34, the `--steps` flag reports only that a trace is available. Try feeding in more complex expressions; the engine should handle any combination of the functions and rules implemented in the library.

Lean 4

```

-- Build the project
$ lake build

-- Differentiate x^2 * sin(x) with steps shown

```

```

$ ./lake/build/bin/symboliculus \
  --diff \
  --expr "x^2 * sin(x)" \
  --var x \
  --steps

-- Output:
-- d/dx(x^2 * sin(x)) = 2 * x * sin(x) + x^2 * cos(x)
--
-- Steps:
-- ► Product Rule → 2 * x * sin(x) + x^2 * cos(x)
--   • Power Rule: 2 * x * sin(x) + x^2 * cos(x)
--   ► Chain Rule (sin) → 2 * x * sin(x) + x^2 * cos(x)
--     • Variable Rule: 1

-- Integrate x * exp(x) with steps
$ ./lake/build/bin/symboliculus \
  --integral \
  --expr "x * exp(x)" \
  --var x \
  --steps

-- Output:
--  $\int (x * \exp(x)) \, dx = x * \exp(x) - \exp(x) + C$ 
--
-- (step trace available)

```

Exercises

1. Build the project and verify the two examples above run correctly.
2. Add a `--simplify` mode that just simplifies an expression without differentiating or integrating.
3. Add `--definite a b` flags so the CLI evaluates $\int_a^b f(x) \, dx$ using FTC Part 2 (find antiderivative, evaluate at endpoints, subtract).
4. Add a REPL mode (`--repl`) that reads expressions from stdin in a loop, displaying results interactively.

Chapter 36

Correctness, Incompleteness, and What Comes Next

36.1 What We Can Prove: Correctness of Differentiation

The differentiation engine *applies correct rules*: each case of `diff` transcribes a theorem from Part III. Can we turn that observation into a machine-checked theorem about the program? Mostly yes — but being precise about the “mostly” is the most instructive part of this section.

First obstacle: `Float` is not $\mathbb{R}$. The tempting statement is “`(diff x e).1.eval env` equals the derivative of `fun t => e.eval (env[x := t])`.” It is doubly broken. Formally, `eval` produces `Float`, and a derivative (Mathlib’s `deriv`, or our `HasDerivAt`) only makes sense over a normed field — which `Float` is not: it is a finite set of 2^{64} bit patterns with rounding arithmetic (not even associative: try `#eval (0.1 + 0.2) + 0.3 == 0.1 + (0.2 + 0.3)`). And substantively, the statement is *false* for `Float`: the sum rule $(u + v)' = u' + v'$ holds exactly over $\mathbb{R}$, but `Float` addition rounds, so no exact equality of this shape can survive. Correctness of numerical code is always a statement about the *ideal semantics*, with rounding handled separately (that is the subject of numerical analysis).

So we give `Expr` a second, real-valued semantics — reading an expression as a function $\mathbb{R} \rightarrow \mathbb{R}$ of the single variable `x` — and state correctness against *that*. The stored `Float` constants are read through a map `ι : Float → Real` (think “the exact rational this bit pattern denotes”), of which we require only `ι 0 = 0` and `ι 1 = 1`. This section uses Mathlib (`Real.sin`, `HasDerivAt`, and friends); it is the one place the capstone project needs it.

Lean 4

```
-- A real-valued semantics for Expr: interpret e as a function of
-- the single variable x. (noncomputable: it lives in Real.)
noncomputable def Expr.evalR ι ( : Float → Real) (x : String) :
  Expr → Real → Real
| .Const c    => fun _ => ι c
| .Var y      => fun t => if y == x then t else 0
```

```

| .Add u v  => fun t => u.evalR ι x t + v.evalR ι x t
| .Sub u v  => fun t => u.evalR ι x t - v.evalR ι x t
| .Mul u v  => fun t => u.evalR ι x t * v.evalR ι x t
| .Div u v  => fun t => u.evalR ι x t / v.evalR ι x t
| .Pow u v  => fun t => u.evalR ι x t ^ v.evalR ι x t  -- Real.rpow
| .Neg u    => fun t => -(u.evalR ι x t)
| .Sin u    => fun t => Real.sin (u.evalR ι x t)
| .Cos u    => fun t => Real.cos (u.evalR ι x t)
| .Tan u    => fun t => Real.tan (u.evalR ι x t)
| .Exp u    => fun t => Real.exp (u.evalR ι x t)
| .Ln u     => fun t => Real.log (u.evalR ι x t)
| .Abs u    => fun t => |u.evalR ι x t|
    
```

Theorem: Correctness of `diff` (rule-by-rule, over $\mathbb{R}$)

Read expressions through `evalR`. Then each rule `diff` applies is correct: whenever the sub-expressions are differentiable (with the derivatives claimed by the recursive calls), the expression built by the corresponding `diff` case has the claimed derivative. For the total fragment — `Const`, `Var`, `Add`, `Sub`, `Mul`, `Neg`, `Sin`, `Cos`, `Exp` — this holds unconditionally; for `Div`, `Pow`, `Ln`, and `Abs` it holds under the usual side conditions (nonvanishing denominator, positive base, etc.), which is why a single global equation cannot be stated without qualifications.

In Lean we prove the two base cases of `diff` itself, and the inductive steps as *rule-level* lemmas (shown here for `Add` and `Sin`; the remaining total-fragment cases follow the same two-line pattern and are the extended exercise below). Note how much Mathlib is doing for us: `hu.add hv` is the sum rule, and `hu.sin` is the chain rule for sine.

Lean 4

```

-- Base case: constants differentiate to 0.
theorem diff_correct_const ι ( : Float → Real) (ιh0 : ι 0 = 0)
  (x : String) (c : Float) (t : Real) :
  HasDerivAt ((Expr.Const c).evalR ι x)
    (((diff x (.Const c)).1).evalR ι x t) t := by
  simpa [diff, Expr.evalR, ιh0] using hasDerivAt_const t ι (c)

-- Base case: the variable differentiates to 1, other variables to 0.
theorem diff_correct_var ι ( : Float → Real) (ιh0 : ι 0 = 0)
  (ιh1 : ι 1 = 1) (x y : String) (t : Real) :
  HasDerivAt ((Expr.Var y).evalR ι x)
    (((diff x (.Var y)).1).evalR ι x t) t := by
  by_cases h : y == x
  · simpa [diff, h, Expr.evalR, ιh1] using hasDerivAt_id' t
  · simpa [diff, h, Expr.evalR, ιh0] using hasDerivAt_const t (0 : Real)

-- Inductive step for Add: the SUM RULE of Chapter 11, at the level
    
```

```

-- of expression trees (the rule diff's Add case applies, before its
-- final simplification pass).
theorem add_rule_correct ι( : Float → Real) (x : String)
  {u v u' v' : Expr} {t : Real}
  (hu : HasDerivAt (u.evalR ι x) (u'.evalR ι x t) t)
  (hv : HasDerivAt (v.evalR ι x) (v'.evalR ι x t) t) :
  HasDerivAt ((Expr.Add u v).evalR ι x)
    ((Expr.Add u' v').evalR ι x t) t :=
hu.add hv

-- Inductive step for Sin: the CHAIN RULE, ditto.
theorem sin_rule_correct ι( : Float → Real) (x : String)
  {u u' : Expr} {t : Real}
  (hu : HasDerivAt (u.evalR ι x) (u'.evalR ι x t) t) :
  HasDerivAt ((Expr.Sin u).evalR ι x)
    ((Expr.Mul (Expr.Cos u) u').evalR ι x t) t := by
simpa [Expr.evalR] using hu.sin

```

(If you prefer the statement in terms of Mathlib's `deriv` function: `HasDerivAt f L t` implies `deriv f t = L` — that is Mathlib's `HasDerivAt.deriv` — so each lemma above yields the corresponding `deriv` equation.)

Second obstacle: the simplifier folds constants in `Float`. Why lemmas per rule instead of one induction over `e`? Assembling the full theorem for the *shipped diff* needs one more ingredient: every compound case of `diff` finishes with `fullSimplify`, so we would need `simplify` to preserve `evalR`-semantics. And here honesty bites: it does not, exactly. `simplify` rewrites `Add (Const a) (Const b)` to `Const (a + b)` — *Float* addition, which rounds. Over the reals this rule is only valid if $\iota(a + b) = \iota a + \iota b$, and no map from a 64-bit type into $\mathbb{R}$ can make rounding arithmetic exact (the identity-style rules like $e + 0 \rightarrow e$ are fine; constant *folding* is not). This is not pedantry — it is precisely the gap between symbolic and floating-point computation, and formalisation forced us to notice it.

Extended Exercise: the full correctness theorem

1. Prove the remaining total-fragment rule lemmas (`Sub`, `Mul`, `Neg`, `Cos`, `Exp`) in the style of `add_rule_correct`, using Mathlib's `HasDerivAt.sub`, `.mul`, `.neg`, `.cos`, `.exp`.
2. Define a simplification-free `diffRaw` (drop the `fullSimplify` calls) and prove by structural induction, from your rule lemmas, that `diffRaw` is correct on the total fragment.
3. Restrict `simplify` to its semantics-preserving rules (drop the four constant-folding cases), prove it preserves `evalR`, and conclude correctness for the corresponding variant of `diff`.
4. Ponder (or prove!): for the *original* `simplify` the preservation statement is false. Exhibit concrete `Float` constants witnessing this.

36.2 What We Cannot Decide: The Risch Algorithm

Remark

Historical Note: The Undecidability of Integration

The question of whether a given elementary function has an elementary antiderivative is one of the deepest algorithmic questions in mathematics.

Liouville proved in 1835 that e^{-x^2} , $\sin(x)/x$, and $\sqrt{1-k^2\sin^2 x}$ have no elementary antiderivatives — no antiderivative expressible in terms of polynomials, exponentials, logarithms, and trigonometric functions via composition and arithmetic. His proof used an algebraic structure on *differential fields* that constrains the form any antiderivative must take, then derives contradictions when no such form exists. This was among the first examples of a natural mathematical problem shown to be “impossible” within a given class of functions.

Robert Risch solved the general case in 1969–1970 with a complete algorithm: given any elementary function, the Risch algorithm either finds an elementary antiderivative or proves none exists. One fine print matters, though. The algorithm repeatedly needs to decide whether certain expressions are identically zero, and by **Richardson’s theorem** (1968) zero-equivalence for a rich enough class of closed-form expressions (involving π , $\ln 2$, e^x , $\sin x$, $|x|$, ...) is *undecidable* in general. So Risch’s procedure is an algorithm *modulo a zero-equivalence oracle*: fully decidable on well-behaved differential-field towers (for instance, purely transcendental ones with constant field $\mathbb{Q}$), and heuristic at the ragged edges where constant recognition itself is unsolvable. In practice, a full implementation requires handling dozens of cases across algebraic, logarithmic, and exponential extensions. The most complete implementations, in systems such as Mathematica, Maple, and Axiom, run to tens of thousands of lines — and even they implement the algebraic case only partially.

Symbolab and the pedagogical approach. Symbolab — which inspired this capstone — does not implement Risch. Instead it pattern-matches against the techniques taught in calculus courses: substitution, integration by parts, partial fractions, trigonometric substitution. It displays the matching technique as a “step” in human-readable form. This is less complete than Risch but far more pedagogically useful: the steps shown are the ones a student would write. Symboliculus follows the same philosophy. Our correctness theorem guarantees that everything we *do* return is correct; it says nothing about what we fail to find.

Unlike differentiation, integration is fundamentally harder.

Theorem: Risch’s Theorem (1969)

There is a procedure (the **Risch algorithm**) that, given an elementary function f presented over a suitable differential field, either:

1. Computes an elementary antiderivative F with $F' = f$, or

2. Proves that no elementary antiderivative exists.

The procedure is a genuine decision procedure *relative to an oracle for deciding whether expressions are identically zero*; by Richardson’s theorem that sub-problem is undecidable for general closed-form expressions, so “symbolic integration is decidable” must always carry this asterisk.

Remark

The Risch algorithm is *extremely* complex to implement fully — the most complete implementations (as in Mathematica or Maple) run to tens of thousands of lines of code and handle dozens of special cases, and none is complete for the algebraic case. Our engine covers the cases taught in a calculus course (polynomial, trig, exponential, integration by parts, u -sub, partial fractions) but not the general algorithm. Examples of elementary functions with no elementary antiderivative: e^{-x^2} , $\sin(x)/x$, $\sqrt{1 - k^2 \sin^2 x}$ (elliptic integral).

36.3 Connections to Neighbouring Fields

Symboliculus is not an isolated project — every major component opens a door to a larger subject worth exploring on its own:

- **Expr and programming-language theory.** The `Expr` inductive type is a *term algebra* in the sense of the lambda calculus and universal algebra: constructors are introduction rules, pattern matching is elimination, and `eval` is a denotational semantics. Adding binders, capture-avoiding substitution, and rewrite rules turns it into a full term rewriting system — the gateway to PL theory.
- **Strategy dispatch.** The integration engine’s strategy chain — try `directLookup`, fall back to `tryUSub`, then `tryIBP`, etc. — is ordered, first-match-wins dispatch over `Option`, short-circuiting on the first `some`. Serious computer algebra systems generalise this to backtracking search over a space of rewriting tactics — the same design space as proof search in Lean itself.
- **Correctness proofs and differential Galois theory.** The field of rational functions over which the Risch algorithm operates is a *differential field*. Whether an elementary antiderivative exists is answered by Liouville’s theorem and, more deeply, by Galois theory applied to differential equations (Picard–Vessiot theory) — a beautiful subject if you have met Galois theory for polynomials.
- **Numerical integration.** When the symbolic engine returns `none`, real systems fall back to numerical integration (Gaussian quadrature, Simpson’s rule) — numerical analysis, the other half of the Float discussion from the previous section.

- **Optimisation and the Hessian.** The multivariable optimisation chapter (Ch. 26) is the calculus behind machine-learning optimisation: gradient descent follows $-\nabla f$, and the Hessian controls its convergence speed.

36.4 Open Problems and Extensions

1. **Implement the full Risch algorithm.** Start with the “transcendental case” (integrands involving e^x and $\ln x$ only); the algebraic case (involving $\sqrt{\cdot}$) is substantially harder.
2. **Prove correctness of `integrate`.** The correctness theorem would state: if `integrate x e = some (F, _)`, then `diff x F = e` (up to simplification). This requires a more careful treatment of `fullSimplify` preserving semantic equality.
3. **Add multivariate support.** Extend `diff` to handle partial derivatives ($\partial f / \partial x_i$) and then `integrate` to handle iterated integrals.
4. **Bridge `MyReal` to Mathlib’s `Real`.** Part I’s `MyReal` and Mathlib’s `Real` arise from the same Cauchy-sequence construction. Make that precise: build the field isomorphism `MyReal ≈ Real` and use it to transport a Part I theorem across. Then extend Section 36.1’s rule lemmas into the full `diff-computes-HasDerivAt` bridge.
5. **Step-counting complexity.** How many integration strategies does the engine try before giving up? Analyse the worst-case and average-case complexity for polynomial, rational, and trigonometric families.

Exercises

1. Work the Extended Exercise of Section 36.1: prove the remaining rule-correctness lemmas (`Sub`, `Mul`, `Neg`, `Cos`, `Exp`), then tackle the side-condition cases (`Div`, `Ln`, `Pow`) with appropriate hypotheses.
2. Implement the simplest non-trivial case of the Risch algorithm: the “rational function” case, which decides when $\int p(x)/q(x) dx$ is elementary (always) and computes it via partial fractions.
3. Add a “verify” mode to the CLI: given an alleged antiderivative F and integrand f , differentiate F and check symbolically whether $F' = f$.
4. Write a test suite (`Tests.lean`) with 20 derivative and 20 integral examples, running `diff` and `integrate` and checking the results match known values using `Expr.eval` at sample points.

Appendix A

A Lean 4 Primer

This appendix is the promised zero-prerequisites tour of every Lean 4 construct and tactic used in this book. Each entry says what the construct does, shows a small self-contained example (every example here is machine-checked), and — where it illuminates — shows the lower-level term the tactic is sugar for. That last part is worth your attention: Lean tactics are not magic, they are *programs that build proof terms*, and seeing the terms once demystifies them permanently.

Entries marked **(Mathlib)** require `import Mathlib` (see the setup instructions in Section 6.1); everything else is core Lean.

A.1 Declarations

`inductive` — defining new types

An `inductive` declaration creates a brand-new type from a list of *constructors* — the only ways to make values of the type. The kernel automatically knows that constructors are injective and disjoint, and generates an induction principle (Section A.2).

Lean 4

```
inductive Tree where
  | leaf : Tree
  | node : Tree → Tree → Tree
deriving Repr -- auto-generate printing, so #eval can show Trees
```

`deriving Repr` asks Lean to synthesise a printer; `deriving BEq` (used by the capstone's `Expr`) synthesises the boolean equality test `==`.

`def` and pattern matching

A `def` introduces a function or value. Functions on inductive types are usually written by *pattern matching*: one equation per constructor. Recursion is allowed as long as some argument structurally shrinks (Lean checks this; when it cannot, you must write `partial def`, as the integration engine of Chapter 33 does, and you give up the ability to reason about the function in proofs).

Lean 4

```
def Tree.size : Tree → Nat
| .leaf      ⇒ 1
| .node l r ⇒ l.size + r.size
```

The leading dot in `.leaf` means “the constructor of the type being matched” — shorthand for `Tree.leaf`. Because the function is named `Tree.size`, the *dot notation* `t.size` works for any `t : Tree`.

theorem, example, and Prop

In Lean, propositions are types (of sort `Prop`) and proofs are programs: to prove P is to construct a term of type P . A `theorem` is a named proof; an `example` is an anonymous one (used throughout this appendix). The body can be a direct *term* or a *tactic block* introduced by `by`, in which tactics manipulate the current *goal* until none remain.

Lean 4

```
example (p q : Prop) (hp : p) (h : p → q) : q := h hp -- term style

example (p q : Prop) : p → (p → q) → q := by          -- tactic style
  intro hp h
  apply h
  exact hp
```

Type classes and instance

A *type class* is an interface: `Add α` says “ α has a `+`”. An `instance` registers an implementation, and Lean finds it automatically whenever `+` is used — this is how `MyNat` gets `+` in Chapter 1 and how `MyNat.le` becomes `≤`.

Lean 4

```
structure Complex where
  re : Float
  im : Float
  deriving Repr

instance : Add Complex where
  add a b := ⟨a.re + b.re, a.im + b.im⟩

#eval (Complex.mk 1 2) + (Complex.mk 3 4)
-- { re := 4.000000, im := 6.000000 }
```

The angle brackets `⟨...⟩` are the *anonymous constructor*: Lean infers which constructor to apply from the expected type. They also build proofs of $\exists$ and $\wedge$ — `⟨k, hk⟩` proves $\exists k, P k$ by supplying the witness and the proof.

@[simp] — the simplifier’s rule set

The attribute @[simp] marks an equation as a rewrite rule for the simp tactic, which rewrites left-to-right until no rule applies. Marking a *definition* @[simp] (as Chapter 1 does with MyNat.add) makes simp unfold it.

Lean 4

```
@[simp] theorem Tree.size_node (l r : Tree) :
  (Tree.node l r).size = l.size + r.size := rfl

example (t : Tree) : (Tree.node t .leaf).size = t.size + 1 := by
  simp [Tree.size]
```

A.2 Core Tactics

rfl — true by computation

Proves any goal of the form $a = a$ after both sides compute to the same normal form. Definitional unfolding is free: $2 + 3 = 5$ is rfl because the kernel can evaluate it.

Lean 4

```
example : 2 + 3 = 5 := rfl
```

intro — peel off hypotheses

If the goal is $p \rightarrow q$ (or $\forall x, P x$), intro hp moves the antecedent into the context as hypothesis hp, leaving goal q. It is the tactic form of writing `fun hp => ...`.

exact and apply — supply the proof

exact h closes the goal with the term h, which must match exactly. apply h works backwards: if $h : p \rightarrow q$ and the goal is q, then apply h replaces the goal by p. (See the intro/apply/exact example above — the term-style proof h hp is what the three tactics assemble.)

rw — rewrite with an equation

rw [h], for $h : a = b$, replaces occurrences of a in the goal by b (use rw [← h] for the other direction, and rw [h] at h2 to rewrite inside a hypothesis).

Lean 4

```
example (a b : Nat) (h : a = b) : a + a = b + b := by rw [h]
```

simp — the simplifier

Rewrites with every @[simp] rule (plus any extras you pass in brackets) until nothing changes, then tries to close the goal. `simp [foo]` adds `foo` to the rule set; `simp at h` simplifies a hypothesis; `simp using h` simplifies both the goal and `h`, then closes the goal with it. `simp only [...]` uses *only* the listed rules — more predictable, common in this book.

Lean 4

```
example (n : Nat) : (n + 0) * 1 = n := by simp
```

induction ...with — structural induction

`induction n with` splits the goal into one case per constructor, giving the step case an induction hypothesis. Under the hood there is no primitive “induction”: every `inductive` type `T` comes with a *recursor* `T.rec`, a function stating exactly the induction principle, and the tactic merely applies it. Seeing this once is worth a hundred words:

Lean 4

```
theorem zero_add_nat (n : Nat) : 0 + n = n := by
  induction n with
  | zero      => rfl
  | succ n ih => rw [Nat.add_succ, ih]

-- ...and its desugaring: applying the recursor directly. (The
-- `show` re-states the goal in beta-reduced form so `rw` can see it.)
theorem zero_add_nat' (n : Nat) : 0 + n = n :=
  Nat.rec (motive := fun n => 0 + n = n)
    rfl -- base case
    (fun n ih => by
      show 0 + (n + 1) = n + 1
      rw [Nat.add_succ, ih]) -- step case
  n
```

The `motive` is the property being proved, as a function of `n` — what a paper proof calls “ $P(n)$ ”. Chapter 1’s `mynat_ind` is precisely `MyNat`’s recursor restricted to `Prop`.

cases — case analysis without induction

Like `induction` but with no induction hypothesis: just split on the constructors. Works on data (`cases k with | zero => ... | succ k => ...`) and on logical connectives, since `Or`, `And`, `Exists` are themselves inductive types:

Lean 4

```
example (p q : Prop) (h : p ∨ q) : q ∨ p := by
  cases h with
  | inl hp ⇒ exact Or.inr hp
  | inr hq ⇒ exact Or.inl hq
```

obtain / rcases — destructure in one step

`obtain ⟨n, hn⟩ := h` takes $h : \exists n, P\ n$ apart into a witness n and a proof hn ; `rcases` is the same with nested patterns (`⟨n, hgt, hlt⟩` flattens a $\wedge$ inside an $\exists$) and alternation (`h1 | h2` for $\vee$). The desugaring is a `match` — or, fully explicitly, the eliminator `Exists.elim`:

Lean 4

```
example (h : ∃ n : Nat, n > 5) : ∃ n : Nat, n > 3 := by
  obtain ⟨n, ⟩hn := h
  exact ⟨n, by ⟩omega

-- ...and the desugaring: a match on the anonymous constructor
example (h : ∃ n : Nat, n > 5) : ∃ n : Nat, n > 3 :=
  match h with
  | ⟨n, ⟩hn ⇒ ⟨n, by ⟩omega

-- ...or, fully explicit, the eliminator:
example (h : ∃ n : Nat, n > 5) : ∃ n : Nat, n > 3 :=
  Exists.elim h (fun n hn ⇒ ⟨n, by ⟩omega)

-- rcases with a nested pattern
example (h : ∃ n : Nat, n > 5 ∧ n < 10) : ∃ n : Nat, n < 10 := by
  rcases h with ⟨n, hgt, ⟩hlt
  exact ⟨n, ⟩hlt
```

(Why can a `match` take an $\exists$ apart? Because `Exists` is an ordinary inductive type with one constructor `Exists.intro`, so matching on it is just `cases`.)

constructor — build the goal's constructor

If the goal's head is an inductive with one applicable constructor ($\wedge, \leftrightarrow$, a structure), `constructor` applies it, turning e.g. a goal $p \wedge q$ into two goals p and q . The `·` bullets then focus one subgoal each.

Lean 4

```
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp
  · exact hq
```


calc — chained (in)equalities

A `calc` block strings together steps $a \leq b := \text{proof}$, each line picking up where the previous ended. Mixed relations are allowed when transitivity makes sense ($=$ with $\leq$, $\leq$ with $<$, ...). The desugaring: each adjacent pair is glued by the `Trans` type class's `Trans.trans`:

Lean 4

```
example (a b c : Nat) (h1 : a ≤ b) (h2 : b ≤ c) : a ≤ c + 1 := by
  calc a ≤ b      := h1
      _ ≤ c      := h2
      _ ≤ c + 1 := Nat.le_succ c

-- ...and its desugaring: chained Trans.trans applications
example (a b c : Nat) (h1 : a ≤ b) (h2 : b ≤ c) : a ≤ c + 1 :=
  Trans.trans (Trans.trans h1 h2) (Nat.le_succ c)
```

have, show, and suffices — structuring proofs

`have h : P := proof` adds an intermediate fact to the context. `show P` re-states the current goal (useful to force an unfolding, or just for readability). `suffices h : P by tac` argues top-down: “it suffices to prove P , because `tac` derives the goal from it”; the remaining goal is then P .

Lean 4

```
example (a b : Nat) (h : a = b) : b = a := by
  have h2 : a = b := h
  show b = a
  exact h2.symm

example (n : Nat) (h : n ≥ 10) : n > 3 := by
  suffices hs : n > 5 by omega
  omega
```

omega — linear integer arithmetic, decided

A decision procedure for linear arithmetic over `Nat` and `Int` (with $+$, constants multiples, $\leq$, $<$, $=$, `min`/`max`, $\%$ by constants). If the goal follows from the linear facts in context, `omega` closes it — no hints needed. It knows nothing about multiplication of variables or about `Real`.

Lean 4

```
example (a b : Nat) (h : a + 2 * b = 10) (hb : b ≥ 3) : a ≤ 4 := by
  omega
```

decide — run the decision procedure

For goals whose truth is *decidable by computation* (a `Decidable` instance exists), `decide` simply evaluates it and checks the answer is `true`. Works for concrete arithmetic and for quantifiers over finite types; useless for infinite quantifiers or `Real`.

Lean 4

```
example : 17 * 3 = 51 := by decide
example : ∀ n : Fin 5, n.val < 10 := by decide
```

grind — core Lean’s general closer

A core-Lean automation tactic combining congruence closure with arithmetic; in Part I we use it for routine `Rat` algebra (where `omega` does not apply and Mathlib’s `linarith` is not yet available).

Lean 4

```
example (a b : Rat) (h : a < b) : a - b < 0 := by grind
```

funext — functions equal pointwise are equal

Not a tactic but a lemma (usable with `apply`): to prove $f = g$ for functions, prove $\forall x, f\ x = g\ x$. Chapter 5’s `addReal` uses it to compare two lifted functions.

Lean 4

```
example (f g : Nat → Nat) (h : ∀ n, f n = g n) : f = g := funext h
```

A.3 Quotients: `Quot`, `Quot.mk`, `Quot.lift`, `Quot.sound`

Given a relation $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$, the type `Quot r` is “ α with r -related elements glued together.” Four primitives run the whole show:

- `Quot.mk r a` — the class of a (write $[a]$ on paper);
- `Quot.sound : r a b → Quot.mk r a = Quot.mk r b` — related representatives give *equal* classes;
- `Quot.lift f h` — descend $f : \alpha \rightarrow \beta$ to `Quot r → β`, *provided* the proof h that f respects r (well-definedness!);
- `Quot.ind` — to prove something for every class, prove it for every `Quot.mk r a`.

A complete miniature (integers-mod-2 as a quotient of `Nat`):

Lean 4

```
def parityRel (m n : Nat) : Prop := m % 2 = n % 2
```

```

def Parity : Type := Quot parityRel

def Parity.ofNat (n : Nat) : Parity := Quot.mk parityRel n

-- Descend a function to the quotient: needs a respect proof.
def Parity.isEven : Parity → Bool :=
  Quot.lift (fun n ⇒ n % 2 == 0)
    (fun a b h ⇒ by simp [parityRel] at h; simp [h])

-- Equate two representatives: Quot.sound turns relatedness into
-- equality of equivalence classes.
theorem parity_two_zero : Parity.ofNat 2 = Parity.ofNat 0 :=
  Quot.sound (by simp [parityRel])

#eval (Parity.ofNat 6).isEven -- true

```

This is exactly the machinery behind `MyInt` (Chapter 2) and `MyReal` (Chapter 5); there, the “respect” proofs are the well-definedness arguments, and binary operations need a nested `Quot.lift` with one respect proof per argument.

A.4 Mathlib Tactics (Parts II–VII)

All entries below require `import Mathlib`.

`linarith` — linear arithmetic over ordered fields (Mathlib)

The workhorse of our ε - δ proofs: closes any goal that is a linear consequence of linear hypotheses over `Real` (or `Rat`, `Int`, ...). Extra facts can be passed inline: `linarith [abs_nonneg x]`.

Lean 4

```

example (x y : Real) (h1 : x < 2 * y) (h2 : y ≤ 3) : x < 6 := by
  linarith

```

`nlinarith` — a nonlinear boost (Mathlib)

`linarith` after multiplying hypotheses together and adding squares; the bracket hints tell it which products to consider. Heuristic, not complete — when it fails, feed it the key square.

Lean 4

```

example (a : Real) : 0 ≤ a ^ 2 - 2 * a + 1 := by
  nlinarith [sq_nonneg (a - 1)]

```

norm_num — numeric normalisation (Mathlib)

Evaluates concrete numeric expressions in any ordered field or ring — like `decide`, but for types such as `Real` where nothing literally computes, plus extensions (primality, ...).

Lean 4

```
example : (2 : Real) + 2 ≠ 5 := by norm_num
example : (127 : Nat).Prime := by norm_num
```

positivity — positivity by structure (Mathlib)

Proves goals $0 < e$, $0 \leq e$, or $e \neq 0$ by recursing over the syntax of `e` (products of positives are positive, ...). Ubiquitous for our “ $\delta > 0$ ” side-goals.

Lean 4

```
example (x : Real) (hx : 0 < x) : 0 < x ^ 2 / (x + 1) := by positivity
```

push_neg — normalise negations (Mathlib)

Pushes `¬` inward through quantifiers and connectives: $\neg \forall$ becomes $\exists \neg$, $\neg(a \leq b)$ becomes $b < a$, and so on — exactly the “negate the definition of convergence” step of analysis. (Recent Mathlib versions are renaming this to `push Not`; `push_neg` still works, possibly with a deprecation notice.)

Lean 4

```
example (P : Nat → Prop) (h : ¬ ∀ n, P n) : ∃ n, ¬ P n := by
  push_neg at h
  exact h
```

by_contra — proof by contradiction (Mathlib)

`by_contra hx` assumes the negation of the goal as `hx` and asks you to derive `False`. It usually travels with `push_neg at hx`. The desugaring is the classical axiom-powered lemma `Classical.byContradiction : (¬p → False) → p`:

Lean 4

```
example (x : Real) (h : ∀ eps : Real, eps > 0 → x ≤ eps) : x ≤ 0 := by
  by_contra hx
  push_neg at hx
  have := h (x / 2) (by linarith)
  linarith

-- ...which is sugar for Classical.byContradiction:
```

```
example (p : Prop) (h : ¬p) : p :=
  Classical.byContradiction fun hnp => h hnp
```

ring and ring_nf — commutative-ring identities (Mathlib)

`ring` proves any identity valid in every commutative ring by normalising both sides to a canonical polynomial form; `ring_nf` performs the same normalisation without requiring it to close the goal (we use it to tidy expressions inside `|·|` before applying the triangle inequality).

Lean 4

```
example (a b : Real) : (a + b) ^ 2 = a ^ 2 + 2 * a * b + b ^ 2 := by
  ring
```

set — name a subexpression (Mathlib)

`set s := e with hs` abbreviates a recurring expression: it replaces every occurrence of `e` by a new local `s` and records `hs : s = e`. Purely organisational, but it keeps ε - δ proofs readable.

Lean 4

```
example (a b : Real) (h : a + b > 2) : (a + b) ^ 2 > 4 := by
  set s := a + b with hs
  nlinarith
```

field_simp — clear denominators (Mathlib)

Multiplies out fractions, using nonzeroness hypotheses from the context, typically leaving a polynomial goal for `ring` or `linarith`.

Lean 4

```
example (x : Real) (hx : x ≠ 0) : 1 / x + 1 = (1 + x) / x := by
  field_simp
```

by_cases and unfold

Two small players seen in Chapter 36: `by_cases h : p` splits the proof into the case where `p` holds and the case where it fails (classically, any proposition may be split; for decidable `p` it is constructive). `unfold foo` replaces `foo` by its definition in the goal — a blunter, more predictable cousin of `simp only [foo]`.

That is the complete toolbox. If a proof in the book uses something not listed here, it is a named lemma, not a tactic — and Lean’s `exact?`, or a search on https://leanprover-community.github.io/mathlib4_docs, will find it for you.